

AutoLISP / Visual LISP Language Specification

HyperSpec-Style Draft Reference for clautolisp

Codex

June 24, 2026

Contents

1 1 Introduction

1.1 Overview

This document is a HyperSpec-style reference specification for the AutoLISP / Visual LISP language family targeted by ‘clautolisp’.

The short name of this document is **AutoLISP Spec**.

The current document version is **0.9.2**.

Any modification of this document must increment its version number.

This document is intended to be licensed under **CC-BY-SA**.

The external source documents cited, summarized, or discussed in this specification remain under their own copyright and license terms unless explicitly stated otherwise by their respective owners.

It is intended to be usable as a local reference document, not merely as a planning memo. It therefore combines:

- a chapter structure modeled on the Common Lisp HyperSpec,
- chapter-level normative text,
- dictionary-style entries for defined names and syntax classes,
- explicit source notes for each specification element.

Because Autodesk and Bricsys do not publish a single unified standard, this document distinguishes documented, inferred, under-specified, tested, and implementation-defined material.

1.2 Scope

This specification covers:

- core AutoLISP syntax and evaluation,
- Visual LISP language extensions,
- host-visible runtime types,
- CAD interaction surfaces that are language-defining in practice,
- version and vendor divergence that must be tracked by ‘clautolisp’.

This specification does not attempt to reproduce full external standards for:

- DXF object schemas,
- COM type libraries,
- .NET, ObjectARX, or BRX APIs,
- every interactive CAD command.

1.3 Conformance Model

‘clautolisp’ should distinguish at least:

- a core AutoLISP profile,
- an AutoCAD-oriented profile,
- a BricsCAD-oriented profile,
- strict and lax acceptance modes,
- selectable host-environment profiles independent from the native OS.

When an AutoLISP or Visual LISP function family is specified as a wrapper over an underlying host API that is documented elsewhere, the underlying API documentation is part of the normative reference corpus for that family, subject to the calling conventions and restrictions documented by the AutoLISP layer.

This applies in particular to:

- ‘vla-*‘ wrappers over the AutoCAD ActiveX object model,
- ‘vlax-*‘ property, method, variant, safearray, and object-creation functions that expose COM / OLE Automation semantics.

It does **not** follow that every Windows-only or Visual LISP extension function is merely a generic foreign-function binding; some families, such as ‘vlr-*‘ reactors, remain primarily specified by Autodesk’s own AutoLISP / ActiveX documentation.

1.4 Source Notes

- Primary chapter model adapted from the Common Lisp HyperSpec chapter index: [CLHS Chapter Index](<https://www.lispworks.com/documentation/HyperSpec/Front/Contents.htm>)
- AutoLISP language history: [AutoLISP and Visual LISP](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP/files/GUID-49AAEA0E-C422-48C4-87F0-52FCA491BF2C.htm>)
- BricsCAD developer overview: [BricsCAD Developer Reference Overview](https://developer.bricsys.com/bricscad/help/en_US/V24/DevRef/source/DevRef_Overview.htm)
- Status: chapter structure adapted by design; language scope documented and inferred.

1.5 Brief History

The historical genealogy of AutoLISP / Visual LISP shapes several otherwise-arbitrary corners of the language. The chapter-level normative rules — Lisp-1 semantics, dynamic scope, silent-NIL on unbound reference, no tail-call optimization, no multiple values, upper-case folding — are downstream consequences of the choices made by the implementations enumerated below.

- **XLISP (David Betz, 1980s).** AutoLISP was derived from a slimmed XLISP. XLISP itself targeted personal computers as a compact, embeddable Lisp dialect with a single-cell symbol model and dynamic scope. AutoLISP inherited that core.
- **AutoLISP shipped in AutoCAD 2.18, January 1986.** Released as Autodesk’s API for customising AutoCAD. Successive AutoCAD releases extended it through to Release 13 (February 1995).
- **Vital LISP (Basis Software).** A third-party superset of AutoLISP adding an IDE, debugger, compiler, ActiveX (COM) bindings, reactors, and additional general-purpose Lisp functions.
- **Visual LISP (Autodesk, 1997).** Autodesk acquired Vital LISP and rebranded it. Initially shipped as an add-on to AutoCAD R14.
- **Merger into AutoCAD 2000 (March 1999).** Visual LISP replaced the standalone AutoLISP runtime and IDE inside AutoCAD. From this point onward, “AutoLISP” and “Visual LISP” name the same runtime; the documentation distinguishes “core” AutoLISP from the Visual LISP extensions (`vl-*`, `vla-*`, `vla-x-*`, `vlr-*`, etc.).
- **BricsCAD AutoLISP.** Bricsys’s BricsCAD product implements AutoLISP and large parts of Visual LISP from scratch as a vendor-independent compatibility layer. BricsCAD V25 introduced lax-mode tokenizer and `atof` extensions; V26 (this work’s reference target) preserves them. Bricsys maintains its own bug tracker and release notes; defects such as `SR44723` are normative-grade evidence for behaviours that Autodesk’s reference is silent on.
- **clautolisp (this project).** A Common-Lisp-based reimplementa-tion of the AutoLISP runtime designed to match the Lisp-1, dynamically-scoped, host-product-conformant semantics documented in this specification.

The genealogy explains, among other things, why `lambda` produces a function value but does not lexically capture (XLISP roots), why integer division truncates (C-influenced numerics from PC-era XLISP), and why `values` does not exist (the XLISP core never had it).

1.5.1 Source Notes

- [AutoLISP — Wikipedia](<https://en.wikipedia.org/wiki/AutoLISP>)
- [History of AutoLISP — Ron Leigh](<http://ronleigh.com/autolisp/ahistory.htm>)
- [AutoLISP and Visual LISP (Autodesk)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP/files/GUID-49AAEA0E-C422-48C4-87F0-52FCA>htm)
- BricsCAD release notes (per-version, Bricsys help center).

2 2 Syntax

2.1 Overview

AutoLISP uses parenthesized Lisp syntax. Vendor documentation specifies the syntax mostly through examples, concept pages, and individual function documentation rather than through a single formal grammar.

2.2 Definitions

An AutoLISP source contains expressions. Expression classes include:

- symbols,
- integers,
- reals,
- strings,
- lists,
- dotted pairs,
- quoted expressions,
- calls,
- special forms.

2.3 Reader Syntax Entry: Integer Literal

2.3.1 Name

Integer Literal

2.3.2 Class

Reader Syntax

2.3.3 Concrete Syntax

```
integer ::= sign? digit+  
sign    ::= "+" | "-"  
digit   ::= "0" | "1" | ... | "9"
```

2.3.4 Constituents

- optional sign
- one or more decimal digits

2.3.5 Description

An integer literal denotes a signed 32-bit integer value when its magnitude lies in the documented integer range.

2.3.6 Acceptance Rules

- documented examples include ‘2’ and ‘-56’
- no decimal point appears
- literals beyond the maximum integer range may be read as reals instead of integers

2.3.7 Strict/Lax Policy

- strict: accept only the conservative grammar above
- lax: may accept additional host-confirmed integer token forms if tests justify them

2.3.8 Examples

- ‘0‘
- ‘12‘
- ‘-42‘
- ‘+7‘

2.3.9 Compatibility

- Autodesk documents the range and prose semantics
- exact lexical grammar is inferred, not fully formalized

2.3.10 Notes

Some older or host-specific readers may accept additional token spellings, but those are not standardized here.

2.3.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

2.3.12 Source Notes

- [About Integers (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/FRA/AutoCAD-MAC-AutoLisp/files/GUID-EF6114FC-F1E4-4C71-91CC-07D01E6C8ABB.htm>)
- Status: documented in prose, formal syntax inferred.

2.4 Reader Syntax Entry: Real Literal

2.4.1 Name

Real Literal

2.4.2 Class

Reader Syntax

2.4.3 Concrete Syntax

```
real ::= sign? digit+ "." digit* exponent?  
      | sign? digit+ exponent  
exponent ::= ("e" | "E") sign? digit+
```

2.4.4 Constituents

- optional sign
- decimal digits
- decimal point and/or scientific exponent

2.4.5 Description

A real literal denotes a double-precision floating-point number.

2.4.6 Acceptance Rules

- documented examples include ‘3.1’, ‘0.23’, ‘-56.123’, ‘4.1e-6’
- numbers between ‘-1’ and ‘1’ must contain a leading zero according to Autodesk prose

2.4.7 Strict/Lax Policy

- strict: reject ambiguous forms such as ‘.5’ and ‘1.’ until product tests confirm them
- lax: may accept host-confirmed forms beyond the conservative grammar

2.4.8 Examples

- ‘1.0’
- ‘0.5’
- ‘-3.25’
- ‘4.1e-6’

2.4.9 Compatibility

- AutoCAD 2021+ Unicode changes do not directly alter numeric tokens, but reader changes should still be version-qualified

2.4.10 Notes

The published documentation leaves ‘.5’, ‘1.’, and some exponent variants under-specified.

2.4.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

2.4.12 Source Notes

- [About Reals (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP/files/GUID-F2BFD097-295B-4499-BBD9-0A785C377070.htm>)
- Status: partially documented, partially inferred.

2.5 Reader Syntax Entry: String Literal

2.5.1 Name

String Literal

2.5.2 Class

Reader Syntax

2.5.3 Concrete Syntax

```
string ::= "\"" string-char* "\""
```

2.5.4 Constituents

- opening double quote
- zero or more string characters
- closing double quote

2.5.5 Description

A string literal denotes a string object.

2.5.6 Acceptance Rules

- a string is delimited by double quotes
- backslash introduces escape/control sequences

2.5.7 Strict/Lax Policy

- strict: accept only documented or tested escapes
- lax: accept additional escapes only when they do not conflict with documented behavior and compatibility requires them

2.5.8 Examples

- “”abc””
- “”A string with spaces””

2.5.9 Compatibility

- string interpretation changed materially with Autodesk Unicode/LISPSYS changes beginning in 2021
- BricsCAD also documents Unicode-aware file text handling

2.5.10 Notes

The complete escape repertoire remains under-specified in the current source corpus.

2.5.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

2.5.12 Source Notes

- [About Strings (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ESP/AutoCAD-AutoLISP/files/GUID-D7C33A49-9715-4E9B-9D8A-4C2E7BFC0FE5.htm>)
- [LISPSYS](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-Core/files/GUID-1853092D-6E6D-4A06-8956-AD2C3DF203A3.htm>)
- [read + write text files (BricsCAD)](https://developer.bricsys.com/bricscad/help/en_US/V25/DevRef/source/readwritetextfiles.htm)
- Status: documented in principle, incomplete in detail.

2.6 Reader Syntax Entry: Symbol

2.6.1 Name

Symbol Token

2.6.2 Class

Reader Syntax

2.6.3 Concrete Syntax

A symbol token is any token not otherwise read as a number or syntax delimiter, subject to the character restrictions below.

2.6.4 Constituents

- one or more non-delimiter characters

2.6.5 Description

AutoLISP symbol names are case-insensitive. Autodesk documents that they may contain alphanumeric and notation characters, except at least:

- ‘(‘
- ‘)’
- ‘.’
- ‘”‘
- ‘”’
- ‘,’

A symbol name cannot consist solely of numeric characters.

2.6.6 Acceptance Rules

- blanks, newlines, tabs, and parentheses are documented token delimiters for ‘read’

2.6.7 Strict/Lax Policy

- strict: reject tokens that fall outside documented delimiter and reserved-character rules
- lax: may accept host-confirmed punctuation or edge cases when needed for compatibility

2.6.8 Examples

- ‘FOO‘
- ‘bar‘
- ‘c:my-command‘

2.6.9 Compatibility

- print convention is uppercase in Autodesk examples
- exact escaping and extended symbol syntax remain under-specified

2.6.10 Notes

This specification does not standardize CL-style package syntax or vertical-bar escaping.

2.6.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

2.6.12 Source Notes

- [About Symbols and Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP/files/GUID-1855E7B0-2474-49E6-9EE3-8BC54...htm>)
- [read (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/CHS/AutoCAD-MAC-AutoLISP-Reference/files/GUID-5B50BB3E-C244-46E8-85D8-6A2D48B1FE51...htm>)

- Status: partially documented.

2.7 Reader Syntax Entry: List

2.7.1 Name

List

2.7.2 Class

Reader Syntax

2.7.3 Concrete Syntax

```
list ::= "(" element* ")"
```

2.7.4 Description

A list is a sequence of zero or more expressions enclosed in parentheses.

2.7.5 Acceptance Rules

- elements are separated by delimiters and nested syntax

2.7.6 Strict/Lax Policy

- strict and lax modes agree for ordinary list syntax

2.7.7 Examples

- ‘()’
- ‘(A B C)’
- ‘((A B) C)’

2.7.8 Compatibility

Stable across vendors.

2.7.9 Notes

The empty list is semantically ‘nil’.

2.7.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

2.7.11 Source Notes

- [About Lists (AutoLISP)](<https://help.autodesk.com/cloudhelp/2016/ENU/AutoCAD-AutoLISP/files/GUID-F8074AA9-F361-4960-B628-681465B74229.htm>)
- Status: documented.

2.8 Reader Syntax Entry: Dotted Pair

2.8.1 Name

Dotted Pair

2.8.2 Class

Reader Syntax

2.8.3 Concrete Syntax

```
dotted-pair ::= "(" expression "." expression ")"
```

2.8.4 Description

A dotted pair is a two-part cons-style representation written with a dot separator.

2.8.5 Acceptance Rules

- used heavily in association lists and entity data

2.8.6 Strict/Lax Policy

- strict and lax modes agree for ordinary dotted-pair syntax

2.8.7 Examples

- ‘(A . 2)’
- ‘(8 . "WALLS)’

2.8.8 Compatibility

Language-central across AutoCAD and BricsCAD.

2.8.9 Notes

Dotted pairs are semantically more important in AutoLISP than in many other Lisp environments because of CAD data conventions.

2.8.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

2.8.11 Source Notes

- [About Dotted Pairs (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/PLK/AutoCAD-AutoLISP/files/GUID-877B87BA-6FA2-4894-9F94-184E7DF25B7D.htm>)
- Status: documented.

2.9 Reader Syntax Entry: Quote

2.9.1 Name

Quote Syntax

2.9.2 Class

Reader Syntax

2.9.3 Concrete Syntax

`quoted-expression ::= "'" expression`

2.9.4 Description

Leading apostrophe is shorthand for ‘quote’.

2.9.5 Acceptance Rules

- the following expression is not evaluated

2.9.6 Strict/Lax Policy

- strict and lax modes agree

2.9.7 Examples

- ‘A’
- ‘(1 2 3)’

2.9.8 Compatibility

Stable across vendors.

2.9.9 Notes

This specification does not standardize backquote/comma syntax yet.

2.9.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

2.9.11 Source Notes

- [quote (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-18F7E287-CB2F-4150-9A07-CE23C3F9E604.htm>)
- Status: documented.

2.10 Other Reader Syntax

The following remain under-specified and are therefore strict/lax configuration points:

- backquote
- comma
- comma-at
- vertical-bar symbol escaping
- sharpsign dispatch
- package-like syntax

2.11 Source Notes

- Current vendor corpus sampled in ‘autolisp-spec/documentation/autolisp-reference-notes.org’
- Status: under-specified pending further research and testing.

3 3 Evaluation and Execution

3.1 Overview

AutoLISP evaluation is Lisp-like but not identical to Common Lisp evaluation. This chapter defines the general evaluation model needed by the rest of the specification.

3.2 Normative Rules

- numbers and strings self-evaluate
- ‘nil’ and ‘T’ are predefined values
- symbols evaluate to their bound values
- ordinary calls evaluate arguments before invocation unless the operator is a special form
- ‘nil’ is false and every non-‘nil’ value is true in conditional contexts

Evaluation order is historically version-sensitive in error-reporting contexts. Autodesk explicitly notes that modern AutoLISP evaluates all arguments before checking argument types in at least some cases, unlike older releases.

3.3 Normative Rules: Argument Evaluation Order

3.3.1 Statement

For every ordinary call (`operator e1 e2 e3 ...`) — i.e. every call whose operator is **not** a special form — the argument expressions `e1`, `e2`, `e3`, ... are evaluated **strictly left-to-right**, with each argument’s side effects visible to the next.

Special forms (`setq`, `if`, `cond`, `and`, `or`, `while`, `repeat`, `foreach`, `lambda`, `function`, `defun`, `defun-q`, `progn`, `quote`, `command`, `trace`, `untrace`, `vlax-for`) decide their own evaluation order; their per-entry rules override this default.

3.3.2 Vendor Evidence

BricsCAD V26 / macOS direct probe: a side-effecting `note` function records each argument’s invocation order. `(+ (note 1) (note 2) (note 3))`, `(list (note 1) (note 2) (note 3))`, and an immediately-applied lambda `((lambda (a b c) ...) (note 1) (note 2) (note 3))` all yield the recorded sequence `(1 2 3)`. The order is identical for builtin operators, ordinary user functions, and lambda calls.

3.3.3 Cross-references

- Compare **CLHS 3.1.2.1.2.3** — Common Lisp specifies left-to-right argument evaluation.
- Compare **R7RS 4.1.3** — Scheme specifies that argument evaluation order is unspecified, only that each is fully evaluated before any procedure is applied. AutoLISP picks the stricter contract.

3.3.4 Source Notes

Phase-6 BricsCAD V26 / macOS general-semantics probe — `results/bricscad/macos/20260426T*-s` section A.

3.4 Normative Rules: Variable Scope (Dynamic, Not Lexical)

3.4.1 Statement

AutoLISP variables are **dynamically scoped**. A binding established at function call entry — through a positional parameter, through a (`/ locals`) declaration, or through any subsequent `setq` on those names — is visible to every function called transitively from the body. The binding is restored to its prior value when the function returns. There is no lexical capture: a `lambda` does **not** close over the names referenced by its body. When the lambda is later called, every free reference is resolved against the **active** dynamic-frame chain, then the active namespace.

3.4.2 Concrete Consequences

- (`defun make-adder (/ x) (setq x 10) (lambda (y) (+ x y)))` — calling (`make-adder`) then later applying the returned lambda outside `make-adder`'s extent fails: `x` has no binding any more, so the body's reference to `x` resolves to `nil` (see "Unbound-Variable Reference" below) and the `(+ ... y)` call surfaces **bad argument type <NIL>; expected <NUMBER> at [+]**.
- A counter built with (`defun with-counter (/ k) (setq k 0) (lambda () (setq k (+ k 1)) k)`) does **not** accumulate state across calls; each call from outside `with-counter` sees `k` unbound (and thus `nil`).
- A lambda referencing a **global** (top-level) symbol picks up that global's **current** value at every call, not the value at lambda creation. Mutating the global between two calls of the same lambda changes what subsequent calls return.

3.4.3 Vendor Evidence

BricsCAD V26 / macOS direct probe (section C):

- Captured-local closure call: error **bad argument type <NIL>; expected <NUMBER> at [+]** — the local was popped on return; the lambda saw `nil`.
- Top-level closure: (`c2`) first returns 7 (set inside `maker2`), then 42 after a top-level (`setq x 42`).

- Local counter: every call from outside the maker fails with the same NIL-at-[+] error — no accumulation.

3.4.4 Strict / Lax Policy

Dynamic scoping is **strict** across all supported dialects. No host product is known to expose lexical-capture semantics in its AutoLISP layer; any such extension would require a separate dialect knob.

3.4.5 Cross-references

- Compare **CLHS 3.1.1** — Common Lisp is lexical by default; dynamic scope requires ‘defvar’ / ‘(declare (special ...))’.
- Compare **R7RS 5.2** — Scheme is unconditionally lexical.
- Special Form Entry: LAMBDA, FUNCTION (this chapter) — produce function values; do not capture.
- Single-Cell Symbol Binding (chapter 7) — the dynamic frame chain consults the same per-symbol binding cell.

3.4.6 Source Notes

- [About Local and Global Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP/files/GUID-A8045F62-1CE3-4022-ACC1-CC0E2.htm>) — describes shadowing pragmatically.
- Phase-6 BricsCAD V26 / macOS semantics probe section C.

3.5 Normative Rules: Unbound-Variable Reference

3.5.1 Statement

Evaluating a symbol with no active dynamic binding and no namespace binding yields `nil` rather than signalling an error. Referencing a never-set name is therefore not directly observable; the typical surface symptom is a downstream type error when `nil` is passed to an arithmetic or other type-checking primitive.

This contrasts with both Common Lisp (`unbound-variable` is a condition) and R7RS Scheme (the result is an error). The silent-NIL behaviour is **strict** across every supported AutoLISP host: there is no language-level way for portable AutoLISP source to distinguish "bound to nil" from "never bound" at reference time, and `boundp` — the only predicate exposed for that question — additionally **creates** the symbol and binds it to `nil` as a side effect of being asked (chapter 7, `BOUNDp` entry). Programmer-visible state therefore collapses every never-set name into a `nil`-bound name on first observation.

The consequence: there is no compliant AutoLISP dialect in which a bare reference to an unset symbol can signal an error without breaking deployed code. Strict mode in this specification means **common to every conforming AutoLISP runtime**, not **the most pedantic**; silent-NIL is therefore the strict rule.

3.5.2 Concrete Consequences

- `(setq y (+ never-set-symbol 1))` does not signal **unbound variable**; it signals **bad argument type <NIL>; expected <NUMBER>** at `[+]`.
- `(boundp 'never-set)` returns `nil` but additionally creates the symbol and binds it to `nil` (chapter 7, `BOUNDp`). After this call `never-set` is observably `nil`-bound.
- A program cannot reliably detect "never assigned" once any code path has either referenced or `boundp`-tested the name.

3.5.3 Vendor Evidence

BricsCAD V26 / macOS direct probe section C — every closure test that referenced an out-of-extent local surfaced its error **at the consuming primitive** (`('+)`), not at the variable reference. The error message wrapped the

value ‘<NIL>’, which is the expected payload after the silent-NIL evaluation.

3.5.4 Strict / Lax Policy

Silent-NIL on unbound reference is **strict** across the **strict**, **autocad-2026**, and **bricscad-v26** dialects. **clautolisp** exposes a non-conforming diagnostic knob, **unbound-variable-mode :strict-error**, that programs can opt into through a custom-built dialect descriptor; doing so produces source code that does not run on any real AutoLISP host and is intended only for static-analysis tooling and unit tests.

3.5.5 Cross-references

- Function Entry: **BOUND**P (chapter 7) — the only language-level distinction between bound-to-nil and unbound, with its documented binding side effect.

3.5.6 Source Notes

- Phase-6 BricsCAD V26 / macOS semantics probe section C (indirect evidence; explicit probe pending due to launch-flake on this run).

3.6 Normative Rules: Tail-Call Optimization (None Required)

3.6.1 Statement

AutoLISP does **not** guarantee tail-call optimization. Recursion is stack-bound on every observed host. Programs that rely on bounded space for tail-recursive loops must be rewritten to use the iterative special forms **while**, **repeat**, or **foreach**.

3.6.2 Vendor Evidence

BricsCAD V26 / macOS direct probe section B: a deliberately-written tail-recursive (**count-down n**) succeeds at **n=1000**, then fails at **n=10000**, **100000**, **1000000** with **LISP - system call stack limit exceeded**. The boundary is well below typical Scheme implementations and is consistent with ordinary stack-frame consumption per call.

3.6.3 Cross-references

- Compare **R7RS 5.11.4** — Scheme requires **proper tail recursion** (constant space).
- Compare **CLHS 3.5.1.1** — Common Lisp does **not** require it; behaviour matches AutoLISP.
- Special Form Entry: **WHILE**, **REPEAT**, **FOREACH** (this chapter) — bounded-space iteration alternatives.

3.6.4 Source Notes

Phase-6 BricsCAD V26 / macOS semantics probe section B.

3.7 Normative Rules: Multiple Values (None)

3.7.1 Statement

AutoLISP has no concept of multiple return values. Every form returns exactly one value. There is no `values` primitive, no `multiple-value-bind`, no `call-with-values`. Functions that conceptually return more than one piece of data return a list, a dotted pair, or a host-defined structure (e.g. an entity-data dotted-pair list).

3.7.2 Vendor Evidence

BricsCAD V26 / macOS direct probe: `(values 1 2 3)` raises **no function definition <VALUES>; expected FUNCTION at [eval]**.

3.7.3 Cross-references

- Compare **CLHS 3.1.7** and **R7RS 6.10** — both Common Lisp and Scheme have first-class multiple-value mechanisms.

3.7.4 Source Notes

Phase-6 BricsCAD V26 / macOS semantics probe section D.

3.8 Normative Rules: Symbol Case Folding

3.8.1 Statement

The reader case-folds every unquoted symbol to **upper case** at read time. The case-folding applies uniformly to source text, the result of `(read string)`, and the canonical name returned by `vl-symbol-name`. There is no escape syntax (such as Common Lisp's `|...|` or `\`) for preserving mixed case in symbol names.

3.8.2 Vendor Evidence

BricsCAD V26 / macOS direct probe section G:

- `'foo` prints `F00`.
- `'Fo0` prints `F00`.
- `(eq 'foo 'F00)` returns `T`.
- `(vl-symbol-name 'mixed-Symbol)` returns the string `"MIXED-SYMBOL"`.
- `(read "mixedFromRead")` returns the symbol `MIXEDFROMREAD`.

3.8.3 Strict / Lax Policy

Strict in every supported dialect. The `bricscad-v26` dialect's lax token mode does not relax case folding; it relaxes only token classification (e.g. lone trailing identifier characters on integers). String literals are unaffected.

3.8.4 Cross-references

- Compare **CLHS 23.1** — CL also case-folds by default, but provides `readtable-case` and the `|...|` escape.
- Compare **R7RS 7.1.1** — Scheme is case-sensitive by default since R7RS.

3.8.5 Source Notes

Phase-6 BricsCAD V26 / macOS semantics probe section G.

3.9 Normative Rules: Number Tower and Division

3.9.1 Statement

AutoLISP recognises two numeric types: **integer** and **real** (IEEE-754 double-precision binary floating-point). Mixed-type arithmetic is permitted; the result is **real** whenever any argument is real, otherwise **integer**. Division follows these rules:

- `(/ INT INT)` — integer division, **truncating toward zero**. `(/ 7 2) = 3`; `(/ -7 2)` truncates toward zero to `-3` (subject to per-vendor confirmation; toward-zero is the consistent reading on modern hosts).
- `(/ REAL ANY)` or `(/ ANY REAL)` — real-valued division. `(/ 7.0 2) = 3.5`; `(/ 7 2.0) = 3.5`.
- `(/ INT 0)` — signals **divide by zero**.
- `(/ REAL 0.0)` — undefined-behaviour territory on some hosts (BricsCAD V26 / macOS aborts the application); programs must avoid it.

Integer width on modern AutoLISP is at least 64 bits in BricsCAD V26: `(+ 2147483640 100) = 2147483740` exceeds the historical 31-bit range without promotion or overflow. AutoCAD's per-version width remains 32-bit signed int historically; verification against AutoCAD 2026 is deferred (no licensed seat available).

Floating-point underflow goes to `0.0` rather than to a denormal indicator: `(* 1.0e-300 1.0e-300) = 0.0`.

3.9.2 Vendor Evidence

BricsCAD V26 / macOS direct probe section F.

3.9.3 Strict / Lax Policy

- Truncating integer division and the integer/real promotion rule are **strict** across all dialects.
- Real division by zero is **strict-error** in clautolisp; the `bricscad-v26` dialect's behaviour mirrors the host's documented "divide by zero" error rather than the application abort observed in this build.

- Integer width — clautolisp persists 32-bit signed integers as the strict baseline (BricsCAD release-history compatibility). Wider-integer support is a future dialect knob if AutoCAD evidence demands it.

3.9.4 Cross-references

- Compare **CLHS 12** — CL has a numeric tower with rationals, separates / (exact) from `truncate~`/`~floor~`/`~round`. AutoLISP does not.
- Compare **R7RS 6.2** — Scheme distinguishes exact and inexact numbers.

3.9.5 Source Notes

Phase-6 BricsCAD V26 / macOS semantics probe section F.

3.10 Normative Rules: Equality Predicates

3.10.1 Statement

AutoLISP defines the following equality predicates with the semantics observed in BricsCAD V26 / macOS:

- `(eq a b)` — pointer-level identity for cons cells, distinguishes freshly-allocated objects, but returns **T** for two textually-identical string literals (the host **interns** string literals), and **T** for two textually-identical symbol literals (canonical case-folded name).
- `(equal a b)` — structural equality over cons cells; numerically compares an integer to a real (`(equal 1 1.0)` returns **T**); compares strings by content.
- Truthiness: a value is **true** iff it is not **nil**. `0`, `0.0`, `""` are **true**. **nil** is the unique false value.

3.10.2 Vendor Evidence

BricsCAD V26 / macOS direct probe section H–J:

- `(equal "abc" "abc") = T ; (eq "abc" "abc") = T` — string literals are interned.
- `(equal 1 1.0) = T` — cross-type numeric equality.
- `(equal '(1 2) (list 1 2)) = T ; (eq '(1 2) (list 1 2)) = nil` — structural vs identity.
- `(if 0 'yes 'no) = YES ; (if 0.0 'yes 'no) = YES ; (if "" 'yes 'no) = YES ; (if '() 'yes 'no) = NO.`
- `(eq nil '()) = T ; (null nil) = T ; (null '()) = T` — **nil** and the empty list are the same object.

3.10.3 Cross-references

- Compare **CLHS 14.5.1** — CL's `eq` is finer-grained (does not intern strings).
- Compare **R7RS 6.1** — Scheme distinguishes `eqv?`, `eq?`, and `equal?`.

3.10.4 Source Notes

Phase-6 BricsCAD V26 / macOS semantics probe sections H, I, J.

3.11 Normative Rules: NIL and the Empty List

3.11.1 Statement

The symbol `nil`, the literal `()`, and the literal `'()` all denote the **same** object — the empty list and the false value. `(car nil)` and `(cdr nil)` both return `nil`, **not** an error.

3.11.2 Vendor Evidence

BricsCAD V26 / macOS direct probe section I:

- `(eq nil '()) = T`
- `(car nil) = nil ; (cdr nil) = nil`

3.11.3 Cross-references

- Compare **CLHS 26.1.97** — CL also identifies `nil` with the empty list and accepts `(car nil) = nil`.
- Compare **R7RS 6.4** — in Scheme `nil` is a separate symbol; `()` is the empty list; `(car '())` is an error.

3.11.4 Source Notes

Phase-6 BricsCAD V26 / macOS semantics probe section I.

3.12 Special Form Entry: QUOTE

3.12.1 Name

`'quote'`

3.12.2 Class

Special Form

3.12.3 Syntax

`(quote expr)`
`'expr`

3.12.4 Arguments and Values

- `'expr'`: any expression; not evaluated

3.12.5 Description

Returns `'expr'` without evaluating it.

3.12.6 Evaluation Rules

No subform evaluation occurs.

3.12.7 Return Values

Returns `'expr'`.

3.12.8 Side Effects

None.

3.12.9 Exceptional Situations

Malformed syntax is erroneous.

3.12.10 Examples

- `'(quote A) => A'`
- `“(1 2) => (1 2)”`

3.12.11 Compatibility

Stable across vendors.

3.12.12 Notes

Quote shorthand is defined in chapter 2.

3.12.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.12.14 Source Notes

- [quote (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-18F7E287-CB2F-4150-9A07-CE23C3F9E604.htm>)
- Status: documented.

3.13 Special Form Entry: IF

3.13.1 Name

`'if'`

3.13.2 Class

Special Form

3.13.3 Syntax

`(if testexpr thenexpr [elseexpr])`

3.13.4 Arguments and Values

- `'testexpr'`: an expression
- `'thenexpr'`: an expression
- `'elseexpr'`: an expression; optional

3.13.5 Description

Conditionally evaluates one of two branches.

3.13.6 Evaluation Rules

- evaluate `'testexpr'`
- if result is non-`'nil'`, evaluate `'thenexpr'`
- otherwise evaluate `'elseexpr'` if supplied

3.13.7 Return Values

- returns the selected branch value
- if `'elseexpr'` is absent and the test is false, returns `'nil'`

3.13.8 Side Effects

Side effects are those of:

- the evaluation of ‘testexpr’, which always occurs,
- the evaluation of ‘thenexpr’ when ‘testexpr’ is non-‘nil’,
- the evaluation of ‘elseexpr’ when ‘testexpr’ is ‘nil’ and ‘elseexpr’ is present.

The non-selected branch is not evaluated and has no side effects.

3.13.9 Exceptional Situations

Malformed syntax is erroneous.

3.13.10 Examples

- ‘(if T 1 2) => 1’
- ‘(if nil 1 2) => 2’

3.13.11 Compatibility

Stable across vendors.

3.13.12 Notes

Conditional truth is based on ‘nil’ versus non-‘nil’.

3.13.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.13.14 Source Notes

- [if (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-916F1A5C-FD70-4D66-897E-6DCD666DCB39.htm>)
- Status: documented.

3.14 Special Form Entry: COND

3.14.1 Name

`'cond'`

3.14.2 Class

Special Form

3.14.3 Syntax

```
(cond [(test result ...) ...])
```

3.14.4 Arguments and Values

- each clause contains a test and zero or more result forms

3.14.5 Description

Sequentially selects the first successful clause.

3.14.6 Evaluation Rules

- tests are evaluated in order
- when a test yields non-`'nil'`:
 - if result forms exist, they are evaluated sequentially and the last value is returned
 - otherwise the test value itself is returned

3.14.7 Return Values

- the selected clause value
- `'nil'` if no clause succeeds

3.14.8 Side Effects

Side effects arise from:

- evaluation of each test expression up to and including the first successful one,

- evaluation of the result forms of the selected clause, if any.

No later clause is evaluated after a successful clause is found.

3.14.9 Exceptional Situations

Malformed clause structure is erroneous.

3.14.10 Examples

- ‘(cond ((> 2 1) 'YES) (T 'NO)) => YES’

3.14.11 Compatibility

Stable across vendors.

3.14.12 Notes

‘T’ may be used in the last clause as an always-true test.

3.14.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.14.14 Source Notes

- [cond (AutoLISP)](<https://help.autodesk.com/cloudhelp/2018/ENU/AutoCAD-AutoLISP-Reference/files/GUID-7BA45202-D95F-4F2D-8D83-965024826074.htm>)
- Status: documented.

3.15 Special Form Entry: AND

3.15.1 Name

`'and'`

3.15.2 Class

Special Form

3.15.3 Syntax

`(and [expr ...])`

3.15.4 Arguments and Values

- zero or more expressions

3.15.5 Description

Computes the logical conjunction of a sequence of expressions.

3.15.6 Evaluation Rules

- expressions are evaluated left to right
- evaluation stops as soon as one expression evaluates to `'nil'`
- if no expression evaluates to `'nil'`, all expressions are evaluated

3.15.7 Return Values

Vendor summary documentation describes `'and'` as returning the logical `'AND'` of the expressions.

The exact return-value convention still needs dedicated per-function confirmation in this draft and therefore remains mildly under-specified.

3.15.8 Side Effects

Side effects are those of the expressions evaluated before short-circuiting stops evaluation, or of all expressions if none evaluates to `'nil'`.

3.15.9 Exceptional Situations

Malformed syntax is erroneous.

3.15.10 Examples

- ‘(and T T) => T’
- ‘(and T nil)’ yields false

3.15.11 Compatibility

Autodesk classifies ‘and’ as a special form.

3.15.12 Notes

Because Autodesk’s summary page is brief, the exact value returned in non-trivial truthy cases should still be confirmed by dedicated vendor pages or tests.

3.15.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.15.14 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- [Equality and Conditional Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-C6A7FC58-7A07-4650-9D59-12CAFD194FDA.htm>)
- Status: documented as a special form and by summary; exact return convention still needs tighter confirmation.

3.16 Special Form Entry: SETQ

3.16.1 Name

`'setq'`

3.16.2 Class

Special Form

3.16.3 Syntax

`(setq sym expr [sym expr] ...)`

3.16.4 Arguments and Values

- `'sym'`: a symbol; not evaluated
- `'expr'`: an expression

3.16.5 Description

Assigns values to variables.

3.16.6 Evaluation Rules

- symbols are taken literally
- expressions are evaluated left to right
- assignments occur left to right

3.16.7 Return Values

Returns the value of the last `'expr'`.

3.16.8 Side Effects

Establishes or changes variable bindings. Writes the supplied value into the symbol's **single binding cell** (see chapter 7, "Single-Cell Symbol Binding"); a previous **defun** on the same symbol is therefore overwritten.

3.16.9 Exceptional Situations

Malformed syntax or invalid assignment targets are erroneous.

3.16.10 Examples

- ‘(setq A 10) => 10‘
- ‘(setq A 1 B 2) => 2‘
- ‘(defun foo (x) x) (setq foo 42)’ — ‘foo‘ evaluates to ‘42‘; ‘(foo 5)’ then signals **no function definition**.

3.16.11 Compatibility

Stable across vendors.

3.16.12 Notes

‘set‘ differs by evaluating the symbol designator. AutoLISP is **Lisp-1**: **setq** and **defun** share the same per-symbol binding cell; the most recent write wins (chapter 7).

3.16.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.16.14 Source Notes

- [setq (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-LT-AutoLISP-Reference/files/GUID-2F4B7A7B-7B6F-4E1C-B32E-677506094EAA.htm>)
- Status: documented.

3.17 Special Form Entry: SET

3.17.1 Name

`'set'`

3.17.2 Class

Function-like assignment operator

3.17.3 Syntax

`(set sym expr)`

3.17.4 Arguments and Values

- `'sym'`: an expression whose value designates a symbol
- `'expr'`: an expression

3.17.5 Description

Assigns a value to the symbol designated by the first argument.

3.17.6 Return Values

Returns the assigned value.

3.17.7 Side Effects

Changes the binding of the designated symbol.

3.17.8 Exceptional Situations

Erroneous if the first evaluated argument does not designate a valid assignable symbol.

3.17.9 Examples

- `'(set 'A 3) => 3'`

3.17.10 Compatibility

Stable across vendors.

3.17.11 Notes

Unlike ‘setq’, the symbol argument is evaluated.

3.17.12 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.17.13 Source Notes

- [set (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-041BE20B-CFA6-465B-A98B-1BCBDC810881.htm>)
- Status: documented.

3.18 Special Form Entry: WHILE

3.18.1 Name

‘while’

3.18.2 Class

Special Form

3.18.3 Syntax

(while testexpr [expr ...])

3.18.4 Description

Repeatedly evaluates body forms while the test remains non-‘nil’.

3.18.5 Evaluation Rules

- evaluate ‘testexpr’
- if the result is non-‘nil’, evaluate the body forms in order and then repeat
- stop when ‘testexpr’ evaluates to ‘nil’

3.18.6 Return Values

Returns the value of the last body expression of the last iteration, or ‘nil’ if the body is never run.

3.18.7 Side Effects

Side effects are those of:

- each evaluation of ‘testexpr’,
- each evaluation of the body forms during iterations that occur.

3.18.8 Exceptional Situations

Malformed syntax is erroneous.

3.18.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.18.10 Source Notes

- [while (AutoLISP)](<https://help.autodesk.com/cloudhelp/2018/ENU/AutoCAD-AutoLISP-Reference/files/GUID-E7C900DB-8B66-4109-BEF6-B0A18E8CF6B6.htm>)
- Status: documented in core behavior; concise here.

3.19 Special Form Entry: REPEAT

3.19.1 Name

`'repeat'`

3.19.2 Class

Special Form

3.19.3 Syntax

`(repeat int [expr ...])`

3.19.4 Description

Evaluates body forms a fixed number of times.

3.19.5 Evaluation Rules

- evaluate the repetition count
- if the count permits iteration, evaluate body forms in order for each iteration

3.19.6 Return Values

Returns the final body value or `'nil'` if no body exists.

3.19.7 Side Effects

Side effects are those of:

- evaluation of the repetition count,
- repeated evaluation of the body forms.

3.19.8 Exceptional Situations

Erroneous if the repetition count is not acceptable to the active host semantics.

3.19.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.19.10 Source Notes

- [repeat (AutoLISP)](<https://help.autodesk.com/cloudhelp/2018/ENU/AutoCAD-AutoLISP-Reference/files/GUID-413F72B4-BA37-4E5E-9D51-A0091130A317.htm>)
- Status: documented in core behavior; concise here.

3.20 Special Form Entry: FOREACH

3.20.1 Name

‘foreach‘

3.20.2 Class

Special Form

3.20.3 Syntax

(foreach name list [expr ...])

3.20.4 Description

Iterates over each element of ‘list‘, binding it to ‘name‘.

3.20.5 Evaluation Rules

- evaluate ‘list‘
- for each element of the resulting list, bind it to ‘name‘
- evaluate the body forms in order

3.20.6 Return Values

Returns the final body value or ‘nil‘ if no body exists.

3.20.7 Side Effects

Side effects are those of:

- evaluation of ‘list‘,
- rebinding ‘name‘ for each iteration,
- evaluation of the body forms for each iteration.

3.20.8 Exceptional Situations

Erroneous if the list argument is not acceptable to the active host semantics.

3.20.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.20.10 Source Notes

- [foreach (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-BE6A23C4-4E18-45A6-854E-2DE9574A6925.htm>)
- Status: documented in core behavior; concise here.

3.21 Special Form Entry: OR

3.21.1 Name

`'or'`

3.21.2 Class

Special Form

3.21.3 Syntax

`(or [expr ...])`

3.21.4 Description

Tests expressions sequentially and returns `'T'` if any is non-`'nil'`, otherwise `'nil'`.

3.21.5 Evaluation Rules

Short-circuits on the first non-`'nil'` result.

3.21.6 Return Values

Returns `'T'` or `'nil'`.

3.21.7 Side Effects

Side effects are those of the expressions evaluated up to and including the first non-`'nil'` expression, or of all expressions if all are `'nil'`.

Expressions not reached because of short-circuiting are not evaluated and have no side effects.

3.21.8 Compatibility

This differs from Common Lisp `'or'`, which returns the first non-`'nil'` value. AutoLISP `'or'` returns only `'T'` or `'nil'`. Confirmed against BricsCAD V26 by direct citation: the BricsCAD V26 `'or'` reference page documents the return as "T if any provided argument evaluates as non-NIL; if all provided arguments evaluate as NIL, or if no argument is provided, NIL is returned" with examples `'(or 123 nil) -> T'`. Idioms ported from Common Lisp such as `'(setq x (or x default))'` **clobber** `'x'` with `'T'` rather than preserving

the existing value; portable AutoLISP code must use `'(if (not x) (setq x default))'` instead.

3.21.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: documented (BricsCAD V26 dedicated per-symbol page, return: "T if any provided argument evaluates as non-NIL; ... NIL otherwise").
- Status: documented in both vendors. Phase-5 closure: behaviour confirmed identical (T / nil only) across both vendors; surfaced as a real cross-vendor **and** cross-language hazard for code ported from Common Lisp.

3.21.10 See Also

- `probe-core.lsp` — uses an 'if'-guarded init pattern explicitly because AutoLISP's 'or' does not return the first non-'nil' value (the original `'(setq x (or x default))'` idiom was discovered to clobber 'x' with 'T' against BricsCAD V26 during Phase 5).

3.21.11 Source Notes

- [or (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-64ECF4D5-0714-45FD-8E27-284E9D2FC8B2.htm>)
- [or (BricsCAD V26 DevRef)](https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/or.htm)
- Status: documented (Phase 5 closure; cross-vendor behaviour confirmed).

3.22 Special Form Entry: PROGN

3.22.1 Name

`'progn'`

3.22.2 Class

Special Form

3.22.3 Syntax

`(progn [expr ...])`

3.22.4 Arguments and Values

- zero or more expressions

3.22.5 Description

Evaluates expressions sequentially.

3.22.6 Evaluation Rules

- evaluate each expression left to right

3.22.7 Return Values

- returns the value of the last expression
- returns `'nil'` if no expressions are supplied

3.22.8 Side Effects

Side effects are those of the evaluated expressions.

3.22.9 Exceptional Situations

Malformed syntax is erroneous.

3.22.10 Examples

- `'(progn (setq a 1) (setq b 2) b) => 2'`

3.22.11 Compatibility

Autodesk classifies ‘progn’ as a special form.

3.22.12 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.22.13 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- Status: documented as a special form; detailed entry still partly inferred from stable Lisp usage.

3.23 Special Form Entry: LAMBDA

3.23.1 Name

`'lambda'`

3.23.2 Class

Special Form

3.23.3 Syntax

`(lambda ([arguments] [/ locals ...]) expr ...)`

3.23.4 Arguments and Values

- AutoLISP parameter and local syntax
- body expressions

3.23.5 Description

Creates an anonymous function object.

3.23.6 Evaluation Rules

The body expressions are not evaluated at lambda-creation time. They are evaluated when the resulting function is invoked.

3.23.7 Return Values

Returns an anonymous callable object.

3.23.8 Side Effects

None at creation time, apart from effects of malformed usage signaling errors.

3.23.9 Exceptional Situations

Malformed parameter/local syntax is erroneous.

3.23.10 Examples

- use in functional position
- use with ‘function’ in mapped or callback contexts

3.23.11 Compatibility

Autodesk classifies ‘lambda’ as a special form.

3.23.12 Notes

The exact object model of the resulting anonymous function is not formally specified in the vendor documentation sampled here.

3.23.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.23.14 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- Status: documented as a special form; detailed semantics partly inferred.

3.24 Special Form Entry: FUNCTION

3.24.1 Name

`'function'`

3.24.2 Class

Special Form

3.24.3 Syntax

`(function arg)`

3.24.4 Arguments and Values

- `'arg'`: a function name (symbol) or a lambda expression; **not** evaluated.

3.24.5 Description

Returns `'arg'` without evaluating it, as `'quote'` does. The form additionally signals intent to the Visual LISP byte-compiler, which uses the hint to link and optimise `'arg'` as if it were a built-in operator or `'defun'`-named function (Autodesk: "The function function is identical to the quote function, except it tells the Visual LISP compiler to link and optimize the argument as if it were a built-in function or defun.").

3.24.6 Evaluation Rules

No subform evaluation occurs. The returned value is the unevaluated `'arg'`; resolution to a callable object — symbol lookup, lambda capture — happens at the point the value is **used** (typically inside `'apply'`, `'mapcar'`, `'vl-some'`, `'vl-every'`, or any other higher-order operator), not at the point the `'function'` form was reached.

3.24.7 Return Values

- For a symbol `'arg'`: the symbol itself, dynamically resolved by the consumer.
- For a `'(lambda ...)'` form: the lambda form, reified to a function object when first applied.

3.24.8 Side Effects

None.

3.24.9 Exceptional Situations

- ‘arg’ neither a symbol nor a lambda form: ‘:invalid-function-designator’.
- An undefined function name only surfaces when the value is actually applied — ‘(function missing)’ itself does not signal.

3.24.10 Examples

- ‘(setq v (function +)) (apply v '(1 2)) => 3’
- ‘(setq lam (function (lambda (x) (* x x)))) (apply lam '(5)) => 25’
- ‘(mapcar (function 1+) '(1 2 3)) => (2 3 4)’

3.24.11 Portable Higher-Order-Function Idiom

AutoLISP is nominally a Lisp-1 with dynamic scope (chapter 7), so a parameter bound to a function value should be callable as ‘(v arg)’. The two reference implementations diverge on this point:

- AutoCAD 2022/2026 warns (“parameter used as a function”) and may refuse to dispatch through a parameter binding.
- BricsCAD V25/V26 silently ignores the parameter shadow at call position and resolves the name to the surrounding (global) binding — including when the caller passed a lambda anonymously, which then becomes unreachable.

Code that must run on both vendors therefore treats functions as in a Lisp-2:

Goal	Portable form
Define a named function	‘(defun foo (x) ...)’
Obtain the function value of a name	‘(function foo)’
Anonymous function value	‘(function (lambda (x) ...))’
Call a function held in a variable (parameter...)	‘(apply v (list arg1 ...))’
Pass a function as parameter	pass ‘(function f)’, apply via ‘apply’

Avoid:

```
(defun hof (object op)
  (op object))           ; non-portable direct call through a parameter
```

Prefer:

```
(defun hof (object op)
  (apply op (list object)))
```

3.24.12 Compiled-Code Caveats

Two subtle effects fall out of the compiler hint:

1. Visual LISP may **link** the argument of `'(function foo)'` at compile time. After such compilation, a later `'(defun foo ...)'` from elsewhere does **not** necessarily replace the call site's binding. Example:

```
;; foobar.lsp
(defun foo () 'foo)
(defun bar () (foo))

;; --- separately ---
(load (vlisp-compile 'st "foobar.lsp"))
(defun foo () 'new-foo)
(bar) ; => foo or new-foo, depending on compile-time linking
```

Programs that need to override `'foo'` at runtime should avoid early-bind hints in the compiled producer (use `'(quote foo)'` rather than `'(function foo)'` for symbols that are intended to be redefinable from a host), or use `'apply'` with a symbol passed at runtime.

2. For lambda forms, Autodesk recommends `'function'` over `'quote'` so the byte-compiler can optimise the body once, rather than rebuilding it on every call. Source: Lee Mac, **The Apostrophe and the Quote Function**.

3.24.13 Compatibility

Autodesk classifies `'function'` as a special form. Both vendors document the "identical to quote with compiler hint" semantics. clautolisp implements the same late-resolution semantic: the form returns its unevaluated argument.

3.24.14 Notes

- The portable HOF rule lives at the language level: it does not depend on whether the producer is interpreted, byte-compiled (FAS/VLX), or hosted by clautolisp.
- ‘function’ is widely used with ‘mapcar’, ‘vl-some’, ‘vl-every’, ‘vl-remove-if’, ‘vl-member-if’, and other Visual LISP higher-order utilities. Whether the argument is a symbol or a lambda, the consumer resolves it via the same dispatch path used by ‘apply’.

3.24.15 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- clautolisp: implemented (eval-function-form returns arg unevaluated; apply/mapcar/vl-* honour late resolution against the dynamic scope chain).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.24.16 Source Notes

- [function (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-MAC-AutoLISP-Reference/files/GUID-CF7E5870-561F-42DB-B134-CCD41EF9312D.html>)
- [apply (AutoLISP 2023)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-0574ADA0-0950-456A-9330-A2518421512D.html>)
- [Passing and calling functions (Autodesk Community)](<https://forums.autodesk.com/t5/visual-lisp-autolisp-and-general/passing-and-calling-functions-ltd-p/2463166>)
- Lee Mac, **The Apostrophe and the Quote Function** (<https://www.lee-mac.com/quote.html>)
- TN003 — **Valeurs fonctions en AutoLISP** (Pascal Bourguignon, 2026-05-22)

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- Status: documented.

3.25 Special Form Entry: COMMAND

3.25.1 Name

`'command'`

3.25.2 Class

Special Form

3.25.3 Syntax

`(command [args ...])`

3.25.4 Arguments and Values

- a sequence of command name and command input arguments

3.25.5 Description

Executes an AutoCAD command through the host command processor.

3.25.6 Evaluation Rules

Autodesk classifies `'command'` as a special form because its argument handling and command-loop interaction differ from ordinary function calls.

3.25.7 Return Values

Host-dependent command-execution result; exact return-value conventions are not fully formalized in the current source set.

3.25.8 Side Effects

- executes a host command
- may modify the drawing, system variables, user interaction state, and command stack

3.25.9 Affected By

- current document state
- command versioning
- localization
- ‘PAUSE‘
- ‘*error*‘ mode

3.25.10 Exceptional Situations

Command failure, invalid input, and command-loop state errors are host-dependent.

3.25.11 Compatibility

- central AutoLISP host interaction facility
- related to ‘command-s‘

3.25.12 Notes

This is one of the clearest examples of an AutoLISP construct that cannot be reduced to pure Lisp evaluation semantics.

3.25.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.25.14 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)

- [Query and Command Functions Reference](<https://help.autodesk.com/cloudhelp/2016/ENU/AutoCAD-AutoLISP/files/GUID-F9ABE09D-9A71-4C00-8E5D-5BA7C0A7C0A7.htm>)
- Status: documented in role and category; further host details remain chapter-level.

3.26 Special Form Entry: TRACE

3.26.1 Name

`'trace'`

3.26.2 Class

Special Form

3.26.3 Syntax

`(trace [sym ...])`

3.26.4 Description

Enables tracing of function calls for debugging.

3.26.5 Compatibility

Autodesk classifies `'trace'` as a special form.

3.26.6 Notes

Detailed debugger/tracer output semantics are not fully developed in this draft.

3.26.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.26.8 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- Status: category documented; detailed semantics still sparse.

3.27 Special Form Entry: UNTRACE

3.27.1 Name

‘untrace‘

3.27.2 Class

Special Form

3.27.3 Syntax

(untrace [sym ...])

3.27.4 Description

Disables tracing of function calls for debugging.

3.27.5 Compatibility

Autodesk classifies ‘untrace‘ as a special form.

3.27.6 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.27.7 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- Status: category documented; detailed semantics still sparse.

3.28 Special Form Entry: VLAX-FOR

3.28.1 Name

‘vlax-for‘

3.28.2 Class

Special Form

3.28.3 Syntax

(vlax-for var collection [expr ...])

3.28.4 Description

Iterates over an ActiveX collection object.

3.28.5 Compatibility

- Autodesk classifies ‘vlax-for‘ as a special form
- Windows-only in Autodesk documentation because it depends on ActiveX support

3.28.6 Notes

This construct belongs to the Visual LISP / ActiveX layer, not the core portable AutoLISP layer.

3.28.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.28.8 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- [AutoLISP Extensions Reference (AutoLISP/ActiveX)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-07C3599E-EC87-4EC0-8F0B-8DE4BD150A64.htm>)
- Status: category documented; detailed semantics tied to ActiveX collections.

3.29 Dictionary Coverage Index: Special Forms

The standardized special forms identified in Autodesk documentation are:

- ‘and‘
- ‘command‘
- ‘cond‘
- ‘defun‘
- ‘defun-q‘
- ‘foreach‘
- ‘function‘
- ‘if‘
- ‘lambda‘
- ‘or‘
- ‘progn‘
- ‘quote‘
- ‘repeat‘
- ‘setq‘
- ‘trace‘

- ‘untrace’
- ‘vlax-for’
- ‘while’

3.30 Special Form Entry: DEFUN

3.30.1 Name

‘defun‘

3.30.2 Class

Special Form

3.30.3 Syntax

```
(defun sym ([arguments] [/ locals ...]) expr ...)
```

3.30.4 Arguments and Values

- ‘sym’: function name
- ‘arguments’: parameter names
- ‘/ locals’: local variable declarations within the AutoLISP parameter syntax
- ‘expr’: body forms

3.30.5 Description

Defines a named function. Names beginning with ‘C:‘ define CAD commands callable at the command line.

3.30.6 Evaluation Rules

The parameter/local syntax is AutoLISP-specific and must not be replaced with Common Lisp lambda-list semantics.

3.30.7 Return Values

Under-specified in the current vendor corpus sampled here; implementations commonly return the function name, but this should be version-tested before being treated as normative.

3.30.8 Side Effects

Writes the resulting function value into the symbol's **single binding cell** (see chapter 7, "Single-Cell Symbol Binding"). Any previous binding on 'sym' — variable value, prior function, or built-in — is overwritten and is no longer reachable through the symbol.

3.30.9 Compatibility

- local variable syntax is vendor-documented
- duplicate-name behavior is uniform across vendors at the per-symbol level: the most recent **setq** or **defun** on a symbol wins. This was confirmed for BricsCAD V26 by direct probe ([results/bricscad/macos/20260426T155521Z-namesp](#)) and is consistent with the BricsCAD release-note defect SR44723 fix.

3.30.10 Notes

The slash separating arguments and locals must be surrounded by spaces in Autodesk documentation. AutoLISP is **Lisp-1**: **defun** and **setq** share the same per-symbol binding cell.

3.30.11 Extension (clautolisp): variadic lambda lists with &rest

This subsection documents a clautolisp extension to the **defun** (and **lambda**) lambda list; it is **not** part of the Autodesk-documented syntax. It exists so that user code can cleanly override the variadic built-ins (**princ**, **load**, ...): a fixed-arity redefinition cannot accept the same range of calls the original does, whereas a **&rest** parameter can.

Extended lambda-list grammar:

```
lambda-list      ::= { parameter-name } [ '&rest' rest-parameter-name ] [ '/' { local-name } ]  
parameter-name  ::= symbol .  
rest-parameter-name ::= parameter-name .  
local-name      ::= symbol .
```

&rest and **/** stand for the symbols **&REST** and **/** themselves. The optional **&rest** clause, when present, must appear after the mandatory parameter names and before the **/** locals.

Semantics (mirrors Common Lisp **&rest**, which BricsCAD V26 also implements as an undocumented extension):

- the call must supply at least as many arguments as there are mandatory **parameter-name** entries; they are bound in order;
- any **excess** arguments are gathered into a freshly-consed list bound to **rest-parameter-name** (**nil** when there is no excess);
- with a **&rest** parameter present there is no upper bound on the argument count — extras accumulate in the list rather than being passed positionally;
- **local-name** entries after **/** behave exactly as in the base syntax.

Example — redefining the variadic **princ** over a module output port:

```
(defun funcall (fun &rest arguments)
  (apply fun arguments))

(setq original-princ (function princ))
(defun princ (&rest arguments / value fileHandle)
  (if (null arguments)
      (funcall original-princ)
      (progn
        (setq value (car arguments))
        (setq fileHandle (mymodule-output))
        (if (cdr arguments)
            (setq fileHandle (cadr arguments)))
        (mymodule-write fileHandle value))))
```

Diagnostics surface at **call** time, matching AutoLISP’s late-binding error model:

- **:wrong-number-of-arguments** — fewer arguments than the mandatory formals require;
- **:invalid-rest-parameter** — **&rest** followed by other than exactly one symbol, or **&rest** appearing after **/** (where it would name a local rather than introduce a rest parameter).

Dialect handling: the default clautolisp dialect accepts **&rest** silently. dialect **bricscad** accepts it (and is expected to warn that it is an undocumented BricsCAD extension); dialect **strict** / **autocad** reject it at **defun** time with **:strict-lambda-list-extension**. The dialect-warning wiring is tracked in **issues/open/bricscad-undocumented-clisms.issue**.

3.30.12 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.
- `&rest` extension: clautolisp (implemented 1.1.16); BricsCAD V26 (undocumented, confirmed by Bricsys support); AutoCAD: not available.

3.30.13 Source Notes

- [About Defining a Function (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-LT-AutoLISP/files/GUID-9EDF48C2-1678-4DC3-BFD6-9D4E4E4E4E4E.htm>)
- Status: documented in practical syntax, partially under-specified semantically.

3.31 Special Form Entry: DEFUN-Q

3.31.1 Name

‘defun-q‘

3.31.2 Class

Special Form

3.31.3 Syntax

```
(defun-q sym ([arguments] [/ variables ...]) expr ...)
```

3.31.4 Description

Defines a function in a backward-compatible representation that can be accessed as a list structure.

Historically, older non-compiled AutoLISP implementations represented ‘defun‘ definitions as list structure. Autodesk documents that this changed in AutoCAD 2000. After that change, ordinary ‘defun‘ definitions were no longer safely manipulable as lists.

‘defun-q‘ exists specifically to preserve compatibility with older code that:

- inspects function definitions as list data,
- appends to or rewrites function bodies as lists,
- expects to retrieve a function definition structurally rather than only call it.

Accordingly, ‘defun-q‘ is not just another spelling of ‘defun‘. It defines a callable function while also preserving a list-form definition that can be retrieved and manipulated by compatibility utilities such as ‘defun-q-list-ref‘ and ‘defun-q-list-set‘.

Autodesk explicitly says that ‘defun-q‘ is provided strictly for backward compatibility and should not be used for other purposes.

3.31.5 Evaluation Rules

- the defining form establishes a global function binding
- the defined function is callable in the usual way

- the definition is also retained in list form for reflective compatibility operations

3.31.6 Return Values

Returns the function name.

3.31.7 Side Effects

- defines or redefines a global function
- stores a list-structured representation of the definition for later access by compatibility functions

3.31.8 Compatibility

Autodesk documents duplicate names as tolerated with first occurrence used and later duplicates ignored.

An ordinary ‘defun’ definition should not be assumed to be accessible as list structure in modern AutoCAD semantics. Attempting to use a ‘defun’ function as a list is documented to produce an error, with Autodesk explicitly suggesting ‘defun-q’ instead for old compatibility patterns.

3.31.9 Examples

- define a compatibility-visible function:
 - ‘(defun-q my-startup (x) (print (list x)))’
- retrieve its stored list structure:
 - ‘(defun-q-list-ref ’my-startup) => ((X) (PRINT (LIST X)))’
- older code patterns that splice or append function bodies are the intended use case for ‘defun-q’

3.31.10 Notes

Conceptually, ‘defun-q’ separates two concerns that were historically conflated:

- defining a callable function,
- treating the function definition as ordinary list data.

Modern ‘defun’ covers the first concern. ‘defun-q’ exists only for the second, compatibility-driven concern.

3.31.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.31.12 Source Notes

- [defun-q (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/FRA/AutoCAD-AutoLISP-Reference/files/GUID-5EE138EF-D531-441B-9F12-8D5645C540FE.htm>)
- [About Compatibility of Defun with Earlier Releases of AutoCAD (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ITA/AutoCAD-AutoLISP/files/GUID-16246E02-459F-4408-9CB4-6F2CB306FD9F.htm>)
- [defun-q-list-ref (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ITA/AutoCAD-MAC-AutoLISP-Reference/files/GUID-DAF4A76E-9346-4A68-A0C8-1769.htm>)
- Status: documented.

3.32 Function Entry: DEFUN-Q-LIST-REF

3.32.1 Name

`'defun-q-list-ref'`

3.32.2 Class

Function

3.32.3 Syntax

`(defun-q-list-ref 'function)`

3.32.4 Arguments and Values

- `'function'`: a symbol naming a function

3.32.5 Description

Returns the stored list structure of a function defined with `'defun-q'`.

This is the reflective accessor that makes the special purpose of `'defun-q'` concrete: code can inspect the compatibility-preserved structural definition rather than only call the function.

3.32.6 Return Values

- the function's list definition if the named function was defined as a list
- `'nil'` if the argument does not designate a list-defined function

Autodesk's example shows a result of the form:

```
((X) (PRINT (LIST X)))
```

which corresponds to:

- the argument/local-variable specification
- the body forms

3.32.7 Side Effects

None specified.

3.32.8 Affected By

- whether the target function was defined with ‘defun-q’

3.32.9 Exceptional Situations

The sampled Autodesk reference does not document additional exceptional situations beyond normal function-call validity requirements.

3.32.10 Examples

```
(defun-q my-startup (x) (print (list x)))  
(defun-q-list-ref 'my-startup)  
=> ((X) (PRINT (LIST X)))
```

3.32.11 Compatibility

- intended only for the backward-compatibility workflow surrounding ‘defun-q’
- documented by Autodesk on Windows, Mac OS, and Web

3.32.12 Notes

‘defun-q-list-ref’ should not be understood as a general reflection API over arbitrary function definitions. It is specifically tied to the compatibility representation established by ‘defun-q’.

3.32.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.32.14 Source Notes

- [defun-q-list-ref (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ITA/AutoCAD-MAC-AutoLISP-Reference/files/GUID-DAF4A76E-9346-4A68-A0C8-176991769176.htm>)

- [Function-Handling Functions Reference](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-6804DF2D-E5C4-466A-9869-1A1A1A1A1A1A.htm>)
- Status: documented.

3.33 Function Entry: DEFUN-Q-LIST-SET

3.33.1 Name

`'defun-q-list-set'`

3.33.2 Class

Function

3.33.3 Syntax

`(defun-q-list-set 'sym list)`

3.33.4 Arguments and Values

- `'sym'`: a symbol naming the function to define or redefine
- `'list'`: a list-structured function definition

3.33.5 Description

Defines or redefines a function from a list-structured compatibility representation.

Together, `'defun-q'`, `'defun-q-list-ref'`, and `'defun-q-list-set'` form the compatibility triad for programs that manipulate function definitions as list data.

3.33.6 Return Values

Under-specified in the current source set.

The function-handling summary identifies it as a compatibility function that "defines a function as a list", but the dedicated page was not collected in this pass.

3.33.7 Side Effects

- defines or redefines the named function
- installs a list-based compatibility definition

3.33.8 Affected By

- validity of the supplied list structure
- compatibility behavior of the active dialect

3.33.9 Exceptional Situations

Under-specified in the current source set.

At minimum, invalid list structure should be expected to be erroneous, but the exact contract still needs the dedicated Autodesk page.

3.33.10 Examples

The current source set does not yet contain a fully documented vendor example for this function.

3.33.11 Compatibility

- documented by Autodesk as part of the function-handling family
- intended for backward compatibility only, like ‘defun-q’

3.33.12 Notes

This function is important because it implies that the ‘defun-q’ list representation is not only inspectable but writable.

That makes the preserved list representation semantically active, not merely descriptive.

3.33.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

3.33.14 Source Notes

- [Function-Handling Functions Reference](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-6804DF2D-E5C4-466A-9869-4A8A8A8A8A8A.htm>)
- [D Functions Reference](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-76EE3904-E89B-441A-84FD-C0AB3289742C.htm>)
- Status: partially documented; dedicated per-function page still needed.

3.34 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)
- [About Error Handling (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/PLK/AutoCAD-AutoLISP/files/GUID-027AD2E0-5AC5-48DA-B451-112B7EECEB8E.htm>)
- Status: mixed documented and under-specified in certain details.

4 4 Types and Runtime Classes

4.1 Overview

AutoLISP exposes a practical runtime type system through the ‘type’ function and through function contracts.

4.2 Distinguished Object Entry: NIL

4.2.1 Name

‘nil‘

4.2.2 Class

Distinguished object

4.2.3 Related Types

- ‘LIST‘
- generalized boolean false

4.2.4 Description

‘nil‘ is a distinguished object in AutoLISP.

In documented AutoLISP behavior:

- ‘nil‘ denotes falsity in conditional and predicate contexts,
- ‘nil‘ is also the empty list,
- ‘(type nil)’ returns ‘nil’, not a distinct runtime type designator such as ‘NIL’,
- ‘listp’ returns ‘T’ for ‘nil’,
- ‘null’ and ‘not’ return ‘T’ for expressions that evaluate to ‘nil’.

Accordingly, this specification does not treat ‘nil’ as a separate runtime type in the same sense as ‘INT’, ‘REAL’, ‘STR’, ‘LIST’, or ‘SYM’.

4.2.5 Representation

It is printed as ‘nil’ in value position and corresponds semantically to the empty list ‘()’.

4.2.6 Valid Operations

- ‘listp’ returns ‘T’
- ‘vl-symbolp’ returns ‘nil’
- ‘type’ returns ‘nil’
- conditional contexts treat it as false

4.2.7 Compatibility

This is a core language distinction and must not be collapsed to generic Common Lisp symbol or type behavior.

In particular, the Common Lisp type named ‘nil’ is not an appropriate model for AutoLISP’s documented ‘type’ behavior.

4.2.8 Source Notes

- [type (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-506C9CC8-B0BD-4A4C-B4C2-006750504509.htm>)
- [listp (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/CHS/AutoCAD-AutoLISP-Reference/files/GUID-8755F083-D3BA-45F0-BBAF-D60D3A73240C.htm>)
- [vl-symbolp (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-F51AFE0A-CA6C-428D-8E8D-3607BF5519.htm>)
- [null (AutoLISP)](<https://help.autodesk.com/cloudhelp/2016/DEU/AutoCAD-AutoLISP/files/GUID-2CA2E5F1-297F-4ECA-9500-5FFCD4877126.htm>)
- [not (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ESP/AutoCAD-MAC-AutoLISP-Reference/files/GUID-991AD6A0-61AA-45C9-8C27-CADF9F36BE71.htm>)
- Status: documented by behavior; explicitly not treated here as a separate runtime type.

4.3 Type Entry: Symbol

4.3.1 Name

Symbol

4.3.2 Class

Type

4.3.3 Related Types

- variable names
- function names
- command names

4.3.4 Description

Symbols identify variables, functions, commands, and symbolic data.

4.3.5 Representation

Symbols are case-insensitive in the source language. Printed examples use uppercase.

4.3.6 Valid Operations

- ‘type’
- ‘vl-symbolp’
- ‘atoms-family’
- binding operations

4.3.7 Compatibility

Do not model AutoLISP symbols using raw Common Lisp package semantics.

4.3.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

4.3.9 Source Notes

- [About Symbols and Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP/files/GUID-1855E7B0-2474-49E6-9EE3-8BC54...htm>)
- [type (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-506C9CC8-B0BD-4A4C-B4C2-006750504509...htm>)
- Status: documented in practice.

4.4 Type Entry: ENAME

4.4.1 Name

‘ENAME‘

4.4.2 Class

Host Type

4.4.3 Description

An entity name designates an object in the host drawing database.

4.4.4 Representation

Opaque host-managed object identity.

4.4.5 Valid Operations

- ‘entget‘
- ‘entmod‘
- ‘entlast‘
- ‘ssname‘
- ‘namedobjdict‘

4.4.6 Compatibility

Core to host interaction in both Autodesk and BricsCAD environments.

4.4.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

4.4.8 Source Notes

- [namedobjdict (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-A1E43B30-EF8C-452E-97F5-8AC203111111.htm>)
- Status: documented by host APIs.

4.5 Type Entry: PICKSET

4.5.1 Name

‘PICKSET‘

4.5.2 Class

Host Type

4.5.3 Description

A selection set is a host-managed collection of selected entities.

4.5.4 Representation

Opaque host-managed selection object.

4.5.5 Valid Operations

- ‘ssget‘
- ‘ssname‘
- ‘sslenght‘

4.5.6 Compatibility

Selection-set indexing and limits are version-sensitive and must be tested.

4.5.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

4.5.8 Source Notes

- `[ssname (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-LT-AutoLISP-Reference/files/GUID-EFB83751-9AC5-4C52-AD3D-D971BC560C11.htm>)
- Status: documented in usage.

4.6 Type Entry: FILE

4.6.1 Name

‘FILE’

4.6.2 Class

Type

4.6.3 Description

A file descriptor returned by ‘open’.

4.6.4 Valid Operations

- ‘read-char’
- ‘read-line’
- output functions
- ‘close’

4.6.5 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

4.6.6 Source Notes

- [open (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-089A323F-21FF-4337-99A9-375758E23BA4.htm>)
- Status: documented.

4.7 Source Notes

- [type (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-506C9CC8-B0BD-4A4C-B4C2-006750504509.htm>)
- Status: document chapter still incomplete but structure is stable.

5 5 Data and Control Flow

5.1 Overview

AutoLISP data and control flow are built around symbols, numbers, strings, lists, dotted pairs, and control operators such as ‘if’, ‘cond’, ‘while’, ‘repeat’, and ‘foreach’.

5.2 Normative Rules

- ‘nil’ is both false and the empty list
- points are ordinary lists of two or three numbers
- association lists are central host data structures
- dotted pairs are first-class data

5.3 Variable Entry: NIL

5.3.1 Name

`'nil'`

5.3.2 Class

Predefined value

5.3.3 Value Type

Distinguished object.

5.3.4 Initial Value

Itself.

5.3.5 Description

`'nil'` is the canonical false value and the canonical empty-list value.

Autodesk documentation sampled here does not describe `'nil'` as a separate `'type'` designator. Instead, it describes `'nil'` through its behavior:

- as the value recognized by `'null'` and `'not'`,
- as the empty list accepted by `'listp'`,
- as a value for which `'(type nil)'` returns `'nil'`.

5.3.6 Affected By

- conditional evaluation
- list-processing functions
- type and predicate functions

5.3.7 Exceptional Situations

None specific to `'nil'` as a predefined value are documented in the sampled corpus.

5.3.8 Examples

- ‘(null nil) => T‘
- ‘(not nil) => T‘
- ‘(listp nil) => T‘
- ‘(type nil) => nil‘

5.3.9 Compatibility

This entry is intentionally written as a predefined value entry, not a type entry.

5.3.10 Notes

This specification avoids importing the Common Lisp distinction between the ‘nil‘ type and the ‘null‘ type into AutoLISP unless vendor evidence later requires it.

5.3.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

5.3.12 Source Notes

- [type (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-506C9CC8-B0BD-4A4C-B4C2-006750504509.htm>)
- [listp (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/CHS/AutoCAD-AutoLISP-Reference/files/GUID-8755F083-D3BA-45F0-BBAF-D60D3A73240C.htm>)
- [null (AutoLISP)](<https://help.autodesk.com/cloudhelp/2016/DEU/AutoCAD-AutoLISP/files/GUID-2CA2E5F1-297F-4ECA-9500-5FFCD4877126.htm>)

- [not (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ESP/AutoCAD-MAC-AutoLISP-Reference/files/GUID-991AD6A0-61AA-45C9-8C27-CADF9F36BE71.htm>)
- Status: documented by behavior.

5.4 Source Notes

- [About Point Lists (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP/files/GUID-BFED8707-AB01-49BC-B0A9-D872794311DF.htm>)
- [About Dotted Pairs (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/PLK/AutoCAD-AutoLISP/files/GUID-877B87BA-6FA2-4894-9F94-184E7DF25B7D.htm>)

5.5 Function Entry: APPLY

5.5.1 Name

`'apply'`

5.5.2 Class

Function

5.5.3 Syntax

`(apply 'function list)`

5.5.4 Arguments and Values

- `'function'`: a symbol (defun name) or a lambda expression.
- `'list'`: a list of arguments, or `'nil'` if the function takes no arguments.

5.5.5 Description

Passes a list of arguments to, and executes, the specified function. The function may be a built-in operator, a user-defined function, or a lambda expression.

5.5.6 Return Values

Returns whatever the called function returns: integer, real, string, list, ename, T, or nil.

5.5.7 Side Effects

Side effects are those of the called function.

5.5.8 Examples

- `'(apply '+ '(1 2 3))'` returns `'6'`.
- `'(apply 'strcat '("a" "b" "c"))'` returns `"abc"`.

5.5.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

5.5.10 Source Notes

- [apply (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-0574ADA0-0950-456A-9330-A25184215htm>)
- Status: documented (Phase 3 body fill).

5.6 Function Entry: EVAL

5.6.1 Name

`'eval'`

5.6.2 Class

Function

5.6.3 Syntax

`(eval expr)`

5.6.4 Arguments and Values

- `'expr'`: any AutoLISP expression. Documented value types are integer, real, string, list, symbol, ename, `'T'`, or `'nil'`.

5.6.5 Description

Returns the result of evaluating an AutoLISP expression. Enables dynamic evaluation, including resolving a symbol bound through a value-cell to its current value.

5.6.6 Return Values

Returns the value produced by evaluating `'expr'`. The return type follows the expression type and is one of integer, real, string, list, ename, `'T'`, or `'nil'`.

5.6.7 Side Effects

Side effects are those of the evaluated expression.

5.6.8 Examples

Given `'(setq a 123)'` and `'(setq b 'a)'`:

- `'(eval 4.0)'` returns `'4.0'`.
- `'(eval (abs -10))'` returns `'10'`.
- `'(eval a)'` returns `'123'`.

- ‘(eval b)’ returns ‘123’, because ‘b’ holds the symbol ‘a’ which itself names the value ‘123’.

5.6.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

5.6.10 Source Notes

- [eval (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-D9B3E6CC-A982-4040-AE6E-6FD63D6C54D0.htm>)
- Status: documented (Phase 3 body fill).

5.7 Function Entry: EQ

5.7.1 Name

`'eq'`

5.7.2 Class

Function

5.7.3 Syntax

`(eq expr1 expr2)`

5.7.4 Arguments and Values

- `'expr1'`, `'expr2'`: integer, real, string, list, ename, `'T'`, or `'nil'`.

5.7.5 Description

Determines whether two expressions are **identical** — that is, whether both arguments reference the same object in memory, not merely equivalent values. For lists this is object identity; two structurally equal but distinct lists are not `'eq'`.

5.7.6 Return Values

Returns `'T'` when the expressions reference identical objects, otherwise `'nil'`.

5.7.7 Side Effects

None.

5.7.8 Examples

Given `'(setq f1 '(a b c))'`, `'(setq f2 '(a b c))'`, `'(setq f3 f2)'`:

- `'(eq f1 f3)'` returns `'nil'` — `'f1'` and `'f3'` denote distinct list objects, even though they are structurally equal.
- `'(eq f3 f2)'` returns `'T'` — `'f3'` and `'f2'` refer to the same list object.

5.7.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

5.7.10 Source Notes

- [eq (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-BC81B391-48BF-401E-80D0-05A1B578A0FF.htm>)
- Status: documented (Phase 3 body fill).

5.8 Function Entry: EQUAL

5.8.1 Name

`'equal'`

5.8.2 Class

Function

5.8.3 Syntax

`(equal expr1 expr2 [fuzz])`

5.8.4 Arguments and Values

- `'expr1'`, `'expr2'`: integer, real, string, list, `ename`, `'T'`, or `'nil'`.
- `'fuzz'` (optional): integer or real tolerance value used for inexact numeric comparison.

5.8.5 Description

Determines whether two expressions evaluate to the same value. Unlike `'eq'`, `'equal'` compares structurally: two distinct but element-wise equal lists are `'equal'`. The optional `'fuzz'` argument controls tolerance for real-number comparison and for lists of reals such as point coordinates; two reals computed by different methods can differ slightly, and `'fuzz'` lets such values compare equal when the absolute difference is within tolerance.

5.8.6 Return Values

Returns `'T'` if the expressions are equal under the rules above, otherwise `'nil'`.

5.8.7 Side Effects

None.

5.8.8 Examples

- Structural equality on lists: `'(equal '(a b c) '(a b c))'` returns `'T'`.
- Numeric tolerance: with `'(setq a 1.123456)'` and `'(setq b 1.123457)'`, `'(equal a b)'` returns `'nil'`, while `'(equal a b 0.000001)'` returns `'T'`.

5.8.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

5.8.10 Source Notes

- `[equal (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-7E85CB8F-B4DA-42F3-ABD3-89342A11F11F.htm>)
- Status: documented (Phase 3 body fill).

6 6 Iteration

6.1 Overview

The primary standardized iteration constructs explicitly documented in the sampled corpus are ‘while’, ‘repeat’, and ‘foreach’.

6.2 Dictionary Entries

See chapter 3 entries for ‘while’, ‘repeat’, and ‘foreach’.

6.3 Source Notes

- [About Special Forms (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP/files/GUID-F992EAA3-AFFC-425D-B7C9-6A9CB0EA2768.htm>)

7 7 Symbols and Bindings

7.1 Overview

Symbols name values and functions. AutoLISP variable management is documented pragmatically rather than through a formal environment calculus.

7.2 Normative Rule: Single-Cell Symbol Binding (Lisp-1)

7.2.1 Statement

Each AutoLISP symbol carries exactly **one** binding cell. `setq` and `defun` both write to that single cell; either operation overwrites whatever the other wrote previously. There is no separate function cell distinct from the variable cell. In every position — value position or function-call position — the evaluator dispatches on the same stored value.

This places AutoLISP / Visual LISP in the **Lisp-1** family (a single namespace shared by variables and functions, like Scheme), not the **Lisp-2** family (separate cells per role, like Common Lisp).

7.2.2 Concrete Consequences

- After `(setq foo 42)` followed by `(defun foo (x) (+ 1 x))`, evaluating the bare symbol `foo` returns the function object — not 42. The `setq`-supplied integer is gone.
- After `(defun bar (x) ...)` followed by `(setq bar 42)`, calling `(bar 5)` raises **no function definition** — the function is gone.
- `(setq myfunc eq)` followed by `(myfunc "1" "1")` must succeed: the variable's stored value (the `eq` subroutine) is dispatched as the call's function. Bricsys logged this as defect **SR44723** and shipped the fix accordingly.
- `defun-q` obeys the same single-cell rule, with the additional property that the function value remains list-accessible through `defun-q-list-ref` / `defun-q-list-set`.
- A `(defun ... (/ x ...) ...)` local — i.e. a name appearing after the slash in a lambda list — establishes a **dynamic shadowing** of the same single cell during the call's extent, then restores the prior binding on exit. Local declarations therefore work uniformly for symbols whose outer binding is a value, a function, or both successively.

7.2.3 Vendor Evidence

- BricsCAD V26 release-note defect **SR44723** explicitly documents the expected behaviour: `(setq myfunc eq)` followed by `(myfunc "1" "1")` must run the `eq` implementation. The fix landed in mainline V25 and is preserved through V26.

- BricsCAD V26 / macOS direct probe (this repository, [results/bricscad/macos/20260426T1555](#))
 - TEST1: `(setq probe-foo 42)` then `(defun probe-foo (x) (+ 1 x))` — evaluating the bare symbol `probe-foo` returns `#<<FUNCTION>> ...>`, **not** 42. `(probe-foo 5)` returns 6.
 - TEST2: after `(setq probe-bar 42)` on a previously `defun`'d symbol, calling `(probe-bar 5)` raises **no function definition <PROBE-BAR>; expected FUNCTION at [eval]**. The function value was overwritten by the integer.
- Autodesk reference, **About Using Defun to Define a Function**: "Never use the name of a built-in function or symbol for the sym argument to defun, as this overwrites the original definition and makes the built-in function or symbol inaccessible." The wording does not distinguish a function role from a value role; it speaks of a single overridable definition.
- Autodesk reference, **About Local and Global Variables**: "If a variable name is added to the local variables list of a function, the global variable with the same name is ignored." This describes single-cell dynamic shadowing.

7.2.4 Strict / Lax Policy

The single-cell rule is **strict** in every supported dialect (`strict`, `autocad-2026`, `bricscad-v26`). No host product is known to expose Lisp-2 semantics; the conformance contract treats any Lisp-2-style behaviour as a regression.

7.2.5 Cross-references

- Special Form Entry: `SETQ` (chapter 3) — assigns a value to the single binding cell.
- Special Form Entry: `DEFUN` (chapter 3) — assigns a function value to the single binding cell.
- Special Form Entry: `DEFUN-Q` (chapter 3) — single-cell write plus list-accessible compatibility metadata.
- Special Form Entry: `LAMBDA`, `FUNCTION` (chapter 3) — produce function values that participate in the same cell.

- Function Entry: VL-SYMBOL-VALUE (this chapter) — returns the single cell’s value.
- Function Entry: BOUNDP (this chapter) — predicates the single cell’s bound state.

7.2.6 Source Notes

- [About Symbols and Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP/files/GUID-1855E7B0-2474-49E6-9EE3-8BC54...htm>)
- [About Local and Global Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP/files/GUID-A8045F62-1CE3-4022-ACC1-CC0E2...htm>)
- [About Using Defun to Define a Function (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP/files/GUID-9EDF48C2-1678-4DC3-B...htm>)
- BricsCAD V26 release-note defect SR44723 ((setq myfunc eq) callable as function).
- Phase-6 BricsCAD V26 / macOS namespace probe — [results/bricscad/macos/20260426T1555](#)

7.3 Variable Entry: T

7.3.1 Name

‘T’

7.3.2 Class

Predefined Variable

7.3.3 Value Type

Distinguished true value.

7.3.4 Initial Value

Itself.

7.3.5 Description

Canonical non-‘nil’ truth value.

7.3.6 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.3.7 Source Notes

- [About Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP/files/GUID-CC89231D-E10F-47D0-A1FD-A989D24C7105.htm>)
- Status: documented in practical terms.

7.4 Variable Entry: PI

7.4.1 Name

‘PI’

7.4.2 Class

Predefined Variable

7.4.3 Value Type

Real number.

7.4.4 Initial Value

Approximation of pi.

7.4.5 Description

Predefined mathematical constant.

7.4.6 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.4.7 Source Notes

- [About Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP/files/GUID-CC89231D-E10F-47D0-A1FD-A989D24C7105.htm>)
- Status: documented.

7.5 Variable Entry: PAUSE

7.5.1 Name

‘PAUSE’

7.5.2 Class

Predefined Variable

7.5.3 Description

Special interaction token used in command invocation contexts.

7.5.4 Compatibility

Host-interaction-specific.

7.5.5 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.5.6 Source Notes

- [About Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP/files/GUID-CC89231D-E10F-47D0-A1FD-A989D24C7105.htm>)
- Status: documented at variable overview level.

7.6 Source Notes

- [About Symbols and Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP/files/GUID-1855E7B0-2474-49E6-9EE3-8BC54.htm>)
- [About Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP/files/GUID-CC89231D-E10F-47D0-A1FD-A989D24C7105.htm>)

7.7 Function Entry: ATOM

7.7.1 Name

`'atom'`

7.7.2 Class

Function

7.7.3 Syntax

`(atom item)`

7.7.4 Arguments and Values

- `'item'`: any value

7.7.5 Description

Returns `'T'` if `'item'` is not a list; otherwise returns `'nil'`.

7.7.6 Return Values

- `'T'` if `'item'` is not a list
- `'nil'` otherwise

7.7.7 Side Effects

None.

7.7.8 Affected By

- the AutoLISP distinction between lists and non-lists

7.7.9 Exceptional Situations

None specific are documented.

7.7.10 Examples

- ‘(atom 1) => T‘
- ‘(atom '(a b)) => nil‘
- ‘(atom nil) => T‘

7.7.11 Compatibility

Autodesk explicitly notes that some other Lisp dialects use different conventions, so this behavior should not be silently replaced by non-AutoLISP semantics.

7.7.12 Notes

Because ‘nil‘ is also the empty list in other AutoLISP contexts, the ‘atom‘ result on ‘nil‘ is particularly important and should be tested against target hosts.

7.7.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.7.14 Source Notes

- [atom (AutoLISP)](<https://help.autodesk.com/cloudhelp/2019/KOR/AutoCAD-AutoLISP-Reference/files/GUID-3B74E28D-7C3B-4409-9DFE-0305E9881560.htm>)
- Status: documented.

7.8 Function Entry: BOUNDP

7.8.1 Name

`'boundp'`

7.8.2 Class

Function

7.8.3 Syntax

`(boundp sym)`

7.8.4 Arguments and Values

- `'sym'`: symbol

7.8.5 Description

Tests whether a symbol has a value bound to it.

7.8.6 Return Values

- `'T'` if the symbol has a value
- `'nil'` otherwise

Autodesk explicitly states that if no value is bound to `'sym'`, or if `'sym'` has been bound to `'nil'`, `'boundp'` returns `'nil'`.

It also states that if `'sym'` is an undefined symbol, it is automatically created and assigned `'nil'`.

7.8.7 Side Effects

None.

7.8.8 Exceptional Situations

Erroneous if the argument is not acceptable as a symbol designator under host semantics.

7.8.9 Examples

- ‘(boundp 'a)’

7.8.10 Compatibility

Useful for distinguishing unassigned symbols from symbols intentionally set to ‘nil’.

7.8.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.8.12 Source Notes

- [boundp (AutoLISP)](<https://help.autodesk.com/cloudhelp/2021/ENU/AutoCAD-AutoLISP-Reference/files/GUID-F8491426-8505-4390-825F-CACE15EBB48F.htm>)
- Status: documented.

7.9 Function Entry: NULL

7.9.1 Name

`'null'`

7.9.2 Class

Function

7.9.3 Syntax

`(null expr)`

7.9.4 Arguments and Values

- `'expr'`: expression

7.9.5 Description

Tests whether `'expr'` evaluates to `'nil'`.

7.9.6 Return Values

- `'T'` if the expression evaluates to `'nil'`
- `'nil'` otherwise

7.9.7 Side Effects

Side effects are those of evaluating `'expr'`.

7.9.8 Exceptional Situations

Any error arises from evaluating `'expr'`.

7.9.9 Examples

- `'(null nil) => T'`
- `'(null 1) => nil'`

7.9.10 Compatibility

Closely related to `'not'`, but documented separately.

7.9.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.9.12 Source Notes

- [null (AutoLISP)](<https://help.autodesk.com/cloudhelp/2016/DEU/AutoCAD-AutoLISP/files/GUID-2CA2E5F1-297F-4ECA-9500-5FFCD4877126.htm>)
- Status: documented.

7.10 Function Entry: NOT

7.10.1 Name

`'not'`

7.10.2 Class

Function

7.10.3 Syntax

`(not expr)`

7.10.4 Arguments and Values

- `'expr'`: expression

7.10.5 Description

Tests whether `'expr'` evaluates to `'nil'`.

7.10.6 Return Values

- `'T'` if the expression evaluates to `'nil'`
- `'nil'` otherwise

7.10.7 Side Effects

Side effects are those of evaluating `'expr'`.

7.10.8 Exceptional Situations

Any error arises from evaluating `'expr'`.

7.10.9 Examples

- `'(not nil) => T'`
- `'(not 1) => nil'`

7.10.10 Compatibility

Behaviorally aligned with `'null'` in documented AutoLISP usage.

7.10.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.10.12 Source Notes

- [not (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ESP/AutoCAD-MAC-AutoLISP-Reference/files/GUID-991AD6A0-61AA-45C9-8C27-CADF9F36BE71.htm>)
- Status: documented.

7.11 Function Entry: TYPE

7.11.1 Name

`'type'`

7.11.2 Class

Function

7.11.3 Syntax

`(type item)`

7.11.4 Arguments and Values

- `'item'`: any value

7.11.5 Description

Returns a symbol describing the runtime type category of `'item'`, or `'nil'` for items that evaluate to `'nil'`.

7.11.6 Return Values

Documented return values include symbols such as:

- `'INT'`
- `'REAL'`
- `'STR'`
- `'LIST'`
- `'SYM'`
- `'FILE'`
- `'ENAME'`
- `'PICKSET'`
- `'SUBR'`
- `'USUBR'`

- ‘SAFEARRAY‘
- ‘VARIANT‘
- ‘VLA-object‘

and ‘nil‘ for values that evaluate to ‘nil‘.

7.11.7 Side Effects

None.

7.11.8 Exceptional Situations

No special exceptional situations are documented.

7.11.9 Examples

- ‘(type 1) => INT‘
- ‘(type "x") => STR‘
- ‘(type nil) => nil‘

7.11.10 Compatibility

This function is one of the strongest direct sources for the practical runtime type model of AutoLISP.

7.11.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.11.12 Source Notes

- [type (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-506C9CC8-B0BD-4A4C-B4C2-006750504509.htm>)
- Status: documented.

7.12 Function Entry: VL-SYMBOLP

7.12.1 Name

`'vl-symbolp'`

7.12.2 Class

Visual LISP Function

7.12.3 Syntax

`(vl-symbolp obj)`

7.12.4 Arguments and Values

- `'obj'`: any value

7.12.5 Description

Tests whether `'obj'` is a symbol.

7.12.6 Return Values

- `'T'` if `'obj'` is a symbol
- `'nil'` otherwise

Autodesk explicitly documents that:

- `'(vl-symbolp t) => T'`
- `'(vl-symbolp nil) => nil'`

7.12.7 Side Effects

None.

7.12.8 Compatibility

This entry is particularly important because its treatment of `'nil'` is easy to get wrong if one projects Common Lisp assumptions into AutoLISP.

7.12.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.12.10 Source Notes

- [vl-symbolp (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-F51AFE0A-CA6C-428D-8E8D-3607BF551991.html>)
- Status: documented.

7.13 Function Entry: VL-SYMBOL-NAME

7.13.1 Name

`'vl-symbol-name'`

7.13.2 Class

Visual LISP Function

7.13.3 Syntax

`(vl-symbol-name sym)`

7.13.4 Arguments and Values

- `'sym'`: symbol

7.13.5 Description

Returns the string name of a symbol.

7.13.6 Return Values

String corresponding to the symbol name.

7.13.7 Side Effects

None.

7.13.8 Exceptional Situations

Erroneous if the argument is not a symbol acceptable to the active host semantics.

7.13.9 Compatibility

Belongs to the Visual LISP reflection/introspection layer.

7.13.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.13.11 Source Notes

- [Symbol-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-796EFF94-E904-4F5B-B120-C6D2DCD8646A.htm>)
- Status: documented by family summary; dedicated page still desirable.

7.14 Function Entry: VL-SYMBOL-VALUE

7.14.1 Name

`'vl-symbol-value'`

7.14.2 Class

Visual LISP Function

7.14.3 Syntax

`(vl-symbol-value sym)`

7.14.4 Arguments and Values

- `'sym'`: symbol

7.14.5 Description

Returns the current value of the symbol.

7.14.6 Return Values

The symbol's value.

7.14.7 Side Effects

None beyond any host-defined lookup effects.

7.14.8 Exceptional Situations

Under-specified in the current source set for unbound-symbol behavior; this should be test-qualified.

7.14.9 Compatibility

Useful as a reflective counterpart to ordinary variable evaluation.

7.14.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.14.11 Source Notes

- [vl-symbol-value (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP-Reference/files/GUID-5DAAABCF-7210-4A90-809D-6C27CCEF6000.html>)
- Status: documented, with some behavioral edge cases still to be tested.

7.15 Dictionary Coverage Index: Symbol-Handling Functions

The Autodesk symbol-handling function family includes at least:

- ‘atom‘
- ‘atoms-family‘
- ‘boundp‘
- ‘not‘
- ‘null‘
- ‘numberp‘
- ‘quote‘
- ‘set‘
- ‘setq‘
- ‘type‘
- ‘vl-symbol-name‘
- ‘vl-symbol-value‘
- ‘vl-symbolp‘

Source:

- [Symbol-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-796EFF94-E904-4F5B-B120-C6D2DCD8646A.htm>)

7.16 Function Entry: ATOMS-FAMILY

7.16.1 Name

`'atoms-family'`

7.16.2 Class

Function

7.16.3 Syntax

`(atoms-family format [symlist])`

7.16.4 Arguments and Values

- `'format'`: integer; `'0'` to return symbols as a list of symbol objects, `'1'` to return them as a list of strings.
- `'symlist'` (optional): a list of strings naming symbols to look up. When supplied, the result has the same length as `'symlist'`; for each name, the corresponding result element is the symbol (or its string name) if defined, or `'nil'` if undefined.

7.16.5 Description

Returns a list of currently defined symbols in the active document namespace. With only `'format'`, returns every defined symbol. With `'format'` and `'symlist'`, returns only entries for the named symbols, in the same order, with `'nil'` for any name that is not currently defined.

7.16.6 Return Values

A list whose elements are either symbols or strings, depending on `'format'`.

7.16.7 Side Effects

None.

7.16.8 Examples

- ‘(atoms-family 0)’ returns every defined symbol as a list of symbols.
- ‘(atoms-family 1 ’(”CAR” ”CDR” ”XYZ”))’ returns ‘(”CAR” ”CDR” nil)’ if ‘XYZ’ is not defined.

7.16.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

7.16.10 Source Notes

- [atoms-family (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-0B35850D-9E62-446B-BAA2-0598A4BDE111.html>)
- Status: documented (Phase 3 body fill).

7.17 Function Entry: CLAUTOLISP-DOCUMENTATION

7.17.1 Name

`'clautolisp-documentation'`

7.17.2 Class

Function

7.17.3 Syntax

`(clautolisp-documentation name)`

7.17.4 Arguments and Values

- `'name'`: a symbol or a string naming the symbol. Strings are coerced to the symbol's uppercased name; symbols are used directly.

7.17.5 Description

Returns the documentation string attached to the innermost binding cell for `'name'` in the current dynamic frame chain. Documentation is captured at read time: a `';| ... |'` block comment immediately preceding a `'(defun NAME ...)'` or `'(setq NAME ...)'` form becomes the binding's documentation when that form is evaluated.

Because `'clautolisp'` is dynamically scoped and the documentation lives on the binding cell (not the symbol), the result follows the same shadowing rules as value lookup: a `'/-local`, lambda parameter, or `'foreach'` iteration variable that rebinds the name carries its own documentation slot that is restored when the rebinding frame is popped.

The doc-update rules differ between `defun` and `setq`. `'(defun NAME ...)'` always rewrites the doc — installing the new block-comment text or `'nil'` when no block preceded. `'(setq NAME ...)'` rewrites only conditionally: a preceding block installs the new text; without one, an existing function-doc (from an earlier `'defun'`) is cleared (the binding is no longer that function) but an existing variable-doc is preserved (plain mutation does not erase a variable's description).

7.17.6 Return Values

A string when documentation has been recorded for the resolved binding; `'nil'` otherwise (no recorded doc, or no binding for `'name'`).

7.17.7 Side Effects

None.

7.17.8 Examples

- After `'| greet user |; (defun foo (n) (princ n))'`, `'(clautolisp-documentation 'foo)'` returns `" greet user "`.
- After `'(setq foo 42)'` following the above, `'(clautolisp-documentation 'foo)'` returns `'nil'` — the bare `'setq'` cleared the now-stale function-doc.
- After `'| counter |; (setq c 0)'` then `'(setq c (1+ c))'`, `'(clautolisp-documentation 'c)'` returns `" counter "` — plain mutation preserves a variable-doc.
- `'(clautolisp-documentation 'no-such-binding)'` returns `'nil'`.

7.17.9 Availability

- AutoCAD 2026: not documented.
- BricsCAD V26: not documented.
- clautolisp: implemented.
- Status: clautolisp-only extension; the AutoCAD / BricsCAD vendor surface offers no comparable per-binding documentation API.

7.17.10 Source Notes

- No vendor reference. Specified by the `'source-aware-defun-documentation'` issue in this repository; implemented in `'autolisp-builtins-core/source/api.lisp'`.

7.18 Function Entry: CLAUTOLISP-DOCUMENTATION-KIND

7.18.1 Name

`'clautolisp-documentation-kind'`

7.18.2 Class

Function

7.18.3 Syntax

`(clautolisp-documentation-kind name)`

7.18.4 Arguments and Values

- `'name'`: a symbol or a string, as for `'clautolisp-documentation'`.

7.18.5 Description

Returns the **kind** of the documentation attached to `'name'`'s innermost binding: the symbol `'function'` when the binding was last documented by a `'defun'`, the symbol `'variable'` when it was last documented by a `'setq'`, or `'nil'` when no documentation is recorded.

The kind reflects the most recent **documented** assignment, not the current value's runtime type. A binding documented by `'(defun foo ...)'` reports `'function'` even after a subsequent bare `'(setq foo 42)'` only if the function-doc has not been cleared (i.e., another doc-block-bearing form re-documented it as a variable in between).

This builtin is the companion to `'clautolisp-documentation'`; tooling that pretty-prints documentation (such as `'alref-describe'`) uses both to render a `'Function: ...'` or `'Variable: ...'` prefix.

7.18.6 Return Values

The symbol `'function'`, the symbol `'variable'`, or `'nil'`.

7.18.7 Side Effects

None.

7.18.8 Examples

- After ‘`| doc | (defun foo () 1)`’, ‘`(clautolisp-documentation-kind 'foo)`’ returns ‘`function`’.
- After ‘`| doc | (setq bar 0)`’, ‘`(clautolisp-documentation-kind 'bar)`’ returns ‘`variable`’.
- ‘`(clautolisp-documentation-kind 'no-such-binding)`’ returns ‘`nil`’.

7.18.9 Availability

- AutoCAD 2026: not documented.
- BricsCAD V26: not documented.
- clautolisp: implemented.
- Status: clautolisp-only extension; companion to ‘`clautolisp-documentation`’.

7.18.10 Source Notes

- No vendor reference. Specified by the ‘`source-aware-defun-documentation`’ issue in this repository.

8 8 Numbers

8.1 Overview

AutoLISP numbers are divided primarily into integers and reals.

8.2 Normative Rules

- integers are 32-bit signed values in documented modern Autodesk semantics
- reals are double-precision floating-point values
- integer overflow in literal reading may produce a real
- arithmetic overflow on integers is invalid

8.3 Source Notes

- [About Integers (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/FRA/AutoCAD-MAC-AutoLisp/files/GUID-EF6114FC-F1E4-4C71-91CC-07D01E6C8ABB.htm>)
- [About Reals (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP/files/GUID-F2BFD097-295B-4499-BBD9-0A785C377070.htm>)

8.4 Function Entry: +

8.4.1 Name

‘+’

8.4.2 Class

Function

8.4.3 Syntax

(+ [number ...])

8.4.4 Arguments and Values

- zero or more numeric arguments

8.4.5 Description

Returns the sum of the arguments.

8.4.6 Return Values

Numeric sum.

8.4.7 Side Effects

None.

8.4.8 Exceptional Situations

Erroneous for non-numeric arguments or host-specific numeric overflow conditions.

8.4.9 Examples

- ‘(+ 1 2 3) => 6’

8.4.10 Compatibility

Core arithmetic primitive across dialects.

8.4.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.4.12 Source Notes

- [Non-alphabetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-EBDC1072-48BF-4204-80B1-73430DBF58E1.htm>)
- Status: documented by family summary; detailed corner cases still to be tightened.

8.5 Function Entry: -

8.5.1 Name

‘-’

8.5.2 Class

Function

8.5.3 Syntax

(- number [number ...])

8.5.4 Description

Subtracts the second and following arguments from the first.

8.5.5 Return Values

Numeric difference.

8.5.6 Side Effects

None.

8.5.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.5.8 Source Notes

- [Non-alphabetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-EBDC1072-48BF-4204-80B1-73430DBF58E1.htm>)
- Status: documented by family summary.

8.6 Function Entry: *

8.6.1 Name

‘*‘

8.6.2 Class

Function

8.6.3 Syntax

(* [number ...])

8.6.4 Description

Returns the product of the arguments.

8.6.5 Return Values

Numeric product.

8.6.6 Side Effects

None.

8.6.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.6.8 Source Notes

- [Non-alphabetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-EBDC1072-48BF-4204-80B1-73430DBF58E1.htm>)
- Status: documented by family summary.

8.7 Function Entry: /

8.7.1 Name

`‘/’`

8.7.2 Class

Function

8.7.3 Syntax

`(/ number [number ...])`

8.7.4 Description

Divides the first argument by the product of the remaining arguments.

8.7.5 Return Values

Numeric quotient.

8.7.6 Side Effects

None.

8.7.7 Exceptional Situations

Division by zero and invalid numeric arguments are erroneous.

8.7.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.7.9 Source Notes

- [Non-alphabetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-EBDC1072-48BF-4204-80B1-73430DBF58E1.htm>)
- Status: documented by family summary.

8.8 Function Entry: =

8.8.1 Name

`'='`

8.8.2 Class

Function

8.8.3 Syntax

`(= [expr ...])`

8.8.4 Arguments and Values

- one or more expressions acceptable to numeric/string comparison semantics

8.8.5 Description

Compares arguments for equality under AutoLISP comparison semantics.

8.8.6 Return Values

- `'T'` if all arguments compare equal
- `'nil'` otherwise

8.8.7 Side Effects

Side effects are those of evaluating the argument expressions.

8.8.8 Compatibility

Autodesk comparison functions accept strings as well as numbers in documented summaries, so this should not be reduced to a purely numeric predicate without tighter source confirmation.

8.8.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.8.10 Source Notes

- [Non-alphabetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-EBDC1072-48BF-4204-80B1-73430DBF58E1.htm>)
- [Equality and Conditional Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-C6A7FC58-7A07-4650-9D59-12CAFD194FDA.htm>)
- Status: documented by family summary; exact domain rules should be tightened.

8.9 Function Entry: /=

8.9.1 Name

`'/='`

8.9.2 Class

Function

8.9.3 Syntax

`(/= [expr ...])`

8.9.4 Description

Compares arguments for inequality under AutoLISP comparison semantics.

8.9.5 Return Values

- `'T'` if the arguments are pairwise unequal under the active semantics
- `'nil'` otherwise

8.9.6 Side Effects

Side effects are those of evaluating the arguments.

8.9.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.9.8 Source Notes

- [Non-alphabetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-EBDC1072-48BF-4204-80B1-73430DBF58E1.htm>)
- Status: documented by family summary.

8.10 Function Entry: <

8.10.1 Name

`<`

8.10.2 Class

Function

8.10.3 Syntax

`(< [expr ...])`

8.10.4 Description

Returns `T` if each argument is less than the argument to its right.

8.10.5 Return Values

- `T` if the ordered comparison succeeds
- `nil` otherwise

8.10.6 Side Effects

Side effects are those of evaluating the arguments.

8.10.7 Examples

- `(< 1 2 3) => T`

8.10.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.10.9 Source Notes

- [`< (AutoLISP)`](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-67C8900F-0491-4D91-AA05-9136639E7424.htm>)
- Status: documented.

8.11 Function Entry: <=

8.11.1 Name

'<='

8.11.2 Class

Function

8.11.3 Syntax

(<= [expr ...])

8.11.4 Description

Returns 'T' if each argument is less than or equal to the argument to its right.

8.11.5 Return Values

- 'T' on success
- 'nil' otherwise

8.11.6 Side Effects

Side effects are those of evaluating the arguments.

8.11.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.11.8 Source Notes

- [`<=` (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-66B7B146-E279-4C22-9C0A-1F312166C15B.htm>)
- Status: documented.

8.12 Function Entry: >

8.12.1 Name

'>'

8.12.2 Class

Function

8.12.3 Syntax

(> [expr ...])

8.12.4 Description

Returns 'T' if each argument is greater than the argument to its right.

8.12.5 Return Values

- 'T' on success
- 'nil' otherwise

8.12.6 Side Effects

Side effects are those of evaluating the arguments.

8.12.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.12.8 Source Notes

- [> (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ITA/AutoCAD-LT-AutoLISP-Reference/files/GUID-DAB2660E-AB9D-425D-B0C5-C5774164ADCD.htm>)
- Status: documented.

8.13 Function Entry: `>=`

8.13.1 Name

`'>='`

8.13.2 Class

Function

8.13.3 Syntax

`(>= [expr ...])`

8.13.4 Description

Returns `'T'` if each argument is greater than or equal to the argument to its right.

8.13.5 Return Values

- `'T'` on success
- `'nil'` otherwise

8.13.6 Side Effects

Side effects are those of evaluating the arguments.

8.13.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.13.8 Source Notes

- [Non-alphabetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-EBDC1072-48BF-4204-80B1-73430DBF58E1.htm>)
- Status: documented by family summary.

8.14 Function Entry: ABS

8.14.1 Name

‘abs‘

8.14.2 Class

Function

8.14.3 Syntax

(abs number)

8.14.4 Description

Returns the absolute value of a number.

8.14.5 Return Values

Non-negative number of the same general magnitude.

8.14.6 Side Effects

None.

8.14.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.14.8 Source Notes

- [A Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/HUN/AutoCAD-AutoLISP-Reference/files/GUID-985B33C7-0A4B-4FBE-8DC2-htm>)
- Status: documented by alphabetic summary.

8.15 Function Entry: FIX

8.15.1 Name

‘fix‘

8.15.2 Class

Function

8.15.3 Syntax

(fix number)

8.15.4 Description

Truncates a real to an integer.

8.15.5 Return Values

Integer value.

8.15.6 Compatibility

The documentation on integers and reals makes ‘fix‘ especially relevant to the integer-range model.

8.15.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.15.8 Source Notes

- [fix (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-93B5F13B-348E-49E0-A116-0861684506D5.htm>)
- Status: documented.

8.16 Function Entry: FLOAT

8.16.1 Name

‘float‘

8.16.2 Class

Function

8.16.3 Syntax

(float number)

8.16.4 Description

Converts a numeric value to a real.

8.16.5 Return Values

Real value.

8.16.6 Side Effects

None.

8.16.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.16.8 Source Notes

- [zerop (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-3D509EE1-A809-4B78-915C-28ECF483BB72.htm>)
- Status: documented.

8.17 Function Entry: ZEROP

8.17.1 Name

‘zerop’

8.17.2 Class

Function

8.17.3 Syntax

(zerop number)

8.17.4 Description

Tests whether a number is zero.

8.17.5 Return Values

- ‘T’ if the number is zero
- ‘nil’ otherwise

8.17.6 Side Effects

None.

8.17.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.17.8 Source Notes

- [Arithmetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP-Reference/files/GUID-FC14467C-63B9-4400-htm>)
- Status: documented by Autodesk’s arithmetic reference table.

8.18 Function Entry: MINUSP

8.18.1 Name

`'minusp'`

8.18.2 Class

Function

8.18.3 Syntax

`(minusp number)`

8.18.4 Description

Tests whether a number is negative.

8.18.5 Return Values

- `'T'` if the number is negative
- `'nil'` otherwise

8.18.6 Side Effects

None.

8.18.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.18.8 Source Notes

- [Arithmetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP-Reference/files/GUID-FC14467C-63B9-4400-htm>)
- Status: documented by Autodesk's arithmetic reference table.

8.19 Dictionary Coverage Index: Arithmetic and Numeric Functions

The Autodesk arithmetic function family includes at least:

- ‘+’
- ‘-’
- ‘*’
- ‘/’
- ‘~’
- ‘1+’
- ‘1-’
- ‘abs’
- ‘atan’
- ‘cos’
- ‘fix’
- ‘float’
- ‘gcd’
- ‘log’
- ‘logand’
- ‘logior’
- ‘lsh’
- ‘max’
- ‘min’
- ‘minusp’
- ‘rem’
- ‘sin’

- ‘sqrt‘
- ‘zerop‘

The equality and conditional family contributes additional numeric predicates and comparisons:

- ‘=‘
- ‘/=‘
- ‘<‘
- ‘<=‘
- ‘>‘
- ‘>=‘
- ‘boole‘
- ‘eq‘
- ‘equal‘

Sources:

- [Arithmetic Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ENU/AutoCAD-MAC-AutoLISP-Reference/files/GUID-FC14467C-63B9-4400-8C6A-266D21C846AE.htm>)
- [Equality and Conditional Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-C6A7FC58-7A07-4650-9D59-12CAFD194FDA.htm>)

8.20 Function Entry: ATAN

8.20.1 Name

`'atan'`

8.20.2 Class

Function

8.20.3 Syntax

`(atan num1 [num2])`

8.20.4 Arguments and Values

- `'num1'`: integer or real.
- `'num2'` (optional): integer or real.

8.20.5 Description

Returns the arctangent in radians. With one argument, returns the arctangent of `'num1'`. With two arguments, returns the arctangent of `'num1/num2'` and uses the signs of both arguments to choose the correct quadrant. If `'num2'` is zero, the result is $+\pi/2$ or $-\pi/2$ depending on the sign of `'num1'`.

8.20.6 Return Values

A real number in the range $[-\pi/2, +\pi/2]$ for the one-argument form, and in $(-\pi, +\pi]$ for the two-argument form.

8.20.7 Side Effects

None.

8.20.8 Examples

- `'(atan 1)'` returns `'0.785398'`.
- `'(atan 0.5)'` returns `'0.463648'`.
- `'(atan -1.0)'` returns `'-0.785398'`.
- `'(atan 2.0 3.0)'` returns `'0.588003'`.

- ‘(atan 2.0 -3.0)’ returns ‘2.55359’.
- ‘(atan 1.0 0.0)’ returns ‘1.5708’.

8.20.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.20.10 Source Notes

- [atan (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-1A40A16E-8ABE-437D-9888-2F43CEA0B5CE.htm>)
- Status: documented (Phase 3 body fill).

8.21 Function Entry: COS

8.21.1 Name

‘cos‘

8.21.2 Class

Function

8.21.3 Syntax

(cos ang)

8.21.4 Arguments and Values

- ‘ang’: integer or real, an angle expressed in radians.

8.21.5 Description

Returns the cosine of ‘ang‘.

8.21.6 Return Values

A real number.

8.21.7 Side Effects

None.

8.21.8 Examples

- ‘(cos 0.0)’ returns ‘1.0‘.
- ‘(cos pi)’ returns ‘-1.0‘.

8.21.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.21.10 Source Notes

- [cos (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-5F39D690-A986-498D-929A-56BC891E08CD.htm>)
- Status: documented (Phase 3 body fill).

8.22 Function Entry: SIN

8.22.1 Name

`'sin'`

8.22.2 Class

Function

8.22.3 Syntax

`(sin ang)`

8.22.4 Arguments and Values

- `'ang'`: integer or real, an angle expressed in radians.

8.22.5 Description

Returns the sine of `'ang'`.

8.22.6 Return Values

A real number.

8.22.7 Side Effects

None.

8.22.8 Examples

- `'(sin 1.0)'` returns `'0.841471'`.
- `'(sin 0.0)'` returns `'0.0'`.

8.22.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.22.10 Source Notes

- [sin (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-83677851-6FA5-496E-A93C-AEA55ABCB527.htm>)
- Status: documented (Phase 3 body fill).

8.23 Function Entry: EXP

8.23.1 Name

`'exp'`

8.23.2 Class

Function

8.23.3 Syntax

`(exp num)`

8.23.4 Arguments and Values

- `'num'`: integer or real, the exponent to which the natural constant `'e'` is raised.

8.23.5 Description

Returns `'e'` raised to the power `'num'` (the natural antilogarithm).

8.23.6 Return Values

A real number.

8.23.7 Side Effects

None.

8.23.8 Examples

- `'(exp 1.0)'` returns `'2.71828'`.
- `'(exp 2.2)'` returns `'9.02501'`.
- `'(exp -0.4)'` returns `'0.67032'`.

8.23.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.23.10 Source Notes

- [exp (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-FD0C918B-A162-4939-9F4E-FFE8863C3FC8.htm>)
- Status: documented (Phase 3 body fill).

8.24 Function Entry: EXPT

8.24.1 Name

`'expt'`

8.24.2 Class

Function

8.24.3 Syntax

`(expt number power)`

8.24.4 Arguments and Values

- `'number'`: integer or real, the base.
- `'power'`: integer or real, the exponent.

8.24.5 Description

Returns `'number'` raised to the power `'power'`.

8.24.6 Return Values

If both arguments are integers, the result is an integer; otherwise the result is a real.

8.24.7 Side Effects

None.

8.24.8 Examples

- `'(expt 2 4)'` returns `'16'`.
- `'(expt 3.0 2.0)'` returns `'9.0'`.

8.24.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: documented under the BricsCAD-only name ‘power’ **and** presumed to support ‘expt’. The BricsCAD analogue is recorded separately in chapter 24 — see Phase 4 cross-reference.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.24.10 See Also

- BricsCAD analogue: **power** (chapter 24). AutoCAD users portably write ‘(expt base exp)’; BricsCAD users may use either.

8.24.11 Source Notes

- [expt (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-FCAC3150-7491-43DB-8042-37EDE3791A96.htm>)
- Status: documented (Phase 3 body fill).

8.25 Function Entry: LOG

8.25.1 Name

`'log'`

8.25.2 Class

Function

8.25.3 Syntax

`(log num)`

8.25.4 Arguments and Values

- `'num'`: integer or real, a positive number.

8.25.5 Description

Returns the natural logarithm of `'num'` (base `'e'`). The base is fixed; AutoLISP does not provide a multi-argument form.

8.25.6 Return Values

A real number.

8.25.7 Side Effects

None.

8.25.8 Examples

- `'(log 4.5)'` returns `'1.50408'`.
- `'(log 1.22)'` returns `'0.198851'`.

8.25.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible. The BricsCAD-only `'log10'` is documented separately in chapter 24.

- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.25.10 Source Notes

- [log (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4B1B6364-B1CB-4CE7-8C8F-706EA829780F.htm>)
- Status: documented (Phase 3 body fill).

8.26 Function Entry: SQRT

8.26.1 Name

`'sqrt'`

8.26.2 Class

Function

8.26.3 Syntax

`(sqrt num)`

8.26.4 Arguments and Values

- `'num'`: integer or real, a non-negative number.

8.26.5 Description

Returns the square root of `'num'` as a real number.

8.26.6 Return Values

A real number.

8.26.7 Side Effects

None.

8.26.8 Examples

- `'(sqrt 4)'` returns `'2.0'`.
- `'(sqrt 2.0)'` returns `'1.41421'`.

8.26.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.26.10 Source Notes

- [sqrt (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-B8D3BA69-95FE-4036-83BA-6AFE45BADE5.htm>)
- Status: documented (Phase 3 body fill).

8.27 Function Entry: MAX

8.27.1 Name

`'max'`

8.27.2 Class

Function

8.27.3 Syntax

`(max [number number ...])`

8.27.4 Arguments and Values

- `'number'`: zero or more integers or reals.

8.27.5 Description

Returns the largest of the supplied numbers. With no arguments, returns 0.

8.27.6 Return Values

If any argument is a real, the result is a real; otherwise the result is an integer.

8.27.7 Side Effects

None.

8.27.8 Examples

- `'(max 4.07 -144)'` returns `'4.07'`.
- `'(max -88 19 5 2)'` returns `'19'`.
- `'(max 2.1 4 8)'` returns `'8.0'`.

8.27.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.27.10 Source Notes

- [max (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-057C56D3-8A15-406B-95CA-68DD859E64FB.htm>)
- Status: documented (Phase 3 body fill).

8.28 Function Entry: MIN

8.28.1 Name

`'min'`

8.28.2 Class

Function

8.28.3 Syntax

`(min [number number ...])`

8.28.4 Arguments and Values

- `'number'`: zero or more integers or reals.

8.28.5 Description

Returns the smallest of the supplied numbers. With no arguments, returns 0.

8.28.6 Return Values

If any argument is a real, the result is a real; otherwise the result is an integer.

8.28.7 Side Effects

None.

8.28.8 Examples

- `'(min 683 -10.0)'` returns `'-10.0'`.
- `'(min 73 2 48 5)'` returns `'2'`.
- `'(min 73.0 2 48 5)'` returns `'2.0'`.
- `'(min 2 4 6.7)'` returns `'2.0'`.

8.28.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.28.10 Source Notes

- [min (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-F401AD55-57B2-4B1A-9704-D7D8D45AE465.htm>)
- Status: documented (Phase 3 body fill).

8.29 Function Entry: GCD

8.29.1 Name

`'gcd'`

8.29.2 Class

Function

8.29.3 Syntax

`(gcd int1 int2)`

8.29.4 Arguments and Values

- `'int1'`: integer greater than 0.
- `'int2'`: integer greater than 0.

8.29.5 Description

Returns the greatest common divisor of `'int1'` and `'int2'`.

8.29.6 Return Values

An integer.

8.29.7 Side Effects

None.

8.29.8 Examples

- `'(gcd 81 57)'` returns `'3'`.
- `'(gcd 12 20)'` returns `'4'`.

8.29.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.29.10 Source Notes

- [gcd (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-3F748964-B6D7-4136-92D5-81A7DD02ED4A.htm>)
- Status: documented (Phase 3 body fill).

8.30 Function Entry: REM

8.30.1 Name

`'rem'`

8.30.2 Class

Function

8.30.3 Syntax

`(rem [number number ...])`

8.30.4 Arguments and Values

- `'number'`: zero or more integers or reals.

8.30.5 Description

Divides the first number by the second, and returns the remainder. With more than two arguments, evaluates left to right: the remainder of `'(rem n1 n2)'` is divided by `'n3'`, and so on. With no arguments, returns 0; with a single argument, returns that argument.

8.30.6 Return Values

If any argument is a real, the result is a real; otherwise the result is an integer.

8.30.7 Side Effects

None.

8.30.8 Examples

- `'(rem 42 12)'` returns `'6'`.
- `'(rem 12.0 16)'` returns `'12.0'`.
- `'(rem 26 7 2)'` returns `'1'` ($(26 \bmod 7) = 5$, then $(5 \bmod 2) = 1$).

8.30.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: documented under the same name ‘rem’ and additionally provides ‘mod’. See Phase 4 cross-reference to BricsCAD ‘mod’.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.30.10 Source Notes

- [rem (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4CC43BEB-3215-41AE-839B-2DCE6354241A.htm>)
- Status: documented (Phase 3 body fill).

8.31 Function Entry: NUMBERP

8.31.1 Name

`'numberp'`

8.31.2 Class

Function

8.31.3 Syntax

`(numberp item)`

8.31.4 Arguments and Values

- `'item'`: any AutoLISP value (integer, real, string, list, subroutine, `ename`, `T`, or `nil`).

8.31.5 Description

Verifies that `'item'` is a numeric value, that is, an integer or a real.

8.31.6 Return Values

Returns `'T'` if `'item'` is a real or integer; otherwise returns `'nil'`.

8.31.7 Side Effects

None.

8.31.8 Examples

- `'(numberp 4)'` returns `'T'`.
- `'(numberp 3.8348)'` returns `'T'`.
- `'(numberp "Howdy")'` returns `'nil'`.
- After `'(setq a 123)'`, `'(numberp a)'` returns `'T'`.

8.31.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.31.10 Source Notes

- [numberp (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-1E5B591C-F60B-43CB-8CD8-E729D72B1...htm>)
- Status: documented (Phase 3 body fill).

8.32 Function Entry: 1+

8.32.1 Name

`'1+'`

8.32.2 Class

Function

8.32.3 Syntax

`(1+ number)`

8.32.4 Arguments and Values

- `'number'`: integer or real.

8.32.5 Description

Increments `'number'` by 1.

8.32.6 Return Values

The input number plus 1. Return type matches input type: integer for integer input, real for real input.

8.32.7 Side Effects

None.

8.32.8 Examples

- `'(1+ 5)'` returns `'6'`.
- `'(1+ -17.5)'` returns `'-16.5'`.

8.32.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.32.10 Source Notes

- [1+ (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-EF06615A-5BE5-4A86-96CF-643B0712E43A.htm>)
- Status: documented (Phase 3 body fill).

8.33 Function Entry: 1-

8.33.1 Name

`'1-'`

8.33.2 Class

Function

8.33.3 Syntax

`(1- number)`

8.33.4 Arguments and Values

- `'number'`: integer or real.

8.33.5 Description

Decrements `'number'` by 1.

8.33.6 Return Values

The input number minus 1. Return type matches input type.

8.33.7 Side Effects

None.

8.33.8 Examples

- `'(1- 5)'` returns `'4'`.
- `'(1- -17.5)'` returns `'-18.5'`.

8.33.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.33.10 Source Notes

- [1- (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4E3A882A-959D-4B46-BA5E-5A3C1BDB3079.htm>)
- Status: documented (Phase 3 body fill).

8.34 Function Entry: ~

8.34.1 Name

`'~'`

8.34.2 Class

Function

8.34.3 Syntax

`(~ int)`

8.34.4 Arguments and Values

- `'int'`: a positive or negative integer.

8.34.5 Description

Returns the bitwise NOT (one's complement) of `'int'`.

8.34.6 Return Values

An integer.

8.34.7 Side Effects

None.

8.34.8 Examples

- `'(~ 3)'` returns `'-4'`.
- `'(~ 100)'` returns `'-101'`.
- `'(~ -4)'` returns `'3'`.

8.34.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.34.10 Source Notes

- [~ (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-98811E38-FCA8-4ADB-858C-60C3828D7DB4.htm>)
- Status: documented (Phase 3 body fill).

8.35 Function Entry: BOOLE

8.35.1 Name

‘boole’

8.35.2 Class

Function

8.35.3 Syntax

(boole operator int1 [int2 ...])

8.35.4 Arguments and Values

- ‘operator’: integer 0–15, the four-bit truth table identifying which Boolean function to apply (each bit of ‘operator’ selects one row of the two-input truth table).
- ‘int1’, ‘int2’, ...: integers to combine bitwise.

8.35.5 Description

Returns the bitwise Boolean combination of the integer arguments under the truth table identified by ‘operator’. Common operator codes include:

- 1: AND (output 1 only when both input bits are 1).
- 6: XOR (output 1 only when exactly one input bit is 1).
- 7: OR (output 1 when at least one input bit is 1).
- 8: NOR (output 1 only when both input bits are 0).

With more than two integers, the operation is applied left-associatively: ‘(boole op a b c)’ is ‘(boole op (boole op a b) c)’.

8.35.6 Return Values

An integer.

8.35.7 Side Effects

None.

8.35.8 Examples

- ‘(boole 1 12 5)’ returns ‘3’ (logical AND of ‘12’ and ‘5’).
- ‘(boole 4 3 14)’ returns ‘12’ (operator code 4: bits set in ‘int2’ but not in ‘int1’).

8.35.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.35.10 Source Notes

- [boole (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-C8412B41-BED9-4182-94E6-9EDB3C176176.html>)
- Status: documented (Phase 3 body fill).

8.36 Function Entry: LOGAND

8.36.1 Name

`'logand'`

8.36.2 Class

Function

8.36.3 Syntax

`(logand [int int ...])`

8.36.4 Arguments and Values

- `'int'`: zero or more integers.

8.36.5 Description

Returns the bitwise AND of the supplied integers. With no arguments, returns 0.

8.36.6 Return Values

An integer.

8.36.7 Side Effects

None.

8.36.8 Examples

- `'(logand 7 15 3)'` returns `'3'`.
- `'(logand 2 3 15)'` returns `'2'`.
- `'(logand 8 3 4)'` returns `'0'`.

8.36.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.36.10 Source Notes

- [logand (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4E042C63-8BD7-4FED-B9E1-B5CE12D18...htm>)
- Status: documented (Phase 3 body fill).

8.37 Function Entry: LOGIOR

8.37.1 Name

`'logior'`

8.37.2 Class

Function

8.37.3 Syntax

`(logior [int int ...])`

8.37.4 Arguments and Values

- `'int'`: zero or more integers.

8.37.5 Description

Returns the bitwise inclusive OR of the supplied integers. With no arguments, returns 0.

8.37.6 Return Values

An integer.

8.37.7 Side Effects

None.

8.37.8 Examples

- `'(logior 1 2 4)'` returns `'7'`.
- `'(logior 9 3)'` returns `'11'`.

8.37.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.37.10 Source Notes

- [logior (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-A4BF3B68-4988-42F0-AFE6-BFD06D0DE1-htm>)
- Status: documented (Phase 3 body fill).

8.38 Function Entry: LSH

8.38.1 Name

`'lsh'`

8.38.2 Class

Function

8.38.3 Syntax

`(lsh int numbits)`

8.38.4 Arguments and Values

- `'int'`: integer to be shifted.
- `'numbits'`: integer; positive shifts left, negative shifts right.

8.38.5 Description

Logical bitwise shift of `'int'` by `'numbits'` positions. Positive `'numbits'` shifts left; negative shifts right. Zero bits are shifted in, and bits shifted out are discarded.

8.38.6 Return Values

An integer. The result is positive if bit 31 (the sign bit) is 0 after the shift; otherwise the result is negative.

8.38.7 Side Effects

None.

8.38.8 Examples

- `'(lsh 2 1)'` returns `'4'`.
- `'(lsh 2 -1)'` returns `'1'`.
- `'(lsh 40 2)'` returns `'160'`.

8.38.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

8.38.10 Source Notes

- [lsh (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-3BC67533-3F4F-4C01-9053-CD82AA235AE8.htm>)
- Status: documented (Phase 3 body fill).

9 9 Characters and Strings

9.1 Overview

Characters and strings are visible both at reader level and through I/O and string-processing functions.

9.2 Normative Rules

- strings are quoted
- backslash escapes exist
- Unicode semantics changed significantly in AutoCAD 2021 and later

9.3 String Model

Autodesk documentation sampled here supports a semantic view of strings, but not a full implementation-level storage model.

Accordingly, this specification treats a string as:

- an ordered sequence of characters,
- addressable by string-processing functions,
- readable and writable through string and text-I/O facilities,
- subject to version-dependent character encoding behavior.

This specification does **not** currently claim a vendor-documented internal representation such as:

- byte vector,
- UTF-16 code-unit vector,
- Unicode scalar-value vector,
- implementation-specific opaque text object.

At the current evidence level, only the observable behavior is normative.

9.4 Pre-Unicode and Post-Unicode Semantics

9.4.1 Pre-2021 Autodesk Model

Before AutoCAD 2021, Autodesk documents string and character behavior in terms that were effectively tied to legacy text encodings.

In particular, Autodesk’s current function notes say that earlier behavior for functions such as ‘ascii’, ‘chr’, ‘vl-string-*’, and ‘read-char’ reflected ASCII or MBCS-oriented handling rather than full Unicode semantics.

Observable consequences of the pre-2021 model include:

- character-oriented functions were effectively limited by legacy encoding assumptions,
- file text handling depended on older platform encoding behavior,
- string-position and character-code behavior for non-ASCII text was not Unicode-clean in the modern sense.

9.4.2 Post-2021 Autodesk Model

Beginning with AutoCAD 2021, Autodesk explicitly introduced Unicode-related changes controlled by ‘LISPYSYS’.

From the documentation sampled here, the post-2021 model includes at least:

- Unicode-aware string handling,
- revised behavior for string-search and character-code functions,
- revised ‘read-char’ behavior,
- explicit file-opening encoding support,
- an implementation switch through ‘LISPYSYS’.

‘LISPYSYS’ values are documented as selecting different engine behavior:

- legacy ASCII engine behavior,
- Unicode engine behavior,
- Unicode engine behavior with compatibility adjustments, depending on Autodesk release details.

For the purposes of this specification, the important normative point is:

- string semantics after AutoCAD 2021 are versioned and mode-dependent,
- the language cannot be specified correctly without recording the active Unicode mode.

9.4.3 BricsCAD

BricsCAD documentation sampled here also indicates explicit Unicode-aware text-file handling through extended ‘open’ modes.

This suggests that:

- BricsCAD should not be modeled purely as a pre-Unicode AutoLISP environment,
- text and encoding behavior must remain part of the dialect/environment profile.

9.4.4 clautolisp dispatch for LISPSYS

clautolisp accepts (`getvar "LISPSYS"`) and (`setvar "LISPSYS" n`) under every dialect — extensions are always available, the dialect controls diagnostic emission. Per `encoding-dispatch.issue`:

Dialect	GETVAR / SETVAR on LISPSYS
<code>--autocad-2026</code>	silent (native dialect)
<code>--strict</code>	<code>enc-extension-used</code>
<code>--bricscad-v26</code>	<code>enc-foreign-dialect</code> / <code>bricscad</code>
<code>--clautolisp</code>	<code>enc-foreign-dialect</code> / <code>clautolisp</code>

Additionally, (`setvar "LISPSYS" n`) with `n` `\notin` `{0,1,2}` emits `enc-lispsys-out-of-range` under all dialects; the write itself still proceeds (matching the vendor ‘permissive but warn’ behaviour). The diagnostics are advisory — user code that calls `LISPSYS` keeps running under every dialect; only the diagnosis varies. See the `encoding-dispatch.issue` ‘Diagnostics’ section for the full `enc-*` code table.

9.5 Representation-Level Notes

At the observable language level, the following can be said:

- strings preserve character order,

- strings are delimited by double quotes in source representation,
- strings participate in search, comparison, concatenation, and I/O functions,
- the meaning of a "character" is version-sensitive in Unicode-era Autodesk behavior.

At the implementation level, the following remain under-specified in the current source set:

- whether AutoCAD stores strings internally as Unicode code points, UTF-16 code units, or another representation,
- whether string indexing counts code units, Unicode scalar values, grapheme-like user-visible characters, or implementation-defined units in every function,
- whether normalization is performed or preserved,
- the exact interaction between source-file encoding, runtime string representation, and printer behavior in all releases.

Therefore, 'clautolisp' should distinguish:

- external source encoding,
- external file I/O encoding,
- internal string representation,
- observable character/string API semantics.

Only the observable semantics should be treated as normative until stronger evidence is available.

9.6 Strict/Lax Considerations

- strict mode:
 - should avoid undocumented assumptions about internal encoding,
 - should reject or warn about string/character edge cases whose behavior is known to vary across Unicode modes or host versions,

- should prefer explicitly encoded text I/O where supported.
- lax mode:
 - may accept broader compatibility behavior for legacy text handling,
 - may emulate legacy ASCII/MBCS behavior when targeting pre-2021 Autodesk semantics or equivalent host profiles.

9.7 Implementation Guidance

‘clautolisp’ should model strings as abstract text objects with versioned observable behavior, not by prematurely committing the specification to a specific Common Lisp string implementation detail.

The implementation should provide at least:

- a legacy compatibility mode for pre-Unicode behavior,
- a Unicode-aware mode corresponding to modern Autodesk behavior,
- explicit control through dialect/environment/profile configuration,
- compatibility tests for character-code, indexing, file I/O, and search functions.

This is especially important because Common Lisp string behavior is implementation-dependent enough that naive reuse could leak incorrect semantics into AutoLISP compatibility.

9.8 Source Notes

- [About Strings (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ESP/AutoCAD-AutoLISP/files/GUID-D7C33A49-9715-4E9B-9D8A-4C2E7BFC0FE5.htm>)
- [LISPSYS](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-Core/files/GUID-1853092D-6E6D-4A06-8956-AD2C3DF203A3.htm>)
- [ascii (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-03E1B586-72CB-4AB6-A151-B581AE530318.htm>)

- `[vl-string-search (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-4E7AE1DA-1ED5-4D96-A7D3-342411.htm>)
- `[read-char (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2024/KOR/AutoCAD-AutoLISP-Reference/files/GUID-7E94BD14-F018-47D0-88DA-2B08DE32DB2C.htm>)
- `[open (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-089A323F-21FF-4337-99A9-375758E23BA4.htm>)
- `[read + write text files (BricsCAD)]`(https://developer.bricsys.com/bricscad/help/en_US/V25/DevRef/source/readwritetextfiles.htm)
- Status: documented for major behavioral shift and observable effects; internal representation remains under-specified.

9.9 Function Entry: STRCAT

9.9.1 Name

`'strcat'`

9.9.2 Class

Function

9.9.3 Syntax

`(strcat [string ...])`

9.9.4 Arguments and Values

- zero or more strings

9.9.5 Description

Concatenates the argument strings.

9.9.6 Return Values

String.

Autodesk documents that if no arguments are supplied, the result is a zero-length string.

9.9.7 Side Effects

None.

9.9.8 Exceptional Situations

Erroneous if any argument is not acceptable as a string under host semantics.

9.9.9 Examples

- `“(strcat "A" "B") => "AB”`

9.9.10 Compatibility

String behavior is subject to the pre-Unicode / post-Unicode split discussed earlier in this chapter.

9.9.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.9.12 Source Notes

- [strcat (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/HUN/AutoCAD-LT-AutoLISP-Reference/files/GUID-4430B1BF-DBB5-49D1-98F9-711B480976A1.html>)

9.10 Function Entry: STRLEN

9.10.1 Name

`'strlen'`

9.10.2 Class

Function

9.10.3 Syntax

`(strlen [str ...])`

9.10.4 Arguments and Values

- `'str'` — zero or more strings

9.10.5 Description

Returns the length of the string.

9.10.6 Return Values

Integer.

Autodesk documents:

- the sum of the lengths when multiple arguments are supplied
- `'0'` if no arguments are supplied
- `'0'` for the empty string

9.10.7 Side Effects

None.

9.10.8 Compatibility

Autodesk explicitly documents the Unicode-era counting change.

9.10.9 Notes

This function is one of the places where internal representation and observable character counting must not be conflated.

Autodesk documents that AutoCAD 2021 changed the function so Unicode characters are counted correctly when ‘LISPSYS’ is ‘1’ or ‘2’.

9.10.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.10.11 Source Notes

- `[strlen (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-F16AAC5F-5C87-4DC5-A7B9-BDCD25DC507A.htm>)

9.11 Function Entry: SUBSTR

9.11.1 Name

‘substr‘

9.11.2 Class

Function

9.11.3 Syntax

(substr string start [length])

9.11.4 Arguments and Values

- ‘string‘ — string
- ‘start‘ — positive integer starting position
- ‘length‘ — optional positive integer substring length

9.11.5 Description

Returns a substring of ‘string‘.

9.11.6 Return Values

String.

9.11.7 Side Effects

None.

9.11.8 Compatibility

String indexing and character-unit semantics are version-sensitive in Unicode-era environments.

9.11.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.11.10 Source Notes

- [substr (AutoLISP)](<https://help.autodesk.com/cloudhelp/2015/ESP/AutoCAD-AutoLISP/files/GUID-36F8701B-7AB7-47BE-AC31-8508A2DF46A2.htm>)

9.11.11 Notes

- Autodesk documents 1-based indexing: the first character of the string is position ‘1’.
- If ‘length’ is omitted, Autodesk documents that the substring extends to the end of the string.

9.12 Function Entry: STRCASE

9.12.1 Name

`'strcase'`

9.12.2 Class

Function

9.12.3 Syntax

`(strcase string [which])`

9.12.4 Arguments and Values

- `'string'` — string
- `'which'` — optional `'T'` or `'nil'`

9.12.5 Description

Returns a string converted in case according to the supplied option.

9.12.6 Return Values

String.

9.12.7 Side Effects

None.

9.12.8 Compatibility

Case-conversion rules for non-ASCII characters are potentially version-sensitive in Unicode-aware modes.

9.12.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.12.10 Source Notes

- `[strcase (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-108DCD2C-6597-4548-856D-937787AFE5E0.htm>)

9.12.11 Notes

- Autodesk documents that ‘which’ equal to ‘T’ requests lowercase conversion; otherwise uppercase conversion is used.
- Autodesk documents that case mapping follows the currently configured character set.

9.13 Function Entry: ASCII

9.13.1 Name

`'ascii'`

9.13.2 Class

Function

9.13.3 Syntax

`(ascii string)`

9.13.4 Arguments and Values

- `'string'`: string whose first character is examined

9.13.5 Description

Returns the code of the first character in the string.

9.13.6 Return Values

Integer character code.

9.13.7 Side Effects

None.

9.13.8 Compatibility

Autodesk explicitly documents behavior changes after AutoCAD 2021 due to Unicode support.

9.13.9 Notes

This function is one of the clearest probes into the observable character model of a given host/version.

9.13.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.13.11 Source Notes

- [ascii (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-03E1B586-72CB-4AB6-A151-B581AE530318.htm>)
- Status: documented.

9.14 Function Entry: CHR

9.14.1 Name

`'chr'`

9.14.2 Class

Function

9.14.3 Syntax

`(chr integer)`

9.14.4 Arguments and Values

- `'integer'`: character code

Autodesk documents the accepted range as `'1'` to `'65536'` in Unicode-capable behavior.

9.14.5 Description

Returns a one-character string corresponding to the given character code.

9.14.6 Return Values

String of length one under ordinary observable semantics.

9.14.7 Side Effects

None.

9.14.8 Compatibility

Like `'ascii'`, `'chr'` is affected by the pre-Unicode / post-Unicode split.

9.14.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.14.10 Source Notes

- `[chr (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2025/ESP/AutoCAD-MAC-AutoLISP-Reference/files/GUID-5846BFDD-AE5F-4B2F-99E0-0B9DBA667739.htm>)

9.14.11 Notes

- Autodesk documents that AutoCAD 2021 changed ‘chr’ from legacy ASCII-oriented behavior to Unicode-code-point-oriented behavior when ‘LISPSYS’ is ‘1’ or ‘2’.

9.15 Function Entry: ATOI

9.15.1 Name

`'atoi'`

9.15.2 Class

Conversion Function

9.15.3 Syntax

`(atoi string)`

9.15.4 Arguments and Values

- `'string'`: a string representing any number.

9.15.5 Description

Converts `'string'` into an integer. Trailing non-numeric characters terminate parsing; the integer prefix is returned as the value. Decimal-fraction input is truncated toward zero.

9.15.6 Return Values

An integer; `'0'` if the string is not a valid number.

9.15.7 Side Effects

None.

9.15.8 Examples

- `'(atoi "12")'` returns `'12'`.
- `'(atoi "12.34")'` returns `'12'` (truncating decimal input).
- `'(atoi "xyz")'` returns `'0'`.

9.15.9 Implementation-defined Behaviour

The Phase-5 vendor citations below define the documented surface. Several lexical edges are not nailed down by either vendor's prose and remain implementation-defined unless and until product tests exist:

- Leading whitespace handling ('(atoi " 17")').
- Sign acceptance ('(atoi "+17")', '(atoi "-17")').
- Behaviour of leading-zero forms ('(atoi "017")' — decimal vs. octal-like).
- Behaviour of '0x'-prefixed forms ('(atoi "0xbabe")').
- Trailing-junk strictness ('(atoi "17x")').

For `clautolisp` the chosen lex model is: skip leading whitespace, accept optional sign, parse the longest run of decimal digits, truncate the fractional part if a `.` follows the digit run, treat all leading-zero forms as base 10, treat '0x'-prefixed forms as '0' (no autodetection). The probe suite in `probe-atoi.lisp` records the AutoCAD / BricsCAD product behaviour when run via `scripts/run-probes.sh`.

9.15.10 Tested Behaviour (BricsCAD V26 / macOS, 2026-04-26)

The probe suite was run against BricsCAD V26 on macOS via the direct runner. Results from `'results/bricscad/macos/20260426T122808Z/results.sexp'`:

Probe	Returned
'(atoi "17")'	'17'
'(atoi "017")'	'17' (decimal interpretation; not octal)
'(atoi "0xbabe")'	'0' ('0x'-prefix rejected)
'(atoi "+17")'	'17' (leading '+' accepted)
'(atoi "-17")'	'-17' (leading '-' accepted)
'(atoi "17x")'	'17' (parsing stops at first non-digit)
'(atoi " 17")'	'17' (leading whitespace accepted)
'(atoi "3.9")'	'3' (decimal-fraction truncates toward zero)

Confirms a `'strtol(..., 10)'`-style C-library model: skip leading whitespace, optional sign, longest decimal-digit prefix, return zero on no digits. The tested behaviour matches `clautolisp`'s implementation choice, so the lex edges previously marked **implementation-defined** are now **tested** on BricsCAD V26.

9.15.11 Availability

- AutoCAD 2026: documented (Autodesk reference page).
- BricsCAD V26: documented + product-tested on macOS (probe suite, 2026-04-26).
- Status: documented in both vendors; lex-edge behaviour now tested against BricsCAD V26 / macOS. AutoCAD product test deferred until a local install becomes available.

9.15.12 See Also

- `probe-atoi.lsp` — executable Phase-5 probe suite.
- `itoa` — integer-to-string companion.

9.15.13 Source Notes

- `[atoi (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-20EF237C-7079-4E29-860C-B8531D6C7F36.htm>)
- `[atoi (BricsCAD V26 DevRef)]`(https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/atoi.htm)
- Status: documented (Phase 5 closure via vendor citation; product-test coverage deferred until a CAD product is available locally).

9.16 Function Entry: ATOF

9.16.1 Name

`'atof'`

9.16.2 Class

Conversion Function

9.16.3 Syntax

`(atof string)`

9.16.4 Arguments and Values

- `'string'`: a string representing any number.

9.16.5 Description

Converts `'string'` into a double-precision real. A trailing non-numeric tail terminates parsing; the numeric prefix is returned.

9.16.6 Return Values

A double-precision real; `'0.0'` if the string is not a valid number.

9.16.7 Side Effects

None.

9.16.8 Examples

- `'(atof "12")'` returns `'12.0'`.
- `'(atof "12.34")'` returns `'12.34'`.
- `'(atof "xyz")'` returns `'0.0'`.

9.16.9 Implementation-defined Behaviour

The Phase-5 vendor citations below define the documented surface. Several lexical edges are not nailed down by either vendor's prose and remain implementation-defined unless and until product tests exist:

- ‘.5’, ‘.1’ — leading/trailing decimal-point variants.
- Exponent notation (‘1e3’, ‘1E+03’).
- Leading whitespace handling.
- Locale sensitivity, in particular decimal-comma vs decimal-point.
- Acceptance of hexadecimal floating syntax (‘0x1p4’).
- Trailing-junk strictness.

For `clautolisp` the chosen lex model is: skip leading whitespace, accept optional sign, accept the longest prefix matching ‘(digits ’ digits?) | (digits? ’ ’ digits) | digits’ followed by an optional exponent ‘((’e’|’E’) [+]? digits)’, no locale sensitivity (decimal-point only), no hexadecimal floating syntax. At least one digit must appear on **some** side of the dot — vendor AutoLISP reads ‘42.’ as ‘42.0’, ‘5.’ as ‘5.0’, and ‘.5’ as ‘0.5’, distinct from CL’s reading of ‘42.’ as the integer 42. The probe suite in `probe-atof.lisp` records the AutoCAD / BricsCAD product behaviour when run via `scripts/run-probes.sh`.

9.16.10 Tested Behaviour (BricsCAD V26 / macOS, 2026-04-26)

Probe results from ‘results/bricscad/macOS/20260426T122808Z/results.sexp’:

Probe	Returned	Note
‘(atof "3.5")’	‘3.5’	baseline
‘(atof ".5")’	‘0.5’	leading-dot accepted
‘(atof "1.")’	‘1.0’	trailing-dot accepted
‘(atof "1e3")’	‘1000.0’	exponent accepted
‘(atof "017")’	‘17.0’	leading zero treated as decimal
‘(atof "0x1p4")’	‘16.0’	hexadecimal floating syntax accepted — BricsCAD V26 delegates
‘(atof "3,5")’	‘3.0’	decimal-comma rejected; parsing stops at the comma (no locale sensitivity)
‘(atof " 3.5")’	‘3.5’	leading whitespace accepted

The ‘0x1p4 → 16.0’ result is the most significant finding: BricsCAD V26 **does** accept hex-float input. The conservative ‘clautolisp’ lex model rejects it; conformant code that targets BricsCAD V26 may expand the lex model to include hex-floats. Locale sensitivity is **not** present on the macOS build; portable AutoLISP code should not rely on locale.

9.16.11 Availability

- AutoCAD 2026: documented (Autodesk reference page).
- BricsCAD V26: documented + product-tested on macOS (probe suite, 2026-04-26).
- Status: documented in both vendors; lex-edge behaviour now tested against BricsCAD V26 / macOS. AutoCAD product test deferred until a local install becomes available.

9.16.12 See Also

- `probe-atof.lsp` — executable Phase-5 probe suite.
- `rtos` — real-to-string companion.

9.16.13 Source Notes

- `[atof (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-8618A388-E4CF-40E1-813B-057367DD1840.htm>)
- `[atof (BricsCAD V26 DevRef)]`(https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/atof.htm)
- Status: documented (Phase 5 closure via vendor citation; product-test coverage deferred until a CAD product is available locally).

9.17 Function Entry: ITOA

9.17.1 Name

`'itoa'`

9.17.2 Class

Conversion Function

9.17.3 Syntax

`(itoa integer)`

9.17.4 Arguments and Values

- `'integer'` — integer

9.17.5 Description

Converts an integer to its string representation.

9.17.6 Return Values

String.

9.17.7 Side Effects

None.

9.17.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.17.9 Source Notes

- `[itoa (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-LT-AutoLISP-Reference/files/GUID-7E247CA3-95D3-4497-BDE2-6EB0E727DD4D.htm>)

9.18 Function Entry: RTOS

9.18.1 Name

`'rtos'`

9.18.2 Class

Conversion Function

9.18.3 Syntax

`(rtos number [mode [precision]])`

9.18.4 Arguments and Values

- `'number'`: number
- `'mode'`: optional conversion mode
- `'precision'`: optional precision

9.18.5 Description

Converts a real or number to a string according to AutoCAD formatting conventions.

9.18.6 Return Values

String.

9.18.7 Side Effects

None.

9.18.8 Affected By

- numeric formatting settings
- conversion mode and precision

9.18.9 Compatibility

This function is part of the language/host formatting boundary, not purely a generic printer primitive.

9.18.10 Notes

- Autodesk documents the accepted ‘mode’ values as the linear-unit formats corresponding to ‘LUNITS’:
 - ‘1’ scientific
 - ‘2’ decimal
 - ‘3’ engineering
 - ‘4’ architectural
 - ‘5’ fractional
- If ‘mode’ and ‘precision’ are omitted, Autodesk documents that ‘rtos’ uses the current ‘LUNITS’ and ‘LUPREC’ settings.
- Autodesk documents that the resulting string is affected by ‘UNIT-MODE’ for engineering, architectural, and fractional output, and by ‘DIMZIN’ for zero suppression.
- **Locale insensitivity (normative).** ‘rtos’ always uses ‘.’ as the decimal separator regardless of the host’s ‘LC_NUMERIC’ / ‘LANG’ / ‘LC_ALL’ environment, regardless of the OS-level locale, and regardless of the ‘DIMDSEP’ system variable. ‘DIMDSEP’ controls only the dimension-text renderer (see § *System Variable Entry: DIMDSEP*), not the AutoLISP numeric printer. The complementary ‘(atof)’ rejects locale-comma input the same way — see the ‘(atof "3,5") -> 3.0’ probe in § *atof*. clautolisp matches this contract end-to-end: the value printer (‘autolisp-value->string’, princ / prin1 / vl-*to-string) is also locale-insensitive — ‘(vl-princ-to-string 3.14)’ is ‘"3.14"' on every locale, every host. User code that wants a comma-separator UI does the substitution itself; the AutoLISP runtime stays in ‘.-land so that ‘(atof (rtos n))’ round-trips on every machine.

9.18.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.18.12 Source Notes

- [rtos (AutoLISP)](<https://help.autodesk.com/cloudhelp/2018/ENU/AutoCAD-AutoLISP-Reference/files/GUID-D03ABBC2-939A-44DB-8C93-FC63B64DE4A2.htm>)
- [About String Conversions (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-AutoLISP/files/GUID-A38E30D4-684E-44FA-A910-E08522876.htm>)

9.19 Function Entry: ANGTOF

9.19.1 Name

‘angtof’

9.19.2 Class

Conversion Function

9.19.3 Syntax

(angtof string [unit])

9.19.4 Arguments and Values

- ‘string’ — string
- ‘unit’ — optional integer angular-unit selector

9.19.5 Description

Converts an angle string to a numeric angle value.

9.19.6 Return Values

Real angle value if successful; otherwise ‘nil’.

9.19.7 Compatibility

Part of host-oriented numeric formatting and parsing semantics.

9.19.8 Notes

- Autodesk documents the accepted ‘unit’ values as those of ‘AUNITS’:
 - ‘0’ degrees
 - ‘1’ degrees/minutes/seconds
 - ‘2’ grads
 - ‘3’ radians
 - ‘4’ surveyor’s units

- If ‘unit’ is omitted, Autodesk documents that ‘angtof’ uses the current ‘AUNITS’ setting.
- Autodesk documents ‘angtof’ and ‘angtos’ as complementary functions when matching unit conventions are used.

9.19.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.19.10 Source Notes

- [angtof (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ENU/AutoCAD-AutoLISP-Reference/files/GUID-9DE9200B-F27E-4BB6-8AE6-BC5C8A64A527.htm>)
- [About Angular Conversion (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP/files/GUID-725BC3DE-3D4C-4365-A7F9-F4A5A5A5A5A5.htm>)

9.20 Function Entry: ANGTON

9.20.1 Name

‘angton’

9.20.2 Class

Conversion Function

9.20.3 Syntax

(angton angle [unit [precision]])

9.20.4 Arguments and Values

- ‘angle’ — integer or real, interpreted in radians
- ‘unit’ — optional integer angular-unit selector
- ‘precision’ — optional integer precision

9.20.5 Description

Converts a numeric angle value to a formatted string.

9.20.6 Return Values

A string if successful; otherwise ‘nil’.

9.20.7 Affected By

- angle formatting conventions
- unit and precision parameters

9.20.8 Notes

- Autodesk documents the accepted ‘unit’ values as those of ‘AUNITS’:
 - ‘0’ degrees
 - ‘1’ degrees/minutes/seconds
 - ‘2’ grads
 - ‘3’ radians

9.21 Function Entry: DISTOF

9.21.1 Name

‘distof‘

9.21.2 Class

Conversion Function

9.21.3 Syntax

(distof string [mode])

9.21.4 Arguments and Values

- ‘string’ — string
- ‘mode’ — optional integer linear-unit selector

9.21.5 Description

Converts a distance string to a numeric distance value.

9.21.6 Return Values

Real number if successful; otherwise ‘nil‘.

9.21.7 Compatibility

Part of host-oriented unit parsing semantics.

9.21.8 Notes

- Autodesk documents the accepted ‘mode‘ values as those of ‘LUNITS‘:
 - ‘1‘ scientific
 - ‘2‘ decimal
 - ‘3‘ engineering
 - ‘4‘ architectural
 - ‘5‘ fractional

- Autodesk documents ‘distof’ and ‘rtos’ as complementary functions when the same mode conventions are used.
- Autodesk documents that modes ‘3’ and ‘4’ are parsed equivalently: engineering and architectural input forms are both accepted for either of those mode values.

9.21.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.21.10 Source Notes

- [distof (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-CC0490B9-29BC-44C4-A317-5BAA34B0168E.htm>)
- [About String Conversions (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-AutoLISP/files/GUID-A38E30D4-684E-44FA-A910-E08522876.htm>)

9.22 Dictionary Coverage Index: String and Character Functions

The Autodesk string-handling and related conversion families include at least:

- ‘read’
- ‘strcase’
- ‘strcat’
- ‘strlen’
- ‘substr’
- ‘vl-prin1-to-string’
- ‘vl-princ-to-string’

- ‘vl-string->list‘
- ‘vl-string-elt‘
- ‘vl-string-left-trim‘
- ‘vl-string-mismatch‘
- ‘vl-string-position‘
- ‘vl-string-right-trim‘
- ‘vl-string-search‘
- ‘vl-string-subst‘
- ‘vl-string-translate‘
- ‘vl-string-trim‘
- ‘ascii‘
- ‘chr‘
- ‘itoa‘
- ‘rtos‘
- ‘angtof‘
- ‘angtos‘
- ‘atof‘
- ‘atoi‘
- ‘distof‘

Sources:

- [String-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-8543549B-F0D7-43CD-htm>)
- [Conversion Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/KOR/AutoCAD-AutoLISP-Reference/files/GUID-5BDBD4BC-289E-4A49-htm>)

9.23 Function Entry: WCMATCH

9.23.1 Name

`'wcmatch'`

9.23.2 Class

Function

9.23.3 Syntax

`(wcmatch str pattern)`

9.23.4 Arguments and Values

- `'str'`: string to test. Comparison is case-sensitive.
- `'pattern'`: string containing wild-card pattern-match characters.

9.23.5 Description

Performs a wild-card pattern match. Only the first ~500 characters of both arguments are compared. The supported pattern syntax is:

Pattern char	Matches
<code>'#'</code>	a single numeric digit
<code>'@'</code>	a single alphabetic character
<code>'.'</code>	a single non-alphanumeric character
<code>'*'</code>	any character sequence, including empty
<code>'.'</code>	a single character
<code>'^'</code>	negation (must be the first character of the pattern)
<code>'[...]'</code>	any one of the enclosed characters
<code>'[~ ...]'</code>	any character not in the enclosed set
<code>'_'</code>	character range, used inside <code>'[]'</code>
<code>'.'</code>	separates multiple pattern alternatives
<code>'\"'</code>	escape: take the next character literally

The `'.'` separator gives `'wcmatch'` an OR-of-patterns semantics: the result is `'T'` if any of the comma-separated patterns matches.

9.23.6 Return Values

Returns `'T'` when the pattern matches, otherwise `'nil'`.

9.23.7 Side Effects

None.

9.23.8 Examples

- `(wcmatch "Name" "N*")` returns `'T'`.
- `(wcmatch "Name" "???,~*m*,N*")` returns `'T'` — three alternatives joined by `','`.

9.23.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

9.23.10 Source Notes

- `[wcmatch (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-EC257AF7-72D4-4B38-99B6-9B09952A5...htm>)
- Status: documented (Phase 3 body fill).

10 10 Lists, Dotted Pairs, and Association Lists

10.1 Overview

Lists are both code and data. Dotted pairs and association lists are especially important because they carry host entity and table data.

10.2 Normative Rules

- proper lists and dotted pairs are both first-class
- association lists are pervasive in CAD interaction
- points are lists, not a separate primitive vector type

10.3 Source Notes

- [About Lists (AutoLISP)](<https://help.autodesk.com/cloudhelp/2016/ENU/AutoCAD-AutoLISP/files/GUID-F8074AA9-F361-4960-B628-681465B74229.htm>)
- [About Dotted Pairs (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/PLK/AutoCAD-AutoLISP/files/GUID-877B87BA-6FA2-4894-9F94-184E7DF25B7D.htm>)

10.4 Function Entry: CAR

10.4.1 Name

`'car'`

10.4.2 Class

Function

10.4.3 Syntax

`(car lst)`

10.4.4 Arguments and Values

- `'lst'`: list

10.4.5 Description

Returns the first element of a list.

10.4.6 Return Values

The first element of `'lst'`.

If `'lst'` is the empty list, `'car'` returns `'nil'`.

10.4.7 Side Effects

None.

10.4.8 Exceptional Situations

Erroneous if `'lst'` is not accepted as a list by the implementation.

10.4.9 Examples

- `'(car '(a b c)) => A'`
- `'(car '((a b) c)) => (A B)'`
- `'(car '()) => nil'`

10.4.10 Compatibility

Autodesk's dedicated page documents 'car' for lists and explicitly documents empty-list behavior.

That page does not explicitly state dotted-pair behavior. Implementations may support '(car '(a . b)) => A' by Lisp tradition, but this specification treats dotted-pair acceptance by 'car' as inferred or compatibility behavior until vendor evidence is collected.

10.4.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.4.12 Source Notes

- [car (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-2DD1AF33-415C-4C1A-9631-DA958134C53A.htm>)
- Status: documented for lists and empty-list behavior; dotted-pair behavior remains inferred.

10.5 Function Entry: CDR

10.5.1 Name

`'cdr'`

10.5.2 Class

Function

10.5.3 Syntax

`(cdr lst)`

10.5.4 Arguments and Values

- `'lst'`: list, including the empty list; dotted pairs are also explicitly covered by Autodesk's note

10.5.5 Description

Returns all but the first element of the specified list.

10.5.6 Return Values

If `'lst'` is a proper list with two or more elements, returns a list containing all elements of `'lst'` except the first.

If `'lst'` is the empty list, returns `'nil'`.

If `'lst'` is a dotted pair, returns the second element without enclosing it in a list.

10.5.7 Side Effects

None.

10.5.8 Examples

- `'(cdr '(a b c)) => (B C)'`
- `'(cdr '((a b) c)) => (C)'`
- `'(cdr '()) => nil'`
- `'(cdr '(a . b)) => B'`

- `'(cdr '(1 . "Text")) => "Text"`

10.5.9 Compatibility

Unlike `'car'`, Autodesk explicitly documents dotted-pair behavior for `'cdr'`.

10.5.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.5.11 Source Notes

- `[cdr (AutoLISP)](https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP-Reference/files/GUID-F9CD8FF3-022A-4323-BAE7-390174451537.htm)`
- Status: documented.

10.6 Function Entry: CONS

10.6.1 Name

`'cons'`

10.6.2 Class

Function

10.6.3 Syntax

`(cons new-first-element lst-or-atom)`

10.6.4 Arguments and Values

- `'new-first-element'`: any value
- `'lst-or-atom'`: list or atom

10.6.5 Description

Adds an element to the beginning of a list, or constructs a dotted pair.

10.6.6 Return Values

- a proper list if the second argument is a list
- a dotted pair if the second argument is an atom

10.6.7 Side Effects

None.

10.6.8 Examples

- `'(cons 'a '(b c)) => (A B C)'`
- `'(cons '(a) '(b c d)) => ((A) B C D)'`
- `'(cons 'a 2) => (A . 2)'`

10.6.9 Compatibility

This entry is central for understanding the list-versus-dotted-pair model of AutoLISP data.

10.6.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.6.11 Source Notes

- [cons (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP-Reference/files/GUID-33B418E7-DB3D-4CBE-954E-F070F0A7CB2B.htm>)
- Status: documented.

10.7 Function Entry: LIST

10.7.1 Name

`'list'`

10.7.2 Class

Function

10.7.3 Syntax

`(list [expr ...])`

10.7.4 Arguments and Values

- zero or more expressions

10.7.5 Description

Evaluates its arguments and constructs a list containing their values.

10.7.6 Return Values

A list containing the argument values.

If no expressions are supplied, `'list'` returns `'nil'`.

10.7.7 Side Effects

Side effects are those of evaluating the argument expressions.

10.7.8 Examples

- `'(list 'a 'b 'c) => (A B C)'`
- `'(list 'a '(b c) 'd) => (A (B C) D)'`
- `'(list 1 2 3) => (1 2 3)'`
- `'(list) => nil'`

10.7.9 Notes

Autodesk explicitly notes that ‘list’ is frequently used to construct 2D or 3D point values.

The dedicated page also notes that a quoted list literal may be used instead of ‘list’ when there are no variables or undefined items in the list.

10.7.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.7.11 Source Notes

- [list (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-2BA66308-DE64-4BA6-8F57-6E3EC1A93141.htm>)
- Status: documented.

10.8 Function Entry: APPEND

10.8.1 Name

`'append'`

10.8.2 Class

Function

10.8.3 Syntax

`(append list [list ...])`

10.8.4 Arguments and Values

- one or more lists

10.8.5 Description

Returns a list containing the elements of all argument lists in order.

10.8.6 Return Values

A list containing the concatenated elements.

If no arguments are supplied, Autodesk documents that `'append'` returns `'nil'`.

10.8.7 Side Effects

None specified.

10.8.8 Exceptional Situations

Erroneous if arguments are not acceptable list values.

10.8.9 Examples

- `'(append '(a b c) '(d e f)) => (A B C D E F)'`
- `'(append '((a) (b)) '((c) (d))) => ((A) (B) (C) (D))'`

10.8.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.8.11 Source Notes

- [append (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP-Reference/files/GUID-952B175A-D565-43A4-9208-0E6A27A5E742.htm>)
- Status: documented.

10.9 Function Entry: ASSOC

10.9.1 Name

`'assoc'`

10.9.2 Class

Function

10.9.3 Syntax

`(assoc item alist)`

10.9.4 Arguments and Values

- `'item'`: key to search for
- `'alist'`: association list

10.9.5 Description

Searches an association list for a dotted pair whose first element matches `'item'`.

10.9.6 Return Values

Returns the matching dotted pair or `'nil'`.

10.9.7 Side Effects

None.

10.9.8 Examples

- `'(assoc 'size '((name box) (width 3) (size 4.7263) (depth 5))) => (SIZE 4.7263)'`
- `'(assoc 'weight '((name box) (width 3) (size 4.7263) (depth 5))) => nil'`

10.9.9 Compatibility

This is a central function in CAD database idioms because entity and table data are commonly represented as alists.

The dedicated Autodesk page describes ‘item‘ as an integer, real, or string, but its own examples also use symbols. This inconsistency should be kept visible and eventually tested.

10.9.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.9.11 Source Notes

- [assoc (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP-Reference/files/GUID-46309786-DAF6-4C28-8448-599FBC8A4F6A.htm>)
- Status: documented, with noted item-type inconsistency between prose and examples.

10.10 Function Entry: LAST

10.10.1 Name

`'last'`

10.10.2 Class

Function

10.10.3 Syntax

`(last lst)`

10.10.4 Description

Returns the last element or last cons structure of a list, according to AutoLISP list conventions.

10.10.5 Return Values

Returns the last element in the list.

Autodesk's examples show that if the last element is itself a list, that list is returned as the last element.

10.10.6 Side Effects

None.

10.10.7 Examples

- `'(last '(a b c d e)) => E'`
- `'(last '(a b c (d e))) => (D E)'`

10.10.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.10.9 Source Notes

- [last (AutoLISP)](<https://help.autodesk.com/cloudhelp/2025/ENU/AutoCAD-LT-AutoLISP-Reference/files/GUID-463C97F2-B72F-44C6-B19B-659D231AA836.htm>)
- Status: documented.

10.11 Function Entry: LENGTH

10.11.1 Name

`'length'`

10.11.2 Class

Function

10.11.3 Syntax

`(length item)`

10.11.4 Arguments and Values

- `'item'`: list

10.11.5 Description

Returns the number of elements in a list.

10.11.6 Return Values

Integer length.

10.11.7 Side Effects

None.

10.11.8 Examples

- `'(length '(a b c d)) => 4'`
- `'(length '(a b (c d))) => 3'`
- `'(length '()) => 0'`

10.11.9 Compatibility

The dedicated Autodesk page specifies `'length'` for lists. Acceptance of strings or other sequence-like values should not be assumed without further evidence.

10.11.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.11.11 Source Notes

- [length (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ITA/AutoCAD-AutoLISP-Reference/files/GUID-DD227383-1895-46B6-A83B-43000FCC1018.htm>)
- Status: documented.

10.12 Function Entry: MAPCAR

10.12.1 Name

`'mapcar'`

10.12.2 Class

Function

10.12.3 Syntax

`(mapcar function list [list ...])`

10.12.4 Arguments and Values

- `'function'`: function designator
- one or more lists

10.12.5 Description

Applies `'function'` element-wise across the supplied lists and returns a list of results.

10.12.6 Return Values

List of mapped results.

10.12.7 Side Effects

Side effects are those of applying the supplied function to the elements.

10.12.8 Exceptional Situations

Autodesk documents that the function must accept as many arguments as there are list arguments to `'mapcar'`.

10.12.9 Examples

- `'(mapcar '1+ (list a b c)) => (11 21 31)'` in Autodesk's documented example context
- `'(mapcar '(lambda (x) (+ x 3)) '(10 20 30)) => (13 23 33)'`

10.12.10 Compatibility

Closely tied to the semantics of ‘function’, ‘lambda’, and function-valued designators.

10.12.11 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.12.12 Source Notes

- [mapcar (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-8802AE73-1A05-457E-8A51-09677C23E26E.htm>)
- Status: documented.

10.13 Function Entry: MEMBER

10.13.1 Name

‘member‘

10.13.2 Class

Function

10.13.3 Syntax

(member item lst)

10.13.4 Arguments and Values

- ‘item’: search target
- ‘lst’: list

10.13.5 Description

Searches a list for ‘item‘.

10.13.6 Return Values

Returns the remainder of the list, starting with the first occurrence of ‘item‘, or ‘nil‘ if no match is found.

10.13.7 Side Effects

None.

10.13.8 Examples

- ‘(member ‘c ‘(a b c d e)) => (C D E)‘
- ‘(member ‘q ‘(a b c d e)) => nil‘

10.13.9 Compatibility

Detailed matching predicate semantics still need tighter source coverage.

10.13.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.13.11 Source Notes

- [member (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-A2B08751-D966-44F5-9B02-1AAC4DA6AF59.htm>)
- Status: documented.

10.14 Function Entry: NTH

10.14.1 Name

`'nth'`

10.14.2 Class

Function

10.14.3 Syntax

`(nth n lst)`

10.14.4 Arguments and Values

- `'n'`: index
- `'lst'`: list

10.14.5 Description

Returns the element at index `'n'` in `'lst'`.

10.14.6 Return Values

Element at the specified position, or `'nil'` if `'n'` is greater than the highest element number of the list.

10.14.7 Side Effects

None.

10.14.8 Examples

- `'(nth 3 '(a b c d e)) => D'`
- `'(nth 0 '(a b c d e)) => A'`
- `'(nth 5 '(a b c d e)) => nil'`

10.14.9 Compatibility

Autodesk explicitly documents zero-based indexing for `'nth'`.

10.14.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.14.11 Source Notes

- [nth (AutoLISP)](<https://help.autodesk.com/cloudhelp/2016/ENU/AutoCAD-AutoLISP/files/GUID-0330FE17-6E15-4E34-BB50-E9040EABDADB.htm>)
- Status: documented.

10.15 Function Entry: REVERSE

10.15.1 Name

`'reverse'`

10.15.2 Class

Function

10.15.3 Syntax

`(reverse lst)`

10.15.4 Description

Returns a list containing the elements of `'lst'` in reverse order.

10.15.5 Return Values

Reversed list.

10.15.6 Side Effects

None.

10.15.7 Examples

- `'(reverse '((a) b c)) => (C B (A))'`

10.15.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.15.9 Source Notes

- [reverse (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-MAC-AutoLISP-Reference/files/GUID-7669E8F1-2A4F-42C7-AAA8-74D1300F97.htm>)
- Status: documented.

10.16 Function Entry: SUBST

10.16.1 Name

‘subst‘

10.16.2 Class

Function

10.16.3 Syntax

(subst new old expr)

10.16.4 Arguments and Values

- ‘new’: replacement value
- ‘old’: value to replace
- ‘expr’: tree/list structure

10.16.5 Description

Substitutes ‘new‘ for occurrences of ‘old‘ within ‘expr‘.

10.16.6 Return Values

Returns a copy of the list with ‘new‘ substituted for every occurrence of ‘old‘.

If ‘old‘ is not found, Autodesk documents that the original list is returned unchanged.

10.16.7 Side Effects

None specified.

10.16.8 Examples

- ‘(subst ‘qq ‘b ‘(a b (c d) b)) => (A QQ (C D) QQ)‘
- ‘(subst ‘(qq rr) ‘(c d) ‘(a b (c d) b)) => (A B (QQ RR) B)‘

10.16.9 Compatibility

Exact structural sharing versus reconstruction behavior remains to be tightened from dedicated documentation or tests.

10.16.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.16.11 Source Notes

- [subst (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-25214E69-090A-45C3-8210-6D9801255E44.htm>)
- Status: documented.

10.17 Function Entry: LISTP

10.17.1 Name

`'listp'`

10.17.2 Class

Function

10.17.3 Syntax

`(listp item)`

10.17.4 Arguments and Values

- `'item'`: any AutoLISP value

10.17.5 Description

Determines whether `'item'` is a list.

10.17.6 Return Values

Returns `'T'` if `'item'` is a list; otherwise returns `'nil'`.

Autodesk explicitly documents that because `'nil'` is both an atom and a list, `'(listp nil)'` returns `'T'`.

10.17.7 Side Effects

None.

10.17.8 Examples

- `'(listp '(a b c)) => T'`
- `'(listp 'a) => nil'`
- `'(listp 4.343) => nil'`
- `'(listp nil) => T'`

10.17.9 Notes

This entry is one of the clearest vendor statements that ‘nil’ denotes the empty list in AutoLISP value semantics.

10.17.10 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.17.11 Source Notes

- [listp (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/CHS/AutoCAD-AutoLISP-Reference/files/GUID-8755F083-D3BA-45F0-BBAF-D60D3A73240C.htm>)
- Status: documented.

10.18 Function Entry: VL-CONSP

10.18.1 Name

`'vl-consp'`

10.18.2 Class

Visual LISP Predicate

10.18.3 Syntax

`(vl-consp obj)`

10.18.4 Arguments and Values

- `'obj'`: any AutoLISP value

10.18.5 Description

Tests whether `'obj'` is a non-`'nil'` list/cons-like structure under Visual LISP semantics.

10.18.6 Return Values

- `'T'` if `'obj'` is a cons-like list structure
- `'nil'` otherwise

10.18.7 Side Effects

None.

10.18.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.18.9 Source Notes

- `[vl-consp (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-36CFF983-0020-4300-845B-BA9768F1B4E8.htm>)
- Status: documented.

10.19 Function Entry: VL-EVERY

10.19.1 Name

`'vl-every'`

10.19.2 Class

Visual LISP List Function

10.19.3 Syntax

`(vl-every predicate list [list ...])`

10.19.4 Arguments and Values

- `'predicate'`: subroutine or symbol
- `'list'`: one or more lists

10.19.5 Description

Checks whether the predicate is true for every element combination drawn from the supplied lists.

10.19.6 Return Values

Returns `'T'` if the predicate is true for all tested combinations; otherwise returns `'nil'`.

10.19.7 Side Effects

Side effects are those of calling `'predicate'`.

10.19.8 Notes

- Autodesk documents that `'predicate'` may be supplied as:
 - a symbol naming a function,
 - a quoted lambda form,
 - a `'(function (lambda ...))'` form.
- Autodesk documents that elements are taken in lockstep from the supplied lists.

- Autodesk documents that evaluation stops as soon as one of the lists runs out.

10.19.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.19.10 Source Notes

- [vl-every (AutoLISP)](<https://help.autodesk.com/cloudhelp/2018/DEU/AutoCAD-AutoLISP-Reference/files/GUID-8BF61C9E-1382-4168-A778-FEA394A361CC.htm>)
- Status: documented.

10.20 Function Entry: VL-MEMBER-IF

10.20.1 Name

`'vl-member-if'`

10.20.2 Class

Visual LISP List Function

10.20.3 Syntax

`(vl-member-if predicate list)`

10.20.4 Arguments and Values

- `'predicate'`: subroutine or symbol
- `'list'`: list

10.20.5 Description

Determines whether the predicate is true for one of the list members and returns the corresponding tail of the list under member-style semantics.

10.20.6 Return Values

Returns a list tail beginning at the first matching element, or `'nil'`.

10.20.7 Side Effects

Side effects are those of calling `'predicate'`.

10.20.8 Notes

- Autodesk documents that `'predicate'` may be supplied as:
 - a symbol naming a function,
 - a quoted lambda form,
 - a `'(function (lambda ...))'` form.
- Autodesk documents that each element of `'list'` is passed to `'predicate'` in sequence until a non-`'nil'` result is obtained or the list is exhausted.

10.20.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.20.10 Source Notes

- `[vl-member-if (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP-Reference/files/GUID-CFEA98F0-F6DE-4FF4-839A-0F6C8FBE1htm>)
- Status: documented.

10.21 Function Entry: VL-MEMBER-IF-NOT

10.21.1 Name

`'vl-member-if-not'`

10.21.2 Class

Visual LISP List Function

10.21.3 Syntax

`(vl-member-if-not comparator lst)`

10.21.4 Arguments and Values

- `'comparator'`: a symbol, function, or lambda expression; must return `'nil'` or non-`'nil'`.
- `'lst'`: any list of items.

10.21.5 Description

Returns the tail of `'lst'` starting with the first item for which `'comparator'` returns `'nil'`. Validation of each list item stops at the first element whose `'comparator'` evaluation is `'nil'`, and the remainder of the list from that element onward is returned. If `'comparator'` is non-`'nil'` for every element, returns `'nil'`.

10.21.6 Return Values

The list tail beginning at the first element where `'comparator'` returned `'nil'`, or `'nil'` if every element matches the comparator.

10.21.7 Side Effects

Side effects are those of calling `'comparator'`.

10.21.8 Examples

- `'(vl-member-if-not '(lambda (x) (< x 5)) '(1 2 3 4 5 6 7 8 9 10))'` returns `'(5 6 7 8 9 10)'` — the first element for which `'(< x 5)'` is false is `'5'`, and the tail from `'5'` onward is returned.

10.21.9 Availability

- AutoCAD 2026: documented (Autodesk V Functions Reference; included in the family table).
- BricsCAD V26: documented (BricsCAD V26 dedicated per-symbol page).
- Status: documented in both vendors. Phase-5 closure achieved via direct citation of the BricsCAD V26 per-symbol page, which provides the per-element semantics that Autodesk’s family-level prose only summarises.

10.21.10 See Also

- `vl-member-if` — companion that returns the tail from the first element where the comparator is non-‘nil’.

10.21.11 Source Notes

- [V Functions Reference (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-26FF2BE2-B5BF-4DD1-80D0-000000000000.htm>)
- [vl-member-if-not (BricsCAD V26 DevRef)](https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/vl-member-if-not.htm)
- Status: documented (Phase 5 closure; BricsCAD per-symbol page is the authoritative narrative source for the predicate-stop semantics).

10.22 Function Entry: VL-REMOVE

10.22.1 Name

`'vl-remove'`

10.22.2 Class

Visual LISP List Function

10.22.3 Syntax

`(vl-remove item list)`

10.22.4 Description

Removes elements equal to `'item'` from a list.

10.22.5 Return Values

List with the matching elements removed.

10.22.6 Side Effects

None.

10.22.7 Notes

- Autodesk documents that `'item'` may be of any Lisp data type accepted by the implementation.
- Autodesk documents that all matching elements are removed, not just the first.

10.22.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.22.9 See Also

- BricsCAD analogue: `remove` (chapter 24). Same role; different name.
- Pair documented in ‘vendor-inventory-2026.org’ §10 item 7.

10.22.10 Source Notes

- `[vl-remove (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2016/DEU/AutoCAD-AutoLISP/files/GUID-1BF33827-5C9C-49B2-A21B-656E0F429B21.htm>)
- Status: documented.

10.23 Function Entry: VL-REMOVE-IF

10.23.1 Name

`'vl-remove-if'`

10.23.2 Class

Visual LISP List Function

10.23.3 Syntax

`(vl-remove-if predicate list)`

10.23.4 Arguments and Values

- `'predicate'`: subroutine or symbol
- `'list'`: list

10.23.5 Description

Returns all elements of the supplied list that fail the test function.

10.23.6 Return Values

Filtered list.

10.23.7 Side Effects

Side effects are those of calling `'predicate'`.

10.23.8 Notes

- Autodesk documents that `'predicate'` may be supplied as:
 - a symbol naming a function,
 - a quoted lambda form,
 - a `'(function (lambda ...))'` form.
- Autodesk documents that the result contains all elements for which `'predicate'` returns `'nil'`.

10.23.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.23.10 Source Notes

- [vl-remove-if (AutoLISP)](<https://help.autodesk.com/cloudhelp/2019/ENU/AutoCAD-AutoLISP-Reference/files/GUID-9934F630-4697-47BA-84E6-A0430A334111.html>)
- Status: documented.

10.24 Function Entry: VL-REMOVE-IF-NOT

10.24.1 Name

`'vl-remove-if-not'`

10.24.2 Class

Visual LISP List Function

10.24.3 Syntax

`(vl-remove-if-not predicate list)`

10.24.4 Arguments and Values

- `'predicate'`: subroutine or symbol
- `'list'`: list

10.24.5 Description

Returns all elements of the supplied list that pass the test function.

10.24.6 Return Values

Filtered list.

10.24.7 Side Effects

Side effects are those of calling `'predicate'`.

10.24.8 Notes

- Autodesk documents that `'predicate'` may be supplied as:
 - a symbol naming a function,
 - a quoted lambda form,
 - a `'(function (lambda ...))'` form.
- Autodesk documents that the result contains all elements for which `'predicate'` returns a non-`'nil'` value.

10.24.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.24.10 Source Notes

- [vl-remove-if-not (AutoLISP)](<https://help.autodesk.com/cloudhelp/2015/DEU/AutoCAD-AutoLISP/files/GUID-53D12042-8DE3-4DAA-83BD-8ABB376ACA97.htm>)
- Status: documented.

10.25 Function Entry: VL-SOME

10.25.1 Name

`'vl-some'`

10.25.2 Class

Visual LISP List Function

10.25.3 Syntax

`(vl-some predicate list [list ...])`

10.25.4 Arguments and Values

- `'predicate'`: subroutine or symbol
- `'list'`: one or more lists

10.25.5 Description

Checks whether the predicate is non-`'nil'` for one element combination.

10.25.6 Return Values

Returns the first non-`'nil'` value produced by `'predicate'`, or `'nil'` if none.

10.25.7 Side Effects

Side effects are those of calling `'predicate'`.

10.25.8 Notes

- Autodesk documents that `'predicate'` may be supplied as:
 - a symbol naming a function,
 - a quoted lambda form,
 - a `'(function (lambda ...))'` form.
- Autodesk documents that elements are taken in lockstep from the supplied lists.
- Autodesk documents that evaluation stops as soon as `'predicate'` returns a non-`'nil'` value, or when one of the lists runs out.

10.25.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.25.10 Source Notes

- [vl-some (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-2840F793-DA88-4140-8A9D-EBAC47B62F9D.htm>)
- Status: documented.

10.26 Function Entry: VL-LIST*

10.26.1 Name

`'vl-list*`

10.26.2 Class

Visual LISP List Constructor

10.26.3 Syntax

`(vl-list* arg [arg ...])`

10.26.4 Description

Constructs and returns a list.

10.26.5 Return Values

List or dotted-tail structure according to the final argument, under Lisp list* semantics.

10.26.6 Side Effects

None beyond evaluating the arguments.

10.26.7 Notes

- Autodesk documents that `'vl-list*` is like `'list`, except that the last object is placed in the final `'cdr`.
- Autodesk documents these result cases:
 - a single atom argument yields that atom,
 - all-atom arguments of length two yield a dotted pair,
 - a final atom with more preceding arguments yields a dotted list,
 - a final list argument causes its elements to be appended as the tail.

10.26.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.26.9 Source Notes

- [vl-list* (AutoLISP)](<https://help.autodesk.com/cloudhelp/2021/ENU/AutoCAD-AutoLISP-Reference/files/GUID-959DE11B-95E8-4AC3-BFBD-02406977D343.htm>)
- Status: documented.

10.27 Function Entry: VL-LIST->STRING

10.27.1 Name

`'vl-list->string'`

10.27.2 Class

Visual LISP Conversion Function

10.27.3 Syntax

`(vl-list->string list)`

10.27.4 Description

Combines the characters associated with a list of integers into a string.

10.27.5 Return Values

String.

10.27.6 Side Effects

None.

10.27.7 Compatibility

Character-code interpretation is version-sensitive in Unicode-capable environments.

10.27.8 Notes

- Autodesk documents that the argument is a list of non-negative integers.
- Autodesk documents that in Unicode-capable behavior the accepted integer range is '1' to '65536'.
- Autodesk documents that `'(vl-list->string nil)'` returns the empty string.
- Autodesk documents an AutoCAD 2021 behavior change controlled by `'LISPYSYS'`; earlier behavior accepted only legacy character-code ranges and produced different results for non-ASCII text.

10.27.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.27.10 Source Notes

- [vl-list->string (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PTB/AutoCAD-LT-AutoLISP-Reference/files/GUID-02C7058A-F648-48F5-BAF6-2A62E4E1E1E1.htm>)
- [What's New or Changed with AutoLISP (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-LT-AutoLISP/files/GUID-037BF4D4-755E-4A5C-8136-80E85CCEDF3E.htm>)
- Status: documented.

10.28 Dictionary Coverage Index: List Manipulation Functions

The Autodesk list manipulation family includes at least:

- ‘acad_{strlsort}‘
- ‘append‘
- ‘assoc‘
- ‘caddr‘
- ‘cadr‘
- ‘car‘
- ‘cdr‘
- ‘cons‘
- ‘foreach‘
- ‘last‘

- ‘length‘
- ‘list‘
- ‘listp‘
- ‘mapcar‘
- ‘member‘
- ‘nth‘
- ‘reverse‘
- ‘subst‘
- ‘vl-consp‘
- ‘vl-every‘
- ‘vl-member-if‘
- ‘vl-member-if-not‘
- ‘vl-remove‘
- ‘vl-remove-if‘
- ‘vl-remove-if-not‘
- ‘vl-some‘

Source:

- [List Manipulation Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-00A75AFB-5708-4D0C-B18C-F74DDC4D65FA.htm>)

10.29 Function Entry: CADR

10.29.1 Name

`'cadr'`

10.29.2 Class

Function

10.29.3 Syntax

`(cadr list)`

10.29.4 Arguments and Values

- `'list'`: a list. The function expects two or more elements; with fewer elements the result is `'nil'`.

10.29.5 Description

Returns the second element of `'list'`. Equivalent to `'(car (cdr list))'`. Commonly used to extract the Y coordinate from a 2D or 3D point list.

10.29.6 Return Values

The second element of `'list'`, or `'nil'` if `'list'` has fewer than two elements.

10.29.7 Side Effects

None.

10.29.8 Examples

- After `'(setq pt2 '(5.25 1.0))'`, `'(cadr pt2)'` returns `'1.0'`.
- `'(cadr '(4.0))'` returns `'nil'`.
- `'(cadr '(5.25 1.0 3.0))'` returns `'1.0'`.

10.29.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.29.10 Source Notes

- [cadr (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-F8E9E4F0-218D-4587-9D7E-922BE57C9F9B.htm>)
- Status: documented (Phase 3 body fill).

10.30 Function Entry: CADDR

10.30.1 Name

`'caddr'`

10.30.2 Class

Function

10.30.3 Syntax

`(caddr list)`

10.30.4 Arguments and Values

- `'list'`: a list. The function expects three or more elements; with fewer elements the result is `'nil'`.

10.30.5 Description

Returns the third element of `'list'`. Equivalent to `'(car (cdr (cdr list)))'`. Commonly used to extract the Z coordinate from a 3D point list.

10.30.6 Return Values

The third element of `'list'`, or `'nil'` if `'list'` has fewer than three elements.

10.30.7 Side Effects

None.

10.30.8 Examples

- After `'(setq pt3 '(5.25 1.0 3.0))'`, `'(caddr pt3)'` returns `'3.0'`.
- `'(caddr '(5.25 1.0))'` returns `'nil'`.

10.30.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.

- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.30.10 Source Notes

- [caddr (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-0E81B9F0-7AAD-4BB4-98EB-378AEBE5C1.htm>)
- Status: documented (Phase 3 body fill).

10.31 Function Entry: VL-LIST-LENGTH

10.31.1 Name

`'vl-list-length'`

10.31.2 Class

Visual LISP List Function

10.31.3 Syntax

`(vl-list-length list-or-cons-object)`

10.31.4 Arguments and Values

- `'list-or-cons-object'`: a true (proper) list, a dotted (improper) list, or `'nil'`.

10.31.5 Description

Returns the length of a true list. Unlike Common Lisp's `'length'`, which signals an error on a dotted list, `'vl-list-length'` returns `'nil'` for a dotted list, providing graceful handling of malformed list structures.

10.31.6 Return Values

An integer for proper lists; `'nil'` for dotted lists.

10.31.7 Side Effects

None.

10.31.8 Examples

- `'(vl-list-length nil)'` returns `'0'`.
- `'(vl-list-length '(1 2))'` returns `'2'`.
- `'(vl-list-length '(1 2 . 3))'` returns `'nil'`.

10.31.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

10.31.10 Source Notes

- [vl-list-length (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-5BAF3505-B1E9-49BF-9202-AAD36043E1A1.html>)
- Status: documented (Phase 3 body fill).

11 11 Filenames, Paths, and Files

11.1 Overview

File handling is part of the core AutoLISP runtime surface and is version-sensitive because of encoding support and trusted-path/security behavior.

11.2 Normative Rules: Source-File and File-Stream Encoding

11.2.1 Statement

AutoLISP source files and host file streams have two independent encoding axes:

1. **Source-file encoding** — the encoding the runtime uses when `load` (and any reader entry point that takes a filesystem path) reads bytes from disk. Historically AutoLISP ran on Windows with ANSI / MBCS source files, and that legacy default is observable in pre-2025 AutoCAD and pre-V25 BricsCAD. AutoCAD 2025 changed the default to UTF-8; BricsCAD V25/V26 followed suit. The value of the host system variable `LISPSYS` on Autodesk products selects between two engines: 0 keeps the legacy ANSI/MBCS reader, 1 and 2 enable Unicode and accept the optional **encoding** argument on `open`. `LISPSYS` requires an AutoCAD restart to take effect.
2. **File-stream encoding** — the encoding the runtime uses when `open` reads or writes the bytes of a host file the user is doing I/O on. This is the third (optional) argument to `open`. When the argument is omitted, the per-host default applies — ANSI/MBCS for the legacy engine, UTF-8 for the modern engine.

There is no documented per-string encoding API: AutoLISP strings are sequences of code units in the host engine’s native domain (8-bit code page for the legacy engine, Unicode-capable host string for the modern engine). String **content** is exchanged with files through the encoding configured for that file’s stream.

11.2.2 Encoding Names

The third argument to `open` accepts a small documented vocabulary. Implementations should additionally accept the listed aliases since deployed source uses them interchangeably:

Designator	Maps to (CL external-format)
"utf8"	:utf-8
"utf8-bom"	:utf-8 with byte-order mark
"utf16"	:utf-16
"ANSI"	:iso-8859-1 (the conservative 1-1 mapping that never fails on bytes)
"ASCII"	:ascii
"latin1" / "iso-8859-1"	:iso-8859-1
"cp1252" / "windows-1252"	:cp1252

Hosts derived from the BricsCAD codebase additionally accept the C-style `ccs=NAME` syntax embedded in the second (mode) argument, e.g. (`open path "r,ccs=UTF-8"`). The clautolisp `bricscad-v26` dialect honours this through the existing `open-ccs-mode-p` knob; the `strict` and `autocad-2026` dialects do not.

11.2.3 Per-Dialect Defaults

clautolisp resolves the absent third argument of `open` — and the absent `:external-format` keyword on `load` — using the active dialect descriptor:

Dialect	Default source encoding	Default file encoding
:strict	:iso-8859-1	:iso-8859-1
:autocad-2026	:utf-8	:utf-8
:bricscad-v26	:utf-8	:utf-8

The strict choice is **deliberately permissive at the byte level**: a 1-1 byte coding never raises a decoding error on any input, and existing AutoLISP corpora produced under the legacy ANSI/MBCS engine remain loadable. Source code that needs Unicode characters in literals can either (a) use the explicit `--dialect autocad-2026` / `--bricscad-v26` flag, or (b) pass `:external-format :utf-8` on the call site.

11.2.4 Vendor Evidence

- Autodesk reference for `open` (AutoCAD 2024+) documents "utf8" and "utf8-bom" as the only accepted encoding strings; "When a value isn't provided for the argument, the file is assumed to contain multi-byte character set (MBCS) which is the legacy behavior."
- Autodesk LISPSYS sysvar reference: 0 = ASCII / no encoding arg, 1 = Unicode, 2 = Unicode-with-extra-COM. Restart required.

- AutoCAD 2025 release-note quote: "In the previous versions of AutoCAD, the encoding of the file was 'ANSI', now it is 'UTF-8'" This applies to source files emitted by the Visual LISP IDE.
- BricsCAD V26 honours the C-runtime `ccs=NAME` embedded mode-string as documented in earlier Phase-6 probe evidence (`open-ccs-mode-p`).

11.2.5 Cross-references

- Special Form Entry: `LOAD` (chapter 11) — uses the dialect's source-encoding default.
- Function Entry: `OPEN` (chapter 11) — uses the dialect's file-encoding default when its third argument is omitted.

11.2.6 Source Notes

- [`open` (AutoLISP, AutoCAD 2024)](<https://help.autodesk.com/cloudhelp/2024/ENU/AutoCAD-AutoLISP-Reference/files/GUID-089A323F-21FF-4337-99A9-375758E23htm>)
- [LISPSYS system variable (Autodesk)](<https://help.autodesk.com/cloudhelp/2024/ENU/AutoCAD-Customization/files/GUID-49E8AF16-D31E-4DB1-AB14-E0E9htm>)
- [AutoCAD 2025 — UTF-8 default for AutoLISP files (Autodesk Community)](<https://forums.autodesk.com/t5/net-forum/unicode-characters-problem-in-attachment-p/12703850>)
- [BricsCAD Lisp documentation](<https://help.bricsys.com/en-us/document/bricscad/customization/lisp>)

11.3 Function Entry: OPEN

11.3.1 Name

`'open'`

11.3.2 Class

Function

11.3.3 Syntax

`(open filename mode [encoding])`

11.3.4 Arguments and Values

- `'filename'`: string
- `'mode'`: string designating read, write, or append
- `'encoding'`: optional encoding designator in newer Autodesk versions

11.3.5 Description

Opens a file and returns a file descriptor.

11.3.6 Return Values

Returns a `'FILE'` descriptor or `'nil'` on failure, depending on host behavior and context.

11.3.7 Side Effects

Creates or opens an external file channel.

11.3.8 Affected By

- path-resolution rules
- trusted-path rules
- encoding support

11.3.9 Exceptional Situations

Errors are reported for invalid paths, modes, or inaccessible files.

11.3.10 Examples

- ‘(open "foo.txt" "r")‘

11.3.11 Compatibility

- older Autodesk forms lacked explicit encoding
- BricsCAD documents richer text mode handling

11.3.12 Notes

‘clautolisp‘ should decouple path semantics from native OS via environment profiles.

11.3.13 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.3.14 See Also

- BricsCAD V26 extends ‘open‘ with encoding selectors of the form “r,ccs=UNICODE”, “r,ccs=UTF-8”, “r,ccs=UTF-16LE”, “w,ccs=UTF-8”, “w,ccs=UTF-16LE”. AutoCAD does not accept these; AutoCAD’s Unicode story is governed by the ‘LISPSYS‘ system variable instead.
- Divergence documented in ‘vendor-inventory-2026.org‘ §10 item 2.

11.3.15 Source Notes

- [open (AutoLISP), current](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-089A323F-21FF-4337-99A9-375758...htm>)

- [open (AutoLISP), earlier](<https://help.autodesk.com/cloudhelp/2015/ENU/AutoCAD-AutoLISP/files/GUID-089A323F-21FF-4337-99A9-375758E23BA4.htm>)
- [read + write text files (BricsCAD)](https://developer.bricsys.com/bricscad/help/en_US/V25/DevRef/source/readwritetextfiles.htm)
- Status: documented with version drift.

11.4 Function Entry: CLOSE

11.4.1 Name

`'close'`

11.4.2 Class

Function

11.4.3 Syntax

`(close file-desc)`

11.4.4 Arguments and Values

- `'file-desc'`: open file descriptor

11.4.5 Description

Closes an open file.

11.4.6 Return Values

`'nil'`.

11.4.7 Side Effects

Terminates access through `'file-desc'` and flushes buffered output according to host behavior.

11.4.8 Exceptional Situations

Erroneous if `'file-desc'` is not an open file descriptor.

11.4.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.4.10 Source Notes

- [close (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-5308ADB6-7C46-43BE-8B6C-B32FBA8982DB.htm>)

11.5 Function Entry: FINDFILE

11.5.1 Name

`'findfile'`

11.5.2 Class

Function

11.5.3 Syntax

`(findfile filename)`

11.5.4 Arguments and Values

- `'filename'`: string

11.5.5 Description

Searches the AutoCAD library path for the specified file.

11.5.6 Return Values

Returns a pathname string identifying the located file, or `'nil'`.

11.5.7 Side Effects

None.

11.5.8 Affected By

- library-path configuration
- trusted-path configuration
- selected environment profile

11.5.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.5.10 Source Notes

- `[findfile (AutoLISP)](https://help.autodesk.com/cloudhelp/2024/FRA/AutoCAD-LT-AutoLISP-Reference/files/GUID-D671F67D-F92B-41FF-B9FA-A48EF52CF601.htm)`

11.5.11 Notes

- Autodesk documents that ‘filename’ must not contain a directory prefix.
- Autodesk documents that the support-file search path is searched.
- Autodesk documents that URL strings may also be returned when appropriate.

11.6 Function Entry: FINDTRUSTEDFILE

11.6.1 Name

`'findtrustedfile'`

11.6.2 Class

Function

11.6.3 Syntax

`(findtrustedfile filename)`

11.6.4 Arguments and Values

- `'filename'`: string

11.6.5 Description

Searches the trusted file paths for the specified file.

11.6.6 Return Values

Returns a pathname string identifying the located file, or `'nil'`.

11.6.7 Side Effects

None.

11.6.8 Affected By

- trusted-path configuration
- host path-search semantics

11.6.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.6.10 Source Notes

- `[findtrustedfile (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-2A654E6B-C489-4D70-97CB-CA7D94E65111.html>)

11.6.11 Notes

- Autodesk documents that trusted paths are searched instead of the ordinary support-file search path.
- Autodesk documents that URL strings may also be returned when appropriate.

11.7 Function Entry: VL-DIRECTORY-FILES

11.7.1 Name

`'vl-directory-files'`

11.7.2 Class

Visual LISP File Function

11.7.3 Syntax

`(vl-directory-files [directory [pattern [directories]])`

11.7.4 Arguments and Values

- `'directory'` — optional directory pathname; defaults to the current directory when omitted or `'nil'`.
- `'pattern'` — optional wildcard pattern; defaults to `'*.*'`.
- `'directories'` — optional selector:
 - `'-1'` directories only
 - `'0'` files and directories
 - `'1'` files only

11.7.5 Description

Lists files in a directory, optionally filtered by pattern and directory/file selection.

11.7.6 Return Values

List of strings or `'nil'`.

Autodesk documents that only the matching names are returned, not full pathnames.

11.7.7 Side Effects

None.

11.7.8 Compatibility

Part of the Visual LISP extension layer, but documented as broadly available across Autodesk desktop and web platforms.

11.7.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.7.10 Source Notes

- [vl-directory-files (AutoLISP)](<https://help.autodesk.com/cloudhelp/2025/JPN/AutoCAD-AutoLISP-Reference/files/GUID-C28C0CB0-FBBE-4AA8-BAC4-2FF222772htm>)

11.7.11 Notes

- Autodesk documents a Unicode/LISPSYS behavior change in AutoCAD 2021.

11.8 Function Entry: VL-FILE-COPY

11.8.1 Name

‘vl-file-copy‘

11.8.2 Class

Visual LISP File Function

11.8.3 Syntax

(vl-file-copy source-filename destination-filename [append])

11.8.4 Arguments and Values

- ‘source-filename’: source pathname string
- ‘destination-filename’: destination pathname string
- ‘append’: optional append flag

11.8.5 Description

Copies a file or appends the contents of one file to another.

11.8.6 Return Values

Integer byte count or ‘nil‘.

11.8.7 Side Effects

Creates or modifies the destination file.

11.8.8 Exceptional Situations

Errors are reported for missing source files, inaccessible destinations, or invalid pathnames.

11.8.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.8.10 Source Notes

- [vl-file-copy (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP-Reference/files/GUID-12910252-217C-427D-8874-1323611FA74C.htm>)

11.8.11 Notes

- Autodesk documents that the function does not overwrite an existing destination file unless ‘append’ is non-‘nil’.
- Autodesk documents several ‘nil’ cases including missing source, directory source, existing destination, invalid output name, or write-protected destination.

11.9 Function Entry: VL-FILE-DELETE

11.9.1 Name

‘vl-file-delete‘

11.9.2 Class

Visual LISP File Function

11.9.3 Syntax

(vl-file-delete filename)

11.9.4 Arguments and Values

- ‘filename’: pathname string

11.9.5 Description

Deletes a file.

11.9.6 Return Values

‘T‘ or ‘nil‘.

11.9.7 Side Effects

Deletes the external file named by ‘filename‘.

11.9.8 Exceptional Situations

Errors are reported for missing files, permissions failures, or invalid pathnames.

11.9.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.9.10 Source Notes

- [vl-file-delete (AutoLISP)](<https://help.autodesk.com/cloudhelp/2018/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4CEDC718-EEBB-48A8-A9D3-387F216C1htm>)

11.10 Function Entry: VL-FILE-DIRECTORY-P

11.10.1 Name

`'vl-file-directory-p'`

11.10.2 Class

Visual LISP File Predicate

11.10.3 Syntax

`(vl-file-directory-p filename)`

11.10.4 Arguments and Values

- `'filename'`: pathname string

11.10.5 Description

Determines whether a file name refers to a directory.

11.10.6 Return Values

- `'T'` if `'filename'` designates a directory
- `'nil'` otherwise

11.10.7 Side Effects

None.

11.10.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.10.9 Source Notes

- [vl-file-directory-p (AutoLISP)](<https://help.autodesk.com/cloudhelp/2017/ENU/AutoCAD-AutoLISP-Reference/files/GUID-7EEA5563-33A7-453C-9E96-860F75658111.html>)

11.11 Function Entry: VL-FILE-RENAME

11.11.1 Name

`'vl-file-rename'`

11.11.2 Class

Visual LISP File Function

11.11.3 Syntax

`(vl-file-rename old-filename new-filename)`

11.11.4 Arguments and Values

- `'old-filename'`: existing pathname string
- `'new-filename'`: new pathname string

11.11.5 Description

Renames a file.

11.11.6 Return Values

`'T'` or `'nil'`.

11.11.7 Side Effects

Renames or moves the designated file under host file-system semantics.

11.11.8 Exceptional Situations

Errors are reported for missing files, conflicting destinations, or invalid pathnames.

11.11.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.11.10 Source Notes

- `[vl-file-rename (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-188B957C-4CDF-41C7-99BE-8080FA111110.htm>)

11.11.11 Notes

- Autodesk documents that the function fails if the target file already exists.
- Autodesk documents a Unicode/LISPSYS behavior change in AutoCAD 2021.

11.12 Function Entry: VL-FILE-SIZE

11.12.1 Name

`'vl-file-size'`

11.12.2 Class

Visual LISP File Function

11.12.3 Syntax

`(vl-file-size filename)`

11.12.4 Arguments and Values

- `'filename'`: pathname string

11.12.5 Description

Determines the size of a file in bytes.

11.12.6 Return Values

Integer or `'nil'`.

Autodesk documents:

- `'nil'` if the file is not readable
- `'0'` if `'filename'` names a directory or an empty file

11.12.7 Side Effects

None.

11.12.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.12.9 Source Notes

- [vl-file-size (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-646032BF-CAC1-4166-9EBA-2C954DFF3000.html>)

11.12.10 Notes

- Autodesk documents a Unicode/LISPSYS behavior change in AutoCAD 2021.

11.13 Function Entry: VL-FILE-SYSTIME

11.13.1 Name

`'vl-file-systime'`

11.13.2 Class

Visual LISP File Function

11.13.3 Syntax

`(vl-file-systime filename)`

11.13.4 Arguments and Values

- `'filename'`: pathname string

11.13.5 Description

Returns the last modification time of the specified file.

11.13.6 Return Values

List or `'nil'`.

Autodesk documents a list containing:

- year
- month
- day of week
- day of month
- hours
- minutes
- seconds

11.13.7 Side Effects

None.

11.13.8 Compatibility

Autodesk documents this on Windows and macOS, not Web.

11.13.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.13.10 Source Notes

- [vl-file-systime (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-7F6A31B2-7D51-4C4D-B239-206F398D6111.html>)

11.13.11 Notes

- Autodesk documents a Unicode/LISPSYS behavior change in AutoCAD 2021.

11.14 Function Entry: VL-FILENAME-BASE

11.14.1 Name

‘vl-filename-base‘

11.14.2 Class

Visual LISP Pathname Function

11.14.3 Syntax

(vl-filename-base filename)

11.14.4 Arguments and Values

- ‘filename’: pathname string

11.14.5 Description

Returns the base name of a file after stripping the directory path and extension.

11.14.6 Return Values

String.

Autodesk documents the returned string as uppercase on Windows, with platform-native case behavior on macOS and Web examples.

11.14.7 Side Effects

None.

11.14.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.14.9 Source Notes

- `[vl-filename-base (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/CSY/AutoCAD-LT-AutoLISP-Reference/files/GUID-DA20B039-C252-4751-AA2F-EFC314...htm>)

11.14.10 Notes

- Autodesk documents that the function does not check whether the specified file exists.

11.15 Function Entry: VL-FILENAME-DIRECTORY

11.15.1 Name

`'vl-filename-directory'`

11.15.2 Class

Visual LISP Pathname Function

11.15.3 Syntax

`(vl-filename-directory filename)`

11.15.4 Arguments and Values

- `'filename'`: pathname string

11.15.5 Description

Returns the directory path of a file after stripping the name and extension.

11.15.6 Return Values

String.

Autodesk documents that the empty string is returned when no directory component is present.

11.15.7 Side Effects

None.

11.15.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.15.9 Source Notes

- `[vl-filename-directory (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-AAB7ED2B-C119-4B30-A831-71A4D0...htm>)

11.15.10 Notes

- Autodesk documents that slashes and backslashes are both accepted as directory delimiters.
- Autodesk documents that the function does not check whether the specified file exists.

11.16 Function Entry: VL-FILENAME-EXTENSION

11.16.1 Name

`'vl-filename-extension'`

11.16.2 Class

Visual LISP Pathname Function

11.16.3 Syntax

`(vl-filename-extension filename)`

11.16.4 Arguments and Values

- `'filename'`: pathname string

11.16.5 Description

Returns the extension from a file name after stripping out the rest of the name.

11.16.6 Return Values

String extension or `'nil'` if there is no extension under host semantics.

11.16.7 Side Effects

None.

11.16.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.16.9 Source Notes

- `[vl-filename-extension (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/FRA/AutoCAD-AutoLISP-Reference/files/GUID-6AF5AC95-2E38-4D2C-8A81-8D26F7E8017E.html>)

11.16.10 Notes

- Autodesk documents that the returned extension begins with a period.
- Autodesk documents that the function does not check whether the specified file exists.

11.17 Function Entry: VL-FILENAME-MKTEMP

11.17.1 Name

‘vl-filename-mktemp‘

11.17.2 Class

Visual LISP Pathname Function

11.17.3 Syntax

(vl-filename-mktemp [pattern [directory [extension]])

11.17.4 Arguments and Values

- ‘pattern‘ — optional base pattern string.
- ‘directory‘ — optional directory string.
- ‘extension‘ — optional extension string.

11.17.5 Description

Calculates a unique file name to be used for a temporary file.

11.17.6 Return Values

String.

11.17.7 Side Effects

Allocates or reserves a temporary file name under host semantics; whether a file is also created should be treated as implementation-sensitive until confirmed.

11.17.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

11.17.9 Source Notes

- `[vl-filename-mktemp (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/ESP/AutoCAD-MAC-AutoLISP-Reference/files/GUID-F417A5EF-95BB-47EE-B60E-7C017...htm>)

11.17.10 Notes

- Autodesk documents that the returned file name is unique and that no file is created by the function itself.
- Autodesk documents that the extension argument should not include the leading period.
- Autodesk documents a Unicode/LISPSYS behavior change in AutoCAD 2021.

11.18 Dictionary Coverage Index: File-Handling Functions

The Autodesk file-handling family includes at least:

- ‘close’
- ‘findfile’
- ‘findtrustedfile’
- ‘open’
- ‘read-char’
- ‘read-line’
- ‘vl-directory-files’
- ‘vl-file-copy’
- ‘vl-file-delete’
- ‘vl-file-directory-p’
- ‘vl-file-rename’
- ‘vl-file-size’
- ‘vl-file-systime’

- ‘vl-filename-base‘
- ‘vl-filename-directory‘
- ‘vl-filename-extension‘
- ‘vl-filename-mktemp‘

Source:

- [File-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-F70DECFC-DBE1-4F04-htm>)

11.19 Source Notes

- current and historical ‘open‘ references above

12 12 Streams and Text Input

12.1 Overview

AutoLISP stream behavior is exposed mainly through file descriptors and text/character operations rather than through a large first-class stream protocol.

12.2 Function Entry: READ-CHAR

12.2.1 Name

`'read-char'`

12.2.2 Class

Function

12.2.3 Syntax

`(read-char [file-desc])`

12.2.4 Description

Reads a character from a file descriptor or the keyboard buffer.

12.2.5 Return Values

Returns a character code according to the host's character model.

12.2.6 Compatibility

- Autodesk documents Unicode-related return behavior changes in 2021+
- newline normalization is documented

12.2.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

12.2.8 Source Notes

- `[read-char (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2024/KOR/AutoCAD-AutoLISP-Reference/files/GUID-7E94BD14-F018-47D0-88DA-2B08DE32DB2C.htm>)
- Status: documented.

12.3 Function Entry: READ-LINE

12.3.1 Name

`'read-line'`

12.3.2 Class

Function

12.3.3 Syntax

`(read-line file-desc)`

12.3.4 Description

Reads one line from a file.

12.3.5 Return Values

Returns the line without the end-of-line marker, or `'nil'` at end of file.

12.3.6 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

12.3.7 Source Notes

- `[read-line (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2016/FRA/AutoCAD-MAC-Core/files/GUID-AC74D827-0969-4888-91C0-C3149DEC3659.htm>)
- Status: documented.

12.4 Function Entry: WRITE-CHAR

12.4.1 Name

`'write-char'`

12.4.2 Class

Function

12.4.3 Syntax

`(write-char char [file-desc])`

12.4.4 Arguments and Values

- `'char'`: character value or code under host character semantics
- `'file-desc'`: optional output file descriptor

12.4.5 Description

Writes one character to the command line or to an open file.

12.4.6 Return Values

Host-defined success value.

12.4.7 Side Effects

Produces output to the selected destination.

12.4.8 Compatibility

Character interpretation is version-sensitive in Unicode-capable environments.

12.4.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

12.4.10 Source Notes

- [About File Descriptors (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/CHT/AutoCAD-AutoLISP/files/GUID-EDADF856-4D64-4E19-A7BD-5017C4135.htm>)
- [What's New or Changed with AutoLISP (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/CHS/AutoCAD-MAC-AutoLisp/files/GUID-037BF4D4-755E-4A5C-8136-80E85CCEDF3E.htm>)
- Status: documented.

12.5 Function Entry: WRITE-LINE

12.5.1 Name

‘write-line‘

12.5.2 Class

Function

12.5.3 Syntax

(write-line string [file-desc])

12.5.4 Arguments and Values

- ‘string’: string
- ‘file-desc’: optional output file descriptor

12.5.5 Description

Writes a line of text to the command line or to an open file.

12.5.6 Return Values

Host-defined success value.

12.5.7 Side Effects

Produces line-oriented output and appends an end-of-line marker under host file semantics.

12.5.8 Compatibility

End-of-line representation is an external-file concern and may differ from the implementation host defaults.

12.5.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).

- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

12.5.10 Source Notes

- [write-line (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-CB4F3ABC-F0F6-41DA-A911-75B90D9F974.htm>)
- [About File Descriptors (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/CHT/AutoCAD-AutoLISP/files/GUID-EDADF856-4D64-4E19-A7BD-5017C4135.htm>)
- Status: documented.

12.6 Function Entry: READ

12.6.1 Name

`'read'`

12.6.2 Class

Function

12.6.3 Syntax

`(read string)`

12.6.4 Description

Parses the first AutoLISP expression represented in a string.

12.6.5 Return Values

Returns the first object read from the string.

12.6.6 Notes

`'read'` is the main vendor-documented entry point into surface syntax behavior.

12.6.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

12.6.8 Source Notes

- `[read (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2026/CHS/AutoCAD-MAC-AutoLISP-Reference/files/GUID-5B50BB3E-C244-46E8-85D8-6A2D48B1FE51.htm>)
- Status: documented.

13 13 Reader

13.1 Overview

The language has a de facto reader, but not a fully formal reader standard in the published vendor corpus.

13.2 Normative Rules

- chapter 2 defines the standardized reader subset
- chapter 12 ‘read’ defines a documented string-to-object entry point
- all non-core reader syntax remains under-specified until documented or tested

13.3 Implementation Guidance

‘clautolisp’ should implement an explicit AutoLISP reader with:

- source locations,
- version/dialect switches,
- strict/lax acceptance policy,
- host-independent token semantics.

13.4 Source Notes

- [read (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/CHS/AutoCAD-MAC-AutoLISP-Reference/files/GUID-5B50BB3E-C244-46E8-85D8-6A2D48B1FE51.htm>)
- ‘autolisp-spec/documentation/specification-resolution-plan.org’

14 14 Printer and External Representations

14.1 Overview

Vendor documentation sampled so far is much richer on reading than on a formal printer specification.

14.2 Normative Rules

- examples print symbols in uppercase
- dotted pairs and lists print in conventional Lisp notation
- exact printer controls remain under-specified in this draft

14.3 Function Entry: PRIN1

14.3.1 Name

`'prin1'`

14.3.2 Class

Function

14.3.3 Syntax

`(prin1 [expression [fileHandle]])`

14.3.4 Arguments and Values

- `'expression'` (optional): any AutoLISP expression or value. If omitted, `'nil'` is used.
- `'fileHandle'` (optional): handle of a file as obtained from `'(open ...)'`. If omitted or `'nil'`, output goes to the command line.

14.3.5 Description

Prints `'expression'` to the command line, or writes it to the open file `'fileHandle'`. The representation is **machine-readable**: control characters in a string expression (`'\'`, `"`, `'"`, the escaped double-quote, octal `"`) are written as their backslash escape sequences so that the output round-trips through the reader.

14.3.6 Return Values

Returns the evaluated value of `'expression'`. With no argument, `'(prin1)'` returns the **void** symbol value, useful as the last form in a `'defun'` to emit "nothing" to the command line.

14.3.7 Side Effects

Writes the printed representation to the command line or to `'fileHandle'`.

14.3.8 Examples

With `(setq x 123)`:

- `(prin1 x)` prints `'123'` to the command line and returns `'123'`.
- `(prin1 x fh)` writes `'123'` to file handle `'fh'` and returns `'123'`.

14.3.9 Tested Behaviour (BricsCAD V26 / macOS, 2026-04-26)

`(prin1 "hello" file-desc)` returns `"hello"` and writes the seven characters `"hello"` (the literal seven, including the surrounding double quotes — i.e. the round-trippable representation) to the file. The 2-argument form is fully accepted by BricsCAD V26.

14.3.10 Availability

- AutoCAD 2026: documented (Autodesk reference page).
- BricsCAD V26: documented + product-tested on macOS (probe suite, 2026-04-26). 2-argument form accepted; output is round-trippable.
- Status: documented in both vendors and tested against BricsCAD V26 / macOS.

14.3.11 See Also

- `probe-output.lsp` — executable Phase-5 probe suite for the printer surface.
- probe results, BricsCAD V26 / macOS, 2026-04-26.
- `princ` for display-oriented output (no escape rewriting).
- `print` for the bracketed `prin1` + leading newline + trailing space form.

14.3.12 Source Notes

- `[prin1 (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-BF4CEE45-BB6F-443B-A588-A40D9BCE3.htm>)
- `[prin1 (BricsCAD V26 DevRef)]`(https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/prin1.htm)
- Status: documented and product-tested (Phase 5 closure).

14.4 Function Entry: PRINC

14.4.1 Name

`'princ'`

14.4.2 Class

Function

14.4.3 Syntax

`(princ [expression [fileHandle]])`

14.4.4 Arguments and Values

- `'expression'` (optional): any AutoLISP expression or value. If omitted, `'nil'` is used.
- `'fileHandle'` (optional): handle of a file as obtained from `'(open ...)'`. If omitted or `'nil'`, output goes to the command line.

14.4.5 Description

Prints `'expression'` to the command line, or writes it to the open file `'fileHandle'`. The representation is **display-oriented**: unlike `'prin1'`, control characters in a string expression are written verbatim (no `"` / `"` / escaped-double-quote rewriting). With no argument, `'(princ)'` returns the **void** symbol value and is the conventional way to terminate a `'defun'` without emitting anything.

14.4.6 Return Values

Returns the evaluated value of `'expression'`, or **void** with no argument.

14.4.7 Side Effects

Writes the printed representation to the command line or to `'fileHandle'`.

14.4.8 Examples

With `'(setq x 123)'`:

- `'(princ x)'` prints `'123'` to the command line and returns `'123'`.

- `(princ x fh)` writes `'123'` to file handle `'fh'` and returns `'123'`.

14.4.9 Tested Behaviour (BricsCAD V26 / macOS, 2026-04-26)

`(princ "hello" file-desc)` returns `"hello"` and writes the five characters `'hello'` to the file (no surrounding quotes, no trailing newline). Confirms the verbatim representation, distinct from `'prin1'`'s round-trippable representation.

14.4.10 Availability

- AutoCAD 2026: documented (Autodesk reference page).
- BricsCAD V26: documented + product-tested on macOS (probe suite, 2026-04-26). 2-argument form accepted; output is verbatim (no escape rewriting).
- Status: documented in both vendors and tested against BricsCAD V26 / macOS.

14.4.11 See Also

- `probe-output.lsp` — executable Phase-5 probe suite.
- probe results, BricsCAD V26 / macOS, 2026-04-26.
- `prin1` for the round-trippable form.
- `vl-princ-to-string` for the string-returning companion.

14.4.12 Source Notes

- `[princ (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-7DDA298B-A97C-49C9-94BA-46C77B50AE99.htm>)
- `[princ (BricsCAD V26 DevRef)]`(https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/princ.htm)
- Status: documented and product-tested (Phase 5 closure).

14.5 Function Entry: PRINT

14.5.1 Name

`'print'`

14.5.2 Class

Function

14.5.3 Syntax

`(print [expression [fileHandle]])`

14.5.4 Arguments and Values

- `'expression'` (optional): any AutoLISP expression or value. If omitted, `'nil'` is used.
- `'fileHandle'` (optional): handle of a file as obtained from `'(open ...)'`. If omitted or `'nil'`, output goes to the command line.

14.5.5 Description

Same behaviour as `'prin1'` (control-character escapes; round-trippable representation), but with a leading newline before `'expression'` and an extra space afterwards.

14.5.6 Return Values

Returns the evaluated value of `'expression'`.

14.5.7 Side Effects

Writes one newline, the printed representation of `'expression'`, then a single trailing space.

14.5.8 Examples

With `'(setq x 123)'`:

- `'(print x)'` prints `'<newline>123 '` to the command line and returns `'123'`.

- `(print x fh)` writes `<newline>123` to file handle `fh` and returns `123`.

14.5.9 Tested Behaviour (BricsCAD V26 / macOS, 2026-04-26)

`(print "hello" file-desc)` returns `"hello"` and writes the nine characters `"hello"` (literal newline, then the round-trippable seven characters of `prin1`, then a single trailing space) to the file. Confirms the framing rule from the vendor page.

14.5.10 Availability

- AutoCAD 2026: documented (Autodesk reference page).
- BricsCAD V26: documented + product-tested on macOS (probe suite, 2026-04-26). Leading-newline / trailing-space framing confirmed empirically.
- Status: documented in both vendors and tested against BricsCAD V26 / macOS.

14.5.11 See Also

- `probe-output.lsp` — executable Phase-5 probe suite.
- probe results, BricsCAD V26 / macOS, 2026-04-26.
- `prin1` for the unframed round-trippable form.

14.5.12 Source Notes

- `[print (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4EE74BA9-6ED4-4734-9C47-90A81E0B0971.htm>)
- `[print (BricsCAD V26 DevRef)]`(https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/print.htm)
- Status: documented (Phase 5 closure).

14.6 Function Entry: TERPRI

14.6.1 Name

`'terpri'`

14.6.2 Class

Function

14.6.3 Syntax

`(terpri)`

14.6.4 Arguments and Values

None.

14.6.5 Description

Prints a newline character to the command line. Has no file-handle parameter — the printer-surface family takes a file handle on the per-output-call functions (`'prin1'`, `'princ'`, `'print'`), but `'terpri'` is command-line-only.

14.6.6 Return Values

Always returns `'nil'`.

14.6.7 Side Effects

Emits a newline to the command line.

14.6.8 Notes

Bricsys recommends `'(princ)'` (with no arguments) instead of `'(terpri)'` when ending a `'defun'` body, because `'(princ)'` returns **void** and does not print an extra newline.

14.6.9 Tested Behaviour (BricsCAD V26 / macOS, 2026-04-26)

The probe suite emits `'(terpri stream)'` against an open file handle. BricsCAD V26 reports:

Probe	Outcome
<code>'(terpri file-desc)'</code>	error: 'too few / too many arguments at [TERPRI]'
<code>'(princ "A" file-desc) + (terpri file-desc)'</code>	first call writes 'A'; second call errors with the same mess

This empirically confirms BricsCAD V26's documented zero-arity contract. AutoLISP code that wishes to write a newline to a file handle must use `'(princ "" stream)'` (BricsCAD V26 confirmed acceptance: `'princ-string.txt'` test produced `'hello'`). `'terpri'` is reserved for command-line output. The same code shape ported from Common Lisp via `'(terpri stream)'` will not portably work in AutoLISP.

14.6.10 Availability

- AutoCAD 2026: documented (Autodesk reference page).
- BricsCAD V26: documented + product-tested on macOS (probe suite, 2026-04-26). Arity is exactly zero; the 2-argument form `'(terpri stream)'` is rejected at runtime.
- Status: documented in both vendors and tested against BricsCAD V26 / macOS. Phase-5 closure: arity nailed down (zero arguments, command-line-only) by direct citation **and** product test.

14.6.11 See Also

- `probe-output.lsp` — executable Phase-5 probe suite.
- probe results, BricsCAD V26 / macOS, 2026-04-26.

14.6.12 Source Notes

- `[terpri (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-16FB5411-7563-4FB9-AB59-A73457B09111.htm>)
- `[terpri (BricsCAD V26 DevRef)]`(https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/terpri.htm)
- Status: documented and product-tested (Phase 5 closure).

14.7 Function Entry: PROMPT

14.7.1 Name

`'prompt'`

14.7.2 Class

Function

14.7.3 Syntax

`(prompt message)`

14.7.4 Arguments and Values

- `'message'`: string; the message to display in the active output device(s).

14.7.5 Description

Sends `'message'` to the active output device(s). Bricsys explicitly recommends `'prompt'` over `'prin1'` / `'princ'` / `'print'` for command-line output — `'prompt'` is the dedicated end-user message channel, while the printer functions are general-purpose.

14.7.6 Return Values

Always returns `'nil'`.

14.7.7 Side Effects

Writes the message to the active output channels.

14.7.8 Tested Behaviour (BricsCAD V26 / macOS, 2026-04-26)

`'(prompt "AUTOLISP-SPEC-PROMPT>)"` returns `'nil'`. The string was emitted to the BricsCAD command-line widget (visible to the user) but does not surface through file-handle capture, confirming the documented "active output device(s)" channel choice.

14.7.9 Availability

- AutoCAD 2026: documented (Autodesk reference page).
- BricsCAD V26: documented + product-tested on macOS (probe suite, 2026-04-26). Return value confirmed ‘nil’; output goes to the active command-line channel.
- Status: documented in both vendors and tested against BricsCAD V26 / macOS.

14.7.10 See Also

- probe-output.lsp — executable Phase-5 probe suite.
- probe results, BricsCAD V26 / macOS, 2026-04-26.

14.7.11 Source Notes

- [prompt (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-2F702ECF-FEC7-4DEE-8575-88CDEE05F14E.html>)
- [prompt (BricsCAD V26 DevRef)](https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/prompt.htm)
- Status: documented and product-tested (Phase 5 closure).

14.8 Dictionary Coverage Index: Printer Functions

The core printer/output family includes at least:

- ‘prin1’
- ‘princ’
- ‘print’
- ‘prompt’
- ‘terpri’
- ‘vl-prin1-to-string’
- ‘vl-princ-to-string’

Source:

- [P Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-088AD4D0-7088-4465-8CA5-444444444444.htm>)
- [T Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-424BAAF1-69C6-4DBF-9948-444444444444.htm>)

14.9 Source Notes

- inferred from vendor examples across the reference corpus
- Status: mostly inferred and under-specified.

15 15 Errors and Recovery

15.1 Overview

AutoLISP error handling is specified through ‘*error*’, ‘ERRNO’, and Visual LISP catch-all functions, not through a CL-style condition system.

15.2 Normative ERRNO code table

The integer value of ‘ERRNO’ after a documented failure is normative per the Autodesk “Error Codes Reference (AutoLISP)” table. The table was first published in the AutoCAD 2015 ENU AutoLISP help (GUID ‘97327347-2A13-4CBC-BDBF-979C7F1CABD5’) and has remained stable in later releases (Autodesk no longer publishes a standalone code-list page in the 2026 cloudhelp index, but the per-function pages still reference these codes by number).

Code	Meaning
0	No error / success
1	Invalid symbol-table name
2	Invalid entity or selection-set name
3	Exceeded maximum number of selection sets
4	Invalid selection set
5	Improper use of block definition
6	Improper use of xref
7	Object selection: pick failed
8	End of entity file
9	End of block definition file
10	Failed to find last entity
11	Illegal attempt to delete viewport object
12	Operation not allowed during PLINE
13	Invalid handle
14	Handles not enabled
15	Invalid arguments in coordinate transform request
16	Invalid space in coordinate transform request
17	Invalid use of deleted entity
18	Invalid table name
19	Invalid table function argument
20	Attempt to set a read-only variable
21	Zero value not allowed
22	Value out of range
23	Complex REGEN in progress
24	Attempt to change entity type
25	Bad layer name
26	Bad linetype name
27	Bad color name
28	Bad text style name
29	Bad shape name
30	Bad field for entity type
31	Attempt to modify deleted entity
32	Attempt to modify seqend subentity
33	Attempt to change handle
34	Attempt to modify viewport visibility
35	Entity on locked layer
36	Bad entity type
37	Bad polyline entity
38	Incomplete complex entity in block
39	Invalid block name field
40	Duplicate block flag fields
41	Duplicate block name fields
42	Bad normal vector
43	Missing block name
44	Missing block flags
45	Invalid anonymous block
46	Invalid block definition
47	Mandatory field missing

Per Autodesk's note attached to the 'ERRNO' sysvar entry, "ERRNO is not always cleared to zero. Unless it is inspected immediately after an AutoLISP function has reported an error, the error that its value indicates may be misleading." 'ERRNO' is reset to 0 on the **success** path of every builtin that documents itself as "sets ERRNO on failure"; an arbitrary other-builtin success leaves the previous value in place.

The host implementation in 'clautolisp/autolisp-builtins-core' wires the matching 'set-autolisp-errno' calls into the builtins that Autodesk documents in the 'ERRNO' sysvar entry's coupling list (`entget`, `entmake`, `entmod`, `entsel`, `findfile`, `load`, `nentsel`, `open`, `read-line`, `ssget`, `ssname`, `tablet`, `write-line`, `xdsize`). Builtins outside that list – including the 'entdel' / 'entupd' / 'entlast' / 'entnext' family – do not touch 'ERRNO' because Autodesk's documentation does not specify a behaviour for them.

15.3 Source Notes for the ERRNO table

- [Error Codes Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2015/ENU/AutoCAD-AutoLISP/files/GUID-97327347-2A13-4CBC-BDBF-979C7F1CA1B1.html>)
- [ERRNO (System Variable)](<https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-08E19480-F87B-46E9-9C37-64693F95F680>)
- [BricsCAD V25 ERRNO sysvar](<https://help.bricsys.com/en-us/document/system-variable-reference/e/errno-system-variable>)

15.4 Variable Entry: ERROR

15.4.1 Name

‘*error*‘

15.4.2 Class

Special variable / error hook

15.4.3 Value Type

A function of one string argument, or ‘nil‘.

15.4.4 Initial Value

Implementation-dependent.

15.4.5 Description

When defined, handles errors signaled to the AutoLISP environment.

15.4.6 Recovery and Handling

Typical uses include restoring system variables, cleanup, and filtering cancel messages.

15.4.7 Compatibility

Central across AutoCAD-compatible behavior; also documented by BricsCAD.

15.4.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

15.4.9 Source Notes

- [`*error*` (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-CF913180-17CC-43C7-B89F-3BC82FFBEFB9.htm>)
- [About Using the **error** Function (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP/files/GUID-2AC47A3C-7E80-414D-9134-EABAC.htm>)
- [BricsCAD `*error*`](https://developer.bricsys.com/bricscad/help/en_US/V25/DevRef/source/error.htm)
- Status: documented.

15.5 Condition Entry: Catch-All Error Object

15.5.1 Name

Catch-All Error Object

15.5.2 Class

Visual LISP Error Facility

15.5.3 Triggering Situations

Produced by ‘vl-catch-all-apply’ when an error is trapped.

15.5.4 Reported Form

Accessible via ‘vl-catch-all-error-p’ and ‘vl-catch-all-error-message’.

15.5.5 Recovery and Handling

Allows programmatic continuation after trapped errors.

15.5.6 Compatibility

Part of the Visual LISP extension layer.

15.5.7 Source Notes

- [About Catching Errors and Continuing Program Execution (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP/files/GUID-9DDAD8C4-FB10-4C15-B59E-F6E05C1F9672.htm>)
- Status: documented.

15.6 Function Entry: ALERT

15.6.1 Name

`'alert'`

15.6.2 Class

Function

15.6.3 Syntax

`(alert message)`

15.6.4 Arguments and Values

- `'message'`: string

15.6.5 Description

Displays an alert dialog containing an error or warning message.

15.6.6 Return Values

Host-defined dialog result.

15.6.7 Side Effects

Displays a user-interface alert; on some platforms Autodesk documents degraded behavior rather than a native alert box.

15.6.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

15.6.9 Source Notes

- [Error-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/KOR/AutoCAD-MAC-AutoLISP-Reference/files/GUID-D114F1C4-DE8E-422D-8F3F-C43707B8212B.htm>)
- [About Using the **error** Function (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP/files/GUID-2AC47A3C-7E80-414D-9134-EABAC.htm>)
- Status: documented.

15.7 Function Entry: EXIT

15.7.1 Name

`'exit'`

15.7.2 Class

Function

15.7.3 Syntax

`(exit)`

15.7.4 Arguments and Values

None.

15.7.5 Description

Unconditionally exits the running LISP code and terminates LISP processing for the current invocation. It is a non-local exit from the current LISP evaluation, not a process-level termination of the host CAD application.

15.7.6 Return Values

Does not return normally. Bricsys describes the channel-level behaviour as printing the message `'; error : quit / exit abort'` to the prompt as the LISP top-level handles the abort.

15.7.7 Side Effects

Aborts the running LISP evaluation. Outstanding `"*error"` handlers may run as the abort propagates.

15.7.8 Notes

`'exit'` and `'quit'` are documented by Bricsys with identical signatures and identical channel-level effect (both raise the same `'; error : quit / exit abort'` message). Treat them as alternative spellings of the same primitive in the absence of further evidence; reserved separately so that AutoLISP code that names either one continues to read.

15.7.9 Availability

- AutoCAD 2026: documented (Autodesk Error-Handling Functions Reference).
- BricsCAD V26: documented (BricsCAD V26 dedicated per-symbol page).
- Status: documented in both vendors. Phase-5 closure: BricsCAD's per-symbol page makes the abort-message-channel behaviour explicit and confirms that 'exit' and 'quit' are documented identically.

15.7.10 See Also

- `quit` — companion form; identical signature and effect.
- `*error*` — error handler that runs as the abort propagates.
- `vl-exit-with-error`, `vl-exit-with-value` — namespace-scoped non-local exits for VLX.

15.7.11 Source Notes

- [Error-Handling Functions Reference (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-D114F1C4-DE8E-422D-8F3F-C43707B8212B.htm>)
- [About Using the **error** Function (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP/files/GUID-2AC47A3C-7E80-414D-9134-EABAC.htm>)
- [exit (BricsCAD V26 DevRef)](https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/exit.htm)
- Status: documented (Phase 5 closure).

15.8 Function Entry: QUIT

15.8.1 Name

`'quit'`

15.8.2 Class

Function

15.8.3 Syntax

`(quit)`

15.8.4 Arguments and Values

None.

15.8.5 Description

Unconditionally cancels the running LISP code and terminates LISP processing for the current invocation. Bricsys's per-symbol pages distinguish the verb in prose only — `'exit'` "exits" the LISP run, `'quit'` "cancels" it — but the **signature, abort message, and effect on the host** are identical. Treat as an alias of `'exit'` in implementation.

15.8.6 Return Values

Does not return normally. Channel-level behaviour: prints `'; error : quit / exit abort'` to the prompt.

15.8.7 Side Effects

Aborts the running LISP evaluation; `'*error*'` handlers may run as the abort propagates.

15.8.8 Availability

- AutoCAD 2026: documented (Autodesk Error-Handling Functions Reference).
- BricsCAD V26: documented (BricsCAD V26 dedicated per-symbol page).

- Status: documented in both vendors. Phase-5 closure: the BricsCAD per-symbol page nails the previously open question down to "no semantic distinction beyond the verb" — both forms are documented identically and produce the same abort-channel message. The historical hypothesis that some vendor or release distinguished 'exit' and 'quit' is not supported by current vendor evidence.

15.8.9 See Also

- `exit` — companion form; identical signature and effect.

15.8.10 Source Notes

- [Error-Handling Functions Reference (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-D114F1C4-DE8E-422D-8F3F-C43707B8212B.htm>)
- [About Using the **error** Function (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP/files/GUID-2AC47A3C-7E80-414D-9134-EABAC.htm>)
- [quit (BricsCAD V26 DevRef)](https://developer.bricsys.com/bricscad/help/en_US/V26/DevRef/source/quit.htm)
- Status: documented (Phase 5 closure).

15.9 Function Entry: PUSH-ERROR-USING-COMMAND

15.9.1 Name

`*push-error-using-command*`

15.9.2 Class

Error-handling function

15.9.3 Syntax

`(*push-error-using-command*)`

15.9.4 Description

Marks that a custom `*error*` handler will use `command`.

15.9.5 Return Values

Host-defined success value.

15.9.6 Side Effects

Changes the host error-handling mode.

15.9.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

15.9.8 Source Notes

- [Error-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/HUN/AutoCAD-AutoLISP-Reference/files/GUID-D114F1C4-DE8E-422D-htm>)
- [`*push-error-using-command*` (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-620E034A-9151-427F-htm>)

- Status: documented.

15.10 Function Entry: PUSH-ERROR-USING-STACK

15.10.1 Name

`*push-error-using-stack*`

15.10.2 Class

Error-handling function

15.10.3 Syntax

`(*push-error-using-stack*)`

15.10.4 Description

Marks that a custom `*error*` handler will use AutoLISP stack variables.

15.10.5 Return Values

Host-defined success value.

15.10.6 Side Effects

Changes the host error-handling mode.

15.10.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

15.10.8 Source Notes

- [Error-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/HUN/AutoCAD-AutoLISP-Reference/files/GUID-D114F1C4-DE8E-422D-A4E0-444444444444.htm>)
- [`*push-error-using-stack*` (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-LT-AutoLISP-Reference/files/GUID-C28420C9-2210-4EEC-A4E0-444444444444.htm>)

- Status: documented.

15.11 Function Entry: POP-ERROR-MODE

15.11.1 Name

`*pop-error-mode*`

15.11.2 Class

Error-handling function

15.11.3 Syntax

`(*pop-error-mode*)`

15.11.4 Description

Ends the previous `*push-error-using-command*` or `*push-error-using-stack*` mode.

15.11.5 Return Values

Host-defined success value.

15.11.6 Side Effects

Restores the previous error-handling mode.

15.11.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

15.11.8 Source Notes

- [Error-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/HUN/AutoCAD-AutoLISP-Reference/files/GUID-D114F1C4-DE8E-422D-htm>)
- [`*pop-error-mode*` (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/CHS/AutoCAD-MAC-AutoLISP-Reference/files/GUID-60AFEFAD-1DF5-448F-A2E9-D726-htm>)

- Status: documented.

15.12 Normative Rules

- CL errors must not leak directly to AutoLISP code in ‘clautolisp‘
- AutoLISP-visible errors must support ‘*error*’, ‘vl-catch-all-*, and ‘ERRNO‘-style classification
- exact message identity is not always normative because Autodesk documents multiple valid reported errors in some cases

15.13 Source Notes

- [About Error Handling (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/PLK/AutoCAD-AutoLISP/files/GUID-027AD2E0-5AC5-48DA-B451-112B7EECE8F1.htm>)
- [Error Codes Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/HUN/AutoCAD-AutoLISP/files/GUID-97327347-2A13-4CBC-BDBF-979C7F1CA0A1.htm>)

15.14 Dictionary Coverage Index: Error-Handling Functions

The Autodesk error-handling family includes at least:

- ‘*error*‘
- ‘*pop-error-mode*‘
- ‘*push-error-using-command*‘
- ‘*push-error-using-stack*‘
- ‘alert‘
- ‘exit‘
- ‘quit‘
- ‘vl-catch-all-apply‘
- ‘vl-catch-all-error-message‘
- ‘vl-catch-all-error-p‘

Source:

- [Error-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/HUN/AutoCAD-AutoLISP-Reference/files/GUID-D114F1C4-DE8E-422D-htm>)

16 16 Host Interaction

16.1 Overview

AutoLISP is a host-coupled language. Command execution, system variables, entity access, and selection sets are part of ordinary language practice.

16.2 Normative Rules

- host interaction is a language layer, not merely an external library
- product compatibility and operating-system compatibility must be tracked separately
- selected host-environment profiles may emulate Windows-like behavior on non-Windows hosts

16.3 Source Notes

- [Query and Command Functions Reference](<https://help.autodesk.com/cloudhelp/2016/ENU/AutoCAD-AutoLISP/files/GUID-F9ABE09D-9A71-4C00-8E5D-5BA70...htm>)
- project rule from ‘AGENTS.md’

16.4 Function Entry: LOAD

16.4.1 Name

`'load'`

16.4.2 Class

Function

16.4.3 Syntax

`(load filename [onfailure])`

16.4.4 Arguments and Values

- `'filename'`: source or compiled Lisp file designator
- `'onfailure'`: optional failure-handling argument

16.4.5 Description

Loads an AutoLISP or compatible file into the current environment.

16.4.6 Return Values

If loading succeeds, Autodesk documents the return value as unspecified.

If loading fails:

- the value of `'onfailure'` is returned if that argument is supplied,
- if `'onfailure'` is a valid AutoLISP function, it is evaluated,
- if `'onfailure'` is omitted, an AutoLISP error is signaled.

16.4.7 Side Effects

- reads and evaluates forms from the target file
- may establish functions, variables, and other definitions

16.4.8 Affected By

- search path rules
- trusted-path rules
- environment profile
- file encoding rules
- filename extension search rules

16.4.9 Exceptional Situations

File and load errors are host- and version-sensitive.

16.4.10 Compatibility

Autodesk documents the extension search order used when ‘filename’ has no extension:

1. ‘.vlx’
2. ‘.fas’
3. ‘.lsp’

As soon as a match is found, loading stops and that file is loaded.

16.4.11 Notes

Autodesk also documents that VLX support is Windows-only.

16.4.12 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.4.13 Source Notes

- `[load (AutoLISP)](https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-MAC-AutoLISP-Reference/files/GUID-F3639BAA-FD70-487C-AEB5-9E6096EC0255.htm)`
- Status: documented.

16.5 Function Entry: AUTOLOAD

16.5.1 Name

‘autoload‘

16.5.2 Class

Function

16.5.3 Syntax

(autoload filename function-list)

16.5.4 Arguments and Values

- ‘filename’ — string naming the AutoLISP file to be loaded on demand.
- ‘function-list’ — list of strings naming commands whose first use triggers the load.

16.5.5 Description

Registers deferred loading so that the named file is loaded when one of the named functions is invoked.

16.5.6 Return Values

Always returns ‘nil‘.

16.5.7 Side Effects

Installs autoload stubs or equivalent deferred-loading behavior.

16.5.8 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (AutoLISP-family function; no contradicting page found).
- Status: AutoCAD documented, BricsCAD presumed-both.

16.5.9 Compatibility

Part of the standard application-handling family.

16.5.10 Source Notes

- [autoload (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-421B36DE-38EA-4161-9768-01647B5492E8.htm>)

16.5.11 Notes

- Autodesk documents that ‘filename’ may omit both the path and the extension.
- Autodesk documents the extension search set as ‘.lsp’, ‘.fas’, or ‘.vlx’ on Windows, and ‘.lsp’ or ‘.fas’ on macOS.
- Autodesk documents secure-mode restrictions beginning with AutoCAD 2014-based products through ‘SECURELOAD’ and ‘TRUST-EDPATHS’.

16.6 Function Entry: GETVAR

16.6.1 Name

`'getvar'`

16.6.2 Class

Host Interaction Function

16.6.3 Syntax

`(getvar varname)`

16.6.4 Arguments and Values

- `'varname'`: system-variable name designator, usually a string

Autodesk documents `'varname'` as a string naming a system variable.

16.6.5 Description

Retrieves the value of an AutoCAD system variable.

The per-sysvar entries in §16 **System Variables** document every `varname` recognized by AutoCAD 2026 or BricsCAD V25, with the declared type, default, read-only flag, scope, vendor divergence, and source URLs. The normative rules at the head of that chapter (name case-insensitivity, integer clamping, unknown-name semantics, read-only behaviour, angular-units convention) apply uniformly to every `getvar` call.

16.6.6 Return Values

Integer, real, string, list, or `'nil'`.

Autodesk documents that `'nil'` is returned if `'varname'` is not a valid system variable.

16.6.7 Side Effects

None.

16.6.8 Affected By

- host drawing/session state
- selected environment profile

16.6.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.6.10 Source Notes

- [getvar (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP-Reference/files/GUID-9C56BEA8-D473-4305-9E17-BEF7630C3341.htm>)
- [About System and Environment Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/JPN/AutoCAD-MAC-AutoLisp/files/GUID-529819FD-9E9C-4789-BA18-B981BA32DD23.htm>)
- See §16 System Variables for the per-name catalogue and normative rules.

16.6.11 Compatibility

Supported on Windows, macOS, and Web in current Autodesk documentation.

16.7 Function Entry: SETVAR

16.7.1 Name

‘setvar‘

16.7.2 Class

Host Interaction Function

16.7.3 Syntax

(setvar varname value)

16.7.4 Arguments and Values

- ‘varname’: system-variable name designator
- ‘value’: new value

Autodesk documents:

- ‘varname’ as a string
- ‘value’ as an integer, real, string, list, ‘T’, or ‘nil’

16.7.5 Description

Sets an AutoCAD system variable to a specified value.

The per-sysvar entries in §16 **System Variables** document every **varname** recognized by AutoCAD 2026 or BricsCAD V25, with the declared type, default, read-only flag, scope, accepted-value range, vendor divergence, and source URLs. The normative rules at the head of that chapter (name case-insensitivity, integer clamping to [-32768, +32767], type-coercion policy, read-only error behaviour, angular-units convention for **ANGBASE** / **SNAPANG**) apply uniformly to every **setvar** call.

16.7.6 Return Values

Returns ‘value’ if successful.

16.7.7 Side Effects

Mutates host drawing/session state.

16.7.8 Exceptional Situations

Errors if the variable is read-only, absent, or rejects the supplied value.

16.7.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.7.10 Source Notes

- [setvar (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/HUN/AutoCAD-AutoLISP-Reference/files/GUID-B1CE1BFB-2448-47F1-95EC-10E9509F01FA.htm>)
- [About System and Environment Variables (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/JPN/AutoCAD-MAC-AutoLisp/files/GUID-529819FD-9E9C-4789-BA18-B981BA32DD23.htm>)
- See §16 System Variables for the per-name catalogue and normative rules.

16.7.11 Notes

- Autodesk documents that for integer-valued system variables the supplied value must lie between ‘-32768’ and ‘+32767’.
- Autodesk documents that changes made with ‘setvar’ during an active command may not take effect until the next command.
- Autodesk documents special angular semantics for ‘ANGBASE’ and ‘SNAPANG’: ‘setvar’ interprets the values in radians, unlike the interactive ‘SETVAR’ command.

16.7.12 Compatibility

Supported on Windows, macOS, and Web in current Autodesk documentation.

16.8 Function Entry: COMMAND-S

16.8.1 Name

`'command-s'`

16.8.2 Class

Special command-interaction function

16.8.3 Syntax

`(command-s [argument ...])`

Autodesk documents the first argument as the command name:

`(command-s [cmdname [arguments ...]])`

16.8.4 Description

Executes an AutoCAD command and the supplied input.

16.8.5 Return Values

Returns `'nil'` when the command finishes successfully on the supplied arguments.

16.8.6 Side Effects

Invokes command processing and may modify the drawing, command stack, and UI state.

16.8.7 Compatibility

Closely related to `'command'`, but documented separately by Autodesk. Precise sequencing differences should be treated as host-significant.

16.8.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.8.9 Source Notes

- [command-s (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-5C9DC003-3DD2-4770-95E7-7E19A4EE1htm>)
- [About Using AutoCAD Commands (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/JPN/AutoCAD-AutoLISP/files/GUID-75F19C76-78D0-443B-BC39-EB9FEhtm>)

16.8.10 Notes

- Autodesk documents command arguments as strings, reals, integers, and lists/points matching the command prompt sequence.
- Autodesk documents that an empty string is equivalent to pressing Enter.
- Autodesk documents that failure to complete successfully is reported through “*error*”.

16.8.11 Compatibility

Supported on Windows and macOS in current Autodesk documentation.

16.9 Function Entry: VL-CMDF

16.9.1 Name

‘vl-cmdf‘

16.9.2 Class

Visual LISP Command Function

16.9.3 Syntax

(vl-cmdf [argument ...])

16.9.4 Arguments and Values

- ‘argument‘ — strings, reals, integers, lists/points, or entity names, as required by the target command prompt sequence.

16.9.5 Description

Executes an AutoCAD command.

16.9.6 Return Values

Always returns ‘T‘.

16.9.7 Side Effects

Invokes command processing.

16.9.8 Compatibility

Part of the Visual LISP command layer and related to ‘command‘ and ‘command-s‘.

16.9.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.9.10 Source Notes

- `[vl-cmdf (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-6F1AAB6B-D5B1-426A-A463-0CBE93E4D956.htm>)

16.9.11 Notes

- Autodesk documents that ‘`vl-cmdf`’ evaluates all arguments before command execution begins.
- Autodesk contrasts this with ‘`command`’, which may partially execute before an argument-evaluation error is detected.
- Autodesk documents that an empty string is equivalent to pressing Enter, and a no-argument call is equivalent to pressing Esc for most commands.

16.9.12 Compatibility

Supported on Windows, macOS, and Web in current Autodesk documentation.

16.10 Function Entry: INITDIA

16.10.1 Name

`'initdia'`

16.10.2 Class

Application-handling function

16.10.3 Syntax

`(initdia)`

Autodesk documents the fuller syntax as:

`(initdia [dialogflag])`

16.10.4 Description

Forces the next compatible command to use a dialog box rather than command-line prompting.

16.10.5 Return Values

Always returns `'nil'`.

16.10.6 Side Effects

Changes host command-invocation behavior for the next applicable command.

16.10.7 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.10.8 Source Notes

- [initdia (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/JPN/AutoCAD-MAC-AutoLISP-Reference/files/GUID-D0F621C5-6D32-4D79-BC49-290CC4D0F1.htm>)

16.10.9 Notes

- If ‘dialogflag’ is omitted or nonzero, Autodesk documents that the next supported command uses its dialog box.
- If ‘dialogflag’ is ‘0’, Autodesk documents that the effect is cleared.
- Autodesk documents a bounded set of commands that honor ‘initdia’, such as ‘PLOT’, ‘LAYER’, ‘STYLE’, and ‘INSERT’.

16.10.10 Compatibility

Supported on Windows and macOS, not Web.

16.11 Function Entry: STARTAPP

16.11.1 Name

`'startapp'`

16.11.2 Class

Application-handling function

16.11.3 Syntax

`(startapp appcmd [file])`

16.11.4 Arguments and Values

- `'appcmd'`: application command or executable designator
- `'file'`: optional file or argument string

Autodesk documents both as strings.

16.11.5 Description

Starts an external application.

16.11.6 Return Values

Integer greater than `'0'`, or `'nil'`.

16.11.7 Side Effects

Launches an external application or process.

16.11.8 Compatibility

Highly host-environment-sensitive. `'clautolisp'` should model this through the selected environment profile rather than by exposing raw native-host behavior.

16.11.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.11.10 Source Notes

- [startapp (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-30EC493A-674B-4AAE-8C88-358E38CDAB21.htm>)

16.11.11 Notes

- Autodesk documents that if ‘appcmd’ is not a full path, the host searches ‘PATH’ or the platform equivalent.
- Autodesk documents that arguments containing spaces must be quoted explicitly inside the string.

16.11.12 Compatibility

Supported on Windows and macOS, not Web.

16.12 Function Entry: VL-LOAD-ALL

16.12.1 Name

`'vl-load-all'`

16.12.2 Class

Visual LISP application-loading function

16.12.3 Syntax

`(vl-load-all filename)`

16.12.4 Arguments and Values

- `'filename'` — string naming the file to load.

16.12.5 Description

Loads a Visual LISP application for all open documents under host semantics.

16.12.6 Return Values

Returns `'nil'` on success; otherwise signals an error.

16.12.7 Side Effects

Loads application code into multiple document contexts.

16.12.8 Compatibility

Part of the Visual LISP application-loading layer and more document-model-sensitive than `'load'`.

16.12.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.12.10 Source Notes

- [vl-load-all (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/JPN/AutoCAD-MAC-AutoLISP-Reference/files/GUID-116F4D02-4B1A-4298-B38C-5B38FD9828.htm>)
- [To load functions across all document namespaces (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/KOR/AutoCAD-LT-AutoLISP/files/GUID-08F778AF-6A57-41AA-8AF5-3E56FEB1958B.htm>)
- [Namespace Communication Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2025/ITA/AutoCAD-MAC-AutoLISP-Reference/files/GUID-F5DD5472-1695-4CA3-9110-E6008E87BB89.htm>)

16.12.11 Notes

- Autodesk documents that the filename extension is required; unlike ‘load’, ‘vl-load-all’ does not add one.
- Autodesk documents that the file is loaded into all open documents and into documents opened later in the same session.

16.12.12 Compatibility

Supported on Windows, macOS, and Web in current Autodesk documentation.

16.13 Function Entry: ARXLOAD

16.13.1 Name

`'arxload'`

16.13.2 Class

Application-handling function

16.13.3 Syntax

`(arxload appname)`

16.13.4 Arguments and Values

- `'appname'`: ObjectARX application designator

Autodesk documents the actual signature as:

`(arxload application [onfailure])`

with:

- `'application'` — string naming the ObjectARX executable or bundle
- `'onfailure'` — expression to evaluate if loading fails

16.13.5 Description

Loads an ObjectARX application.

16.13.6 Return Values

String naming the application if successful.

If loading fails and `'onfailure'` is supplied, Autodesk documents that the value of `'onfailure'` is returned; otherwise an error is reported.

16.13.7 Side Effects

Loads host extension code into the running CAD session.

16.13.8 Compatibility

Strongly host-specific and outside portable core AutoLISP semantics.

16.13.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.13.10 Source Notes

- [arxload (AutoLISP)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-965A0D2A-CFD0-4D7C-9D2B-2D8188F0DAC8.htm>)

16.13.11 Notes

- Autodesk documents that the file extension may be omitted: ‘arx’ on Windows and ‘.bundle’ on macOS.
- Autodesk documents that a full path is required unless the application is in the support-file search path.
- Autodesk documents that loading an already loaded application is an error, and suggests checking with ‘arx’ first.
- Autodesk documents secure-mode restrictions through ‘SECURELOAD’ and ‘TRUSTEDPATHS’.

16.13.12 Compatibility

Supported on Windows and macOS, not Web.

16.14 Function Entry: ARXUNLOAD

16.14.1 Name

`'arxunload'`

16.14.2 Class

Application-handling function

16.14.3 Syntax

`(arxunload appname)`

16.14.4 Arguments and Values

- `'appname'`: ObjectARX application designator

Autodesk documents the actual signature as:

`(arxunload application [onfailure])`

with:

- `'application'` — string naming an application previously loaded with `'arxload'`
- `'onfailure'` — expression to evaluate if unloading fails

16.14.5 Description

Unloads an ObjectARX application.

16.14.6 Return Values

String naming the application if successful.

If unloading fails and `'onfailure'` is supplied, Autodesk documents that the value of `'onfailure'` is returned; otherwise an error is reported.

16.14.7 Side Effects

Removes host extension code from the running CAD session.

16.14.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.14.9 Source Notes

- [arxunload (AutoLISP)](<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-MAC-AutoLISP-Reference/files/GUID-29092087-D1F7-4DF5-863C-FAF4E4F1B012.html>)

16.14.10 Notes

- Autodesk documents that locked ObjectARX applications cannot be unloaded, and that they are locked by default.
- Autodesk documents that the extension may be omitted: ‘arx’ or ‘crx’ on Windows, ‘bundle’ on macOS.

16.14.11 Compatibility

Supported on Windows and macOS only.

16.15 Function Entry: AUTOARXLOAD

16.15.1 Name

`'autoarxload'`

16.15.2 Class

Application-handling function

16.15.3 Syntax

`(autoarxload filename command-list)`

16.15.4 Arguments and Values

- `'filename'` — string naming the ObjectARX file to be loaded on demand.
- `'command-list'` — list of strings naming commands that trigger the load.

16.15.5 Description

Predefines command names that trigger loading of an associated ObjectARX file.

16.15.6 Return Values

Always returns `'nil'`.

16.15.7 Side Effects

Installs deferred-loading stubs for host extension commands.

16.15.8 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.15.9 Source Notes

- `[autoarxload (AutoLISP)]`(<https://help.autodesk.com/cloudhelp/2023/ENU/AutoCAD-AutoLISP-Reference/files/GUID-E1DDBABF-6F72-4747-B9E2-ACEEEA9FA7.html>)

16.15.10 Notes

- Autodesk documents that the path and extension may be omitted, in which case the support-file search path is used and ‘.arx’ or ‘.bundle’ is supplied by platform convention.
- Autodesk documents that on first use of a listed command, the application is loaded and the command then continues.
- Autodesk documents that secure-mode restrictions apply through ‘SECURELOAD’ and ‘TRUSTEDPATHS’.

16.15.11 Compatibility

Supported on Windows and macOS, not Web.

16.16 Function Entry: SHOWHTMLMODALWINDOW

16.16.1 Name

`'showhtmlmodalwindow'`

16.16.2 Class

Application/UI function

16.16.3 Syntax

`(showhtmlmodalwindow uri)`

16.16.4 Arguments and Values

- `'uri'` — string URL or other URI naming the page to display.

16.16.5 Description

Displays a modal dialog box with a specified URI.

16.16.6 Return Values

Always returns `'nil'`.

16.16.7 Side Effects

Displays modal user interface content based on a URI.

16.16.8 Compatibility

Highly host-UI-specific and potentially version-sensitive.

16.16.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.16.10 See Also

- AutoCAD-Windows-only. BricsCAD has no documented equivalent. Divergence documented in ‘vendor-inventory-2026.org’ §10 item 10.

16.16.11 Source Notes

- [showhtmlmodalwindow (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/JPN/AutoCAD-LT-AutoLISP-Reference/files/GUID-1330BB1E-866E-419A-87D9-BF0C9B000000.html>)

16.16.12 Notes

- Autodesk documents that the dialog’s size and position are automatically persisted across sessions.
- Autodesk documents that the loaded page may use the AutoCAD JavaScript API.

16.16.13 Compatibility

Available on Windows only.

16.17 Function Entry: VL-VBALOAD

16.17.1 Name

`'vl-vbaload'`

16.17.2 Class

Visual LISP application-loading function

16.17.3 Syntax

`(vl-vbaload project)`

16.17.4 Arguments and Values

- `'project'` — string naming the VBA project file.

16.17.5 Description

Loads a VBA project.

16.17.6 Return Values

String containing the file name and path of the loaded project.

16.17.7 Side Effects

Loads VBA project state into the host environment.

16.17.8 Compatibility

Host- and platform-dependent legacy integration facility.

16.17.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.17.10 See Also

- AutoCAD-Windows-only. No BricsCAD analogue: BricsCAD does not embed a VBA module callable from LISP under this name. Divergence documented in ‘vendor-inventory-2026.org’ §10 item 9.

16.17.11 Source Notes

- [vl-vbaload (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/PLK/AutoCAD-LT-AutoLISP-Reference/files/GUID-54FD698E-8D94-4E27-9DF1-3F7E9D451E0A.html>)

16.17.12 Notes

- Autodesk documents that an error occurs if the file is not found.
- Autodesk documents a Unicode-related change in AutoCAD 2021 controlled by ‘LISPSYS’.
- Autodesk documents secure-mode restrictions through ‘SECURELOAD’ and ‘TRUSTEDPATHS’.

16.17.13 Compatibility

Available on AutoCAD for Windows only; not available in AutoCAD LT for Windows, or on macOS or Web.

16.18 Function Entry: VL-VBARUN

16.18.1 Name

`'vl-vbarun'`

16.18.2 Class

Visual LISP application function

16.18.3 Syntax

`(vl-vbarun macro-name)`

16.18.4 Arguments and Values

- `'macro-name'` — string naming a macro in a loaded VBA project.

16.18.5 Description

Runs a VBA macro.

16.18.6 Return Values

Returns `'macro-name'` if successful.

16.18.7 Side Effects

Invokes host VBA automation.

16.18.8 Compatibility

Host- and platform-dependent legacy integration facility.

16.18.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.18.10 See Also

- AutoCAD-Windows-only. No BricsCAD analogue. Divergence documented in ‘vendor-inventory-2026.org’ §10 item 9.

16.18.11 Source Notes

- [vl-vbarun (AutoLISP)](<https://help.autodesk.com/cloudhelp/2024/ENU/AutoCAD-LT-AutoLISP-Reference/files/GUID-75387617-9144-49CB-97E4-03B4CD29973.htm>)
- [To run a VBA macro](<https://help.autodesk.com/cloudhelp/2024/JPN/AutoCAD-Customization/files/GUID-E50E8491-5BCB-4202-B3A9-BF6FD9CF68A3.htm>)

16.18.12 Notes

- Autodesk documents the macro naming syntax as ‘<project!module.macro>’ when a fully qualified macro name is needed.

16.18.13 Compatibility

Available on AutoCAD for Windows only; not available in AutoCAD LT for Windows, or on macOS or Web.

16.19 Function Entry: VLAX-ADD-CMD

16.19.1 Name

`'vlax-add-cmd'`

16.19.2 Class

Visual LISP command-registration function

16.19.3 Syntax

```
(vlax-add-cmd global-name function-name [cmd-flags])
```

Autodesk documents the fuller syntax as:

```
(vlax-add-cmd global-name func-sym [local-name cmd-flags])
```

16.19.4 Arguments and Values

- `'global-name'` — string naming the global command.
- `'func-sym'` — symbol naming a no-argument AutoLISP function.
- `'local-name'` — optional string naming the local command alias; defaults to `'global-name'`.
- `'cmd-flags'` — optional integer command flag word.

16.19.5 Description

Adds commands to the AutoCAD built-in command set.

16.19.6 Return Values

String naming the global command if successful.

16.19.7 Side Effects

Registers new commands in the host command system.

16.19.8 Compatibility

Part of the host integration layer rather than portable core language semantics.

16.19.9 Availability

- AutoCAD 2026: documented (per Autodesk reference page).
- BricsCAD V26: presumed compatible (no contradicting page found).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.19.10 Source Notes

- [vlax-add-cmd (AutoLISP/ActiveX)](<https://help.autodesk.com/cloudhelp/2022/ENU/AutoCAD-AutoLISP-Reference/files/GUID-42392DA4-4A2B-4D34-AA7B-A5ACAF727htm>)

16.19.11 Notes

- Autodesk documents modal and transparent command flags, plus additional flags such as pickset use.
- Autodesk documents that commands are assigned to command groups automatically.
- Autodesk documents that the function should not be used to expose functions that create reactor objects or serve as reactor callbacks.

16.19.12 Compatibility

Supported on Windows, macOS, and Web in current Autodesk documentation.

16.20 Dictionary Coverage Index: Application-Handling Functions

The Autodesk application-handling family includes at least:

- ‘arx‘
- ‘arxload‘
- ‘arxunload‘
- ‘autoarxload‘
- ‘autoload‘

- ‘initdia‘
- ‘load‘
- ‘showhtmlmodalwindow‘
- ‘startapp‘
- ‘vl-load-all‘
- ‘vl-vbaload‘
- ‘vl-vbarun‘
- ‘vlax-add-cmd‘

Source:

- [Application-Handling Functions Reference (AutoLISP)](<https://help.autodesk.com/cloudhelp/2026/CHS/AutoCAD-AutoLISP-Reference/files/GUID-E2B683FE-43FB-4F5D-A9AE-809772FE8D3B.htm>)

16.21 Function Entry: TRANS

16.21.1 Name

‘trans‘

16.21.2 Class

Function

16.21.3 Syntax

(trans pt from to [disp])

16.21.4 Arguments and Values

- ‘pt’: a list of three reals, interpreted as a 3D point or as a 3D displacement.
- ‘from’: an integer, list, or ename naming the source coordinate system. Designators are:
 - ‘0’: WCS — World Coordinate System
 - ‘1’: UCS — current User Coordinate System
 - ‘2’: DCS — viewport Display Coordinate System
 - ‘3’: PSDCS — paper-space DCS (only valid as the **to** designator and only from ‘2’)
 - an ename: the Object Coordinate System of that entity
 - a list (extrusion vector): the OCS implied by that extrusion
- ‘to’: same designator vocabulary as ‘from’.
- ‘disp’ (optional): when non-‘nil’, treats ‘pt’ as a displacement rather than a point (so translation components of the transform are skipped).

16.21.5 Description

Translates a point or a displacement from one coordinate system to another. When converting between systems whose Z-axes differ, this function applies the proper rotation.

16.21.6 Return Values

A 3D point (or displacement) expressed in the ‘to’ coordinate system.

16.21.7 Side Effects

None.

16.21.8 Examples

- ‘(trans ’(1.0 2.0 3.0) 0 1)’ returns ‘(2.0 -1.0 3.0)’ when the UCS is rotated 90° counter-clockwise relative to the WCS.
- ‘(trans text-insert-point text-ename 1)’ converts a text-entity insertion point from its OCS to the current UCS.
- ‘(trans ’(1 2 3) 1 circle-ename)’ converts a UCS displacement to a circle’s OCS.

16.21.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.21.10 Source Notes

- [trans (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-1A316343-0B68-4DBE-8F49-B4D601CB8FCC.htm>)
- Status: documented (Phase 3 body fill).

16.22 Function Entry: GETENV

16.22.1 Name

`'getenv'`

16.22.2 Class

Function

16.22.3 Syntax

`(getenv variable-name)`

16.22.4 Arguments and Values

- `'variable-name'`: string; the environment-variable name. Names must be spelled and cased exactly as stored in the system registry.

16.22.5 Description

Returns the value bound to the named environment variable. Returns `'nil'` if the variable is not defined.

16.22.6 Return Values

A string, or `'nil'`.

16.22.7 Side Effects

None.

16.22.8 Examples

- After ACAD is set to `'/acad/support'`: `(getenv "ACAD")` returns `"/acad/support"`.
- For an undefined variable: `(getenv "NOSUCH")` returns `'nil'`.
- `(getenv "MaxArray")` returns `"10000"` if the variable is set to that value.

16.22.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.22.10 Source Notes

- [getenv (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-0796D4FE-C07B-4DB6-9C01-9D772CAFA111.htm>)
- Status: documented (Phase 3 body fill).

16.23 Function Entry: SETENV

16.23.1 Name

`'setenv'`

16.23.2 Class

Function

16.23.3 Syntax

`(setenv varname value)`

16.23.4 Arguments and Values

- `'varname'`: string; the environment-variable name (must match registry casing exactly).
- `'value'`: string; the value to assign.

16.23.5 Description

Sets the named environment variable to the given string. Some settings might not take effect until the next time AutoCAD is started.

16.23.6 Return Values

The `'value'` argument as a string.

16.23.7 Side Effects

Updates the environment-variable store.

16.23.8 Examples

- `'(setenv "MaxArray" "10000")'` returns `"10000"`.

16.23.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.

- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.23.10 Source Notes

- [setenv (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-0C2E8222-62A8-4529-8A8A-58AAB2A5F1A1.html>)
- Status: documented (Phase 3 body fill).

16.24 Function Entry: GETCFG

16.24.1 Name

`'getcfg'`

16.24.2 Class

Function (obsolete)

16.24.3 Syntax

`(getcfg cfgname)`

16.24.4 Arguments and Values

- `'cfgname'`: string of the form `"AppData/applicationname/sectionname/.../paramname"`, up to 496 characters.

16.24.5 Description

Obsolete. Retrieves application data from the `'AppData'` section of the `'acad20xx.cfg'` file. Autodesk recommends `'vl-registry-read'` for new code.

16.24.6 Return Values

The stored string on success, or `'nil'` if `'cfgname'` is invalid.

16.24.7 Side Effects

None.

16.24.8 Examples

- `'(getcfg "AppData/ArchStuff/WallThk")'` returns `"8"` when `WallThk` is set to 8.

16.24.9 Availability

- AutoCAD 2026: documented (obsolete; use `'vl-registry-read'`).
- BricsCAD V26: presumed compatible (obsolete behaviour).
- Status: AutoCAD documented as obsolete; BricsCAD presumed-both pending Phase 4 verification.

16.24.10 Source Notes

- [getcfg (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-D8D46A8C-82DF-4193-A1B7-3DC3CAC70...htm>)
- Status: documented (Phase 3 body fill).

16.25 Function Entry: SETCFG

16.25.1 Name

`'setcfg'`

16.25.2 Class

Function (obsolete)

16.25.3 Syntax

`(setcfg cfgname cfgval)`

16.25.4 Arguments and Values

- `'cfgname'`: string of the form `"AppData/applicationname/sectionname/.../paramname"`, up to 496 characters.
- `'cfgval'`: string up to 512 characters; longer values are written but cannot be retrieved.

16.25.5 Description

Obsolete. Writes application data to the `'AppData'` section of the `'acad20xx.cfg'` file. Autodesk recommends `'vl-registry-write'` for new code.

16.25.6 Return Values

The `'cfgval'` string on success, or `'nil'` if `'cfgname'` is invalid.

16.25.7 Side Effects

Writes to the AutoCAD configuration file.

16.25.8 Examples

- `'(setcfg "AppData/ArchStuff/WallThk" "8")'` returns `"8"`.

16.25.9 Availability

- AutoCAD 2026: documented (obsolete; use ‘vl-registry-write’).
- BricsCAD V26: presumed compatible (obsolete behaviour).
- Status: AutoCAD documented as obsolete; BricsCAD presumed-both pending Phase 4 verification.

16.25.10 Source Notes

- [setcfg (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-8BABF3B0-C8E1-49C9-A367-9823211C1...htm>)
- Status: documented (Phase 3 body fill).

16.26 Function Entry: GETCNAME

16.26.1 Name

`'getcname'`

16.26.2 Class

Function

16.26.3 Syntax

`(getcname cname)`

16.26.4 Arguments and Values

- `'cname'`: string; a localized command name, or an underscored English command name. Up to 64 characters.

16.26.5 Description

Converts between localized and English command names. Without an underscore, returns the underscored English command name. With an underscore, returns the localized command name. Useful for writing language-agnostic AutoLISP routines.

16.26.6 Return Values

A string, or `'nil'` if `'cname'` is invalid.

16.26.7 Side Effects

None.

16.26.8 Examples

In French AutoCAD:

- `'(getcname "ETIRER")'` returns `"STRETCH"`.
- `'(getcname "STRETCH")'` returns `"ETIRER"`.

16.26.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (BricsCAD also localises command names).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.26.10 Source Notes

- [getcname (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-1DCDFFA6-0372-479D-B00F-D165EC5B8165.htm>)
- Status: documented (Phase 3 body fill).

16.27 Function Entry: LAYOUTLIST

16.27.1 Name

`'layoutlist'`

16.27.2 Class

Function

16.27.3 Syntax

`(layoutlist)`

16.27.4 Arguments and Values

None.

16.27.5 Description

Returns a list naming all paper-space layouts in the current drawing.

16.27.6 Return Values

A list of strings.

16.27.7 Side Effects

None.

16.27.8 Examples

- `'(layoutlist)'` returns `('"Layout1" "Layout2")'` for a typical drawing.

16.27.9 Availability

- AutoCAD 2026: documented (Windows + macOS).
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.27.10 Source Notes

- [layoutlist (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-AE3F338F-0B11-45B0-AB12-2C119203A111.html>)
- Status: documented (Phase 3 body fill).

16.28 Function Entry: INITCOMMANDVERSION

16.28.1 Name

`'initcommandversion'`

16.28.2 Class

Function

16.28.3 Syntax

`(initcommandversion [version])`

16.28.4 Arguments and Values

- `'version'` (optional): integer specifying the command version to use. When omitted, the next supported command runs at its latest version.

16.28.5 Description

Forces the next AutoCAD command to run with the specified version. Only commands that have been updated to support versioning honour this call. Manual interactive use defaults to the latest version (typically 2); LISP / scripting contexts default to version 1, so `'initcommandversion 2'` is often used to invoke modern dialog-driven behaviour from a script, and `'initcommandversion 1'` is used to force command-prompt-only behaviour.

16.28.6 Return Values

Always returns `'T'`.

16.28.7 Side Effects

Latches the version selector for the next command issued via `'command'` or `'command-s'`.

16.28.8 Examples

- Running the FILLET command after `'(initcommandversion 1)'` shows the legacy prompt set.

- The COLOR command, normally dialog-driven from a script, can be forced to prompt at the command line by ‘(initcommandversion 1)’ followed by the COLOR command.

16.28.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.28.10 Source Notes

- [initcommandversion (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-6176FC98-DC5D-433E-8D76-htm>)
- Status: documented (Phase 3 body fill).

16.29 Function Entry: SETFUNHELP

16.29.1 Name

`'setfunhelp'`

16.29.2 Class

Function

16.29.3 Syntax

```
(setfunhelp c:fname [helpfile [topic [command]]])
```

16.29.4 Arguments and Values

- `'c:fname'`: string; the user-defined command name. Must include the `'c:'` prefix.
- `'helpfile'` (optional): string; help file name. Extension is optional; AutoCAD looks for `'chm'` first, then `'hlp'`.
- `'topic'` (optional): string; help topic identifier. For CHM files, supply the topic file name without extension.
- `'command'` (optional): string; initial Help-window state expressed as an `HtmlHelp()` or `WinHelp()` command (e.g. `'HH_DISPLAYTOPIC'`).

16.29.5 Description

Registers a user-defined command so the Help facility opens the supplied file and topic when the user requests help on that command. Calling `'defun'` to redefine the same command name removes its `'setfunhelp'` registration; therefore call `'setfunhelp'` after `'defun'`.

16.29.6 Return Values

Returns the `'c:fname'` string on success, otherwise `'nil'`. The function only validates the `'c:'` prefix; it does not verify that the function or topic exists.

16.29.7 Side Effects

Updates the Help-binding registry for the named command.

16.29.8 Examples

- After `(defun c:test () (getstring "TEST: ") (princ))`, `(setfunhelp "c:test" "acadacr.chm" "line")` associates the

LINE help topic with `'c:test'`.

16.29.9 Availability

- AutoCAD 2026: documented (Windows-centric — CHM/HLP).
- BricsCAD V26: presumed compatible (BricsCAD has its own help system; binding semantics may differ).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.29.10 Source Notes

- `[setfunhelp (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-61B2326A-8DE4-46C4-B9F9-22B8D163A161.htm>)
- Status: documented (Phase 3 body fill).

16.30 Function Entry: HELP

16.30.1 Name

`'help'`

16.30.2 Class

Function

16.30.3 Syntax

`(help [helpfile [topic [command]])`

16.30.4 Arguments and Values

- `'helpfile'` (optional): string; help-file name. Extension is optional; AutoCAD looks for `'chm'` first, then `'hlp'`. An empty string opens the online help.
- `'topic'` (optional): string; topic identifier. For CHM files, supply the topic file name without extension.
- `'command'` (optional, Windows only): string; initial Help-window state expressed as an `HtmlHelp()` / `WinHelp()` argument.

16.30.5 Description

Invokes the Help facility. Place help files in a folder whose path contains no Unicode characters; the Microsoft HTML Help executable is sensitive to that.

16.30.6 Return Values

Returns the `'helpfile'` string on success, otherwise `'nil'`. With no arguments, returns the empty string `''` on success or `'nil'` on failure.

16.30.7 Side Effects

Opens a help window or browser tab.

16.30.8 Examples

- ‘(help "achelp.chm" "mycommand")’ opens the topic for MYCOMMAND in achelp.chm.

16.30.9 Availability

- AutoCAD 2026: documented (Windows, macOS, Web).
- BricsCAD V26: presumed compatible (BricsCAD has a HELP command).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.30.10 Source Notes

- [help (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4CB4A951-1507-4F3D-9F7B-93FF3A2C9850.htm>)
- Status: documented (Phase 3 body fill).

16.31 Function Entry: MENUCMD

16.31.1 Name

`'menucmd'`

16.31.2 Class

Function

16.31.3 Syntax

`(menucmd str)`

16.31.4 Arguments and Values

- `'str'`: string of the form `"menuarea=value"` selecting a menu area and a value. Supported menu areas:
 - `'B1'–'B4'`: BUTTONS menus.
 - `'A1'–'A4'`: AUX (auxiliary device) menus.
 - `'P0'–'P16'`: pull-down menus.
 - `'I'`: image-tile menus.
 - `'T1'–'T4'`: TABLET menus.
 - `'M'`: DIESEL expressions.
 - `'Gmenugroup.nametag'`: a tagged menu inside a named menu-group.

16.31.5 Description

Issues menu commands. Switches between subpages of a menu, enables / disables menu and ribbon items, places marks, displays image-tile menus, and evaluates DIESEL expressions.

The BricsCAD interpretation of `'menucmd'` is documented as supporting only the `'P0'` cursor menu and `'P1'–'P16'` pulldowns; auxiliary, button, icon, DIESEL, screen, and tablet forms are not supported there. See Phase 4 cross-reference.

16.31.6 Return Values

Always returns `'nil'`.

16.31.7 Side Effects

Updates the menu state, evaluates DIESEL, or displays an image-tile menu.

16.31.8 Examples

- Display an image-tile menu: `‘(menucmd "I=moreicons")‘` then `‘(menucmd "I=*)‘`.
- Check, then disable a pulldown item: `‘(setq s (menucmd "P11.3=?"))‘` and `‘(if (= s "") (menucmd "P11.3=~"))‘`.
- Evaluate DIESEL: `‘(menucmd "M=$(edtime,$(getvar,date),DDDD\","D MONTH YYYY")‘`.

16.31.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: documented with reduced surface (only `‘P0‘/‘P1‘–‘P16‘`).
- Status: cross-vendor divergence — see Phase 4 reconciliation in `‘vendor-inventory-2026.org‘ §10`.

16.31.10 See Also

- BricsCAD documents only the `‘P0‘` cursor menu and `‘P1‘–‘P16‘` pull-down forms. AutoCAD additionally documents `‘B1‘–‘B4‘` (BUTTONS), `‘A1‘–‘A4‘` (AUX), `‘T1‘–‘T4‘` (TABLET), `‘I‘` (image-tile), `‘M‘` (DIESEL), and `‘Gmenugroup.nametag‘` forms — none are accepted by BricsCAD.
- Divergence documented in `‘vendor-inventory-2026.org‘ §10` item 4.

16.31.11 Source Notes

- `[menucmd (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-20357966-71D4-4D55-881B-06AFDECD2htm>)
- Status: documented (Phase 3 body fill).

16.32 Function Entry: MENUGROUP

16.32.1 Name

‘menugroup‘

16.32.2 Class

Function

16.32.3 Syntax

(menugroup groupname)

16.32.4 Arguments and Values

- ‘groupname’: string; the menu group to look up.

16.32.5 Description

Verifies that the named menu group is currently loaded.

16.32.6 Return Values

Returns the ‘groupname’ string when the group is loaded; otherwise ‘nil’.
On macOS / Web, the function always returns ‘groupname’.

16.32.7 Side Effects

None.

16.32.8 Examples

Not documented on the vendor page.

16.32.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.32.10 Source Notes

- [menugroup (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-52FBC08B-F53F-4FC0-A5C5-ED3FA2BB8...htm>)
- Status: documented (Phase 3 body fill).

16.33 Function Entry: SNVALID

16.33.1 Name

‘snvalid‘

16.33.2 Class

Function

16.33.3 Syntax

(snvalid sym_name [flag])

16.33.4 Arguments and Values

- ‘sym_{name}’: string; a candidate symbol-table name.
- ‘flag’ (optional): integer controlling vertical-bar handling.
 - ‘0’ (default): vertical bars are disallowed.
 - ‘1’: vertical bars are allowed except as the first or last character.

16.33.5 Description

Validates a symbol-table name against AutoCAD naming rules. The rules depend on the ‘EXTNAMES’ system variable:

- ‘EXTNAMES = 1’ (default): extended naming. Most characters are accepted, except ‘< > / \ ” : ? * | , = ‘ ;’.
- ‘EXTNAMES = 0’: restrictive naming. Only A–Z, 0–9, ‘\$’, ‘_’, and ‘-’ are accepted.

Control characters, graphic characters, the empty string, and vertical bars at name boundaries are always rejected.

16.33.6 Return Values

Returns ‘T’ when the name is valid; otherwise ‘nil’.

16.33.7 Side Effects

None.

16.33.8 Examples

With 'EXTNAMES = 1':

- '(snvalid "hocus-pocus")' returns 'T'.
- '(snvalid "hocus pocus")' returns 'T'.
- '(snvalid "hocus|pocus")' returns 'nil'.
- '(snvalid "hocus|pocus" 1)' returns 'T'.

With 'EXTNAMES = 0':

- '(snvalid "hocus-pocus")' returns 'T'.
- '(snvalid "hocus pocus")' returns 'nil'.

16.33.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.33.10 Source Notes

- [snvalid (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-2EFBE198-9860-456B-A090-206C743A0111.html>)
- Status: documented (Phase 3 body fill).

16.34 Function Entry: ACDIMENABLEUPDATE

16.34.1 Name

`'acdimenableupdate'`

16.34.2 Class

Function

16.34.3 Syntax

`(acdimenableupdate flag)`

16.34.4 Arguments and Values

- `'flag'`: `'T'` to enable automatic updating of associative dimensions, `'nil'` to suppress updates until DIMREGEN is invoked.

16.34.5 Description

Controls whether associative dimensions are automatically refreshed when their referenced geometry changes. Designed for batch-edit scenarios where deferring dimension recomputation until after several modifications avoids repeated work.

16.34.6 Return Values

Always returns `'nil'`.

16.34.7 Side Effects

Updates the document-level associative-dimension auto-update flag.

16.34.8 Examples

- `'(acdimenableupdate nil)'` disables automatic updates.
- `'(acdimenableupdate T)'` re-enables them.

16.34.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (BricsCAD also supports associative dimensions).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.34.10 Source Notes

- [acdimenableupdate (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-7E10D18A-6FD6-4B20-A981-htm>)
- Status: documented (Phase 3 body fill).

16.35 Function Entry: ALLOC

16.35.1 Name

`'alloc'`

16.35.2 Class

Function

16.35.3 Syntax

`(alloc n-alloc)`

16.35.4 Arguments and Values

- `'n-alloc'`: integer; the new segment-allocation size used by `'expand'`. The number controls how many symbols, strings, usubrs, reals, and cons cells are allocated per future `'expand'` call.

16.35.5 Description

Sets the segment-allocation size used by the `'expand'` function.

16.35.6 Return Values

The previous `'n-alloc'` value (an integer).

16.35.7 Side Effects

Updates the AutoLISP allocator's segment-size parameter.

16.35.8 Examples

- `'(alloc 100)'` returns `'1000'` (the previous segment size, on a fresh session with the documented default).

16.35.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (memory-knob behaviour may differ).

- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.35.10 Source Notes

- [alloc (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-DBC72668-153D-48F4-9C19-87053975A97C.htm>)
- Status: documented (Phase 3 body fill).

16.36 Function Entry: EXPAND

16.36.1 Name

`'expand'`

16.36.2 Class

Function

16.36.3 Syntax

`(expand n-expand)`

16.36.4 Arguments and Values

- `'n-expand'`: integer; the number of additional segments to allocate. Each segment provides space for further symbols, strings, usubrs, reals, and cons cells based on the current `'alloc'` setting.

16.36.5 Description

Allocates additional memory for AutoLISP. The granularity of each segment is controlled by the prior `'alloc'` call.

16.36.6 Return Values

An integer: the number of free conses divided by `'n-alloc'`.

16.36.7 Side Effects

Grows the AutoLISP heap by `'n-expand'` segments.

16.36.8 Examples

After `'(alloc 100)'` returns `'1000'`:

- `'(expand 2)'` returns `'82'`, providing 200+ additional symbols/strings/usubrs/reals plus 8200 available cons cells.

16.36.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (memory-knob behaviour may differ).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.36.10 Source Notes

- [expand (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-48C99C97-176D-4D9C-91E2-31B3C67D6111.html>)
- Status: documented (Phase 3 body fill).

16.37 Function Entry: GC

16.37.1 Name

`'gc'`

16.37.2 Class

Function

16.37.3 Syntax

`(gc)`

16.37.4 Arguments and Values

None.

16.37.5 Description

Forces a garbage collection, freeing memory occupied by inaccessible objects.

16.37.6 Return Values

Always returns `'nil'`.

16.37.7 Side Effects

Reclaims AutoLISP heap memory; may pause the runtime briefly.

16.37.8 Examples

Not documented on the vendor page.

16.37.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.37.10 Source Notes

- [gc (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-6CABB3AF-9BED-4237-8A44-D36123246BF4.htm>)
- Status: documented (Phase 3 body fill).

16.38 Function Entry: MEM

16.38.1 Name

`'mem'`

16.38.2 Class

Function

16.38.3 Syntax

`(mem)`

16.38.4 Arguments and Values

None.

16.38.5 Description

Displays AutoLISP memory-usage statistics: garbage-collection counts and timings, dynamic-segment information (page size, used / free pages, largest contiguous free area, segment count), and per-type allocation (LISP stacks, bytecode area, CONS memory, strings, etc.). All heap memory is global; every AutoCAD document shares the same heap.

16.38.6 Return Values

Always returns `'nil'`. The statistics are written to the command line as a side effect.

16.38.7 Side Effects

Writes a multi-line memory report to the AutoCAD command line.

16.38.8 Examples

- A typical `'(mem)'` invocation prints lines such as:

```
; GC calls: 23; GC run time: 298 ms
Dynamic memory segments statistic:
PgSz  Used  Free  FMCL  Segs  Type
512    79    48    48    1  lisp stacks
```

256	3706	423	142	16	bytecode area
4096	320	10	10	22	CONS memory
...					

16.38.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible (the report content is implementation-defined).
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.38.10 Source Notes

- [mem (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-F4AEB953-2117-4BF2-8056-EA1384AC3FFF.htm>)
- Status: documented (Phase 3 body fill).

16.39 Function Entry: TABLET

16.39.1 Name

`'tablet'`

16.39.2 Class

Function

16.39.3 Syntax

`(tablet code [row1 row2 row3 direction])`

16.39.4 Arguments and Values

- `'code'`: integer.
 - `'0'`: retrieve the current digitiser calibration; the trailing arguments are omitted.
 - `'1'`: set the calibration; the trailing arguments are required.
- `'row1'`, `'row2'`, `'row3'`: lists of three reals forming the rows of a 3_3 transformation matrix. The Z component of `'row3'` is normalised to 1.
- `'direction'`: list of three reals; the normal vector to the tablet surface in WCS. Automatically normalised.

16.39.5 Description

Retrieves or sets digitiser (tablet) calibrations. Calibration data lets AutoCAD transform raw digitiser coordinates into drawing-space coordinates.

16.39.6 Return Values

An integer on success. On failure, returns `'nil'` and sets the AutoCAD `'ER-RNO'` system variable; typical failure cause is that the input device is not a tablet.

16.39.7 Side Effects

Updates the AutoCAD digitiser calibration state.

16.39.8 Examples

- Identity calibration: ‘(tablet 1 ’(1 0 0) ’(0 1 0) ’(0 0 1) ’(0 0 1))’.

16.39.9 Availability

- AutoCAD 2026: documented (Windows only).
- BricsCAD V26: presumed compatible (Windows-only digitiser feature).
- Status: AutoCAD-Windows documented; BricsCAD parity TBD.

16.39.10 Source Notes

- [tablet (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-5AD61389-1F8E-497B-8B0B-259488A50...htm>)
- Status: documented (Phase 3 body fill).

16.40 Function Entry: VPORTS

16.40.1 Name

`'vports'`

16.40.2 Class

Function

16.40.3 Syntax

`(vports)`

16.40.4 Arguments and Values

None.

16.40.5 Description

Returns a list of viewport descriptors for the current viewport configuration. The shape of the result depends on the `'TILEMODE'` system variable: tiled viewports (`VPORTS` command) when `'TILEMODE'` is 1, floating viewports (`MVIEW` command) when `'TILEMODE'` is 0. When `'TILEMODE'` is off, viewport number 1 is always paper space.

16.40.6 Return Values

A list of viewport descriptors. Each descriptor is `'(viewport-id (lower-left) (upper-right))'`. With `'TILEMODE'` on, the corners are values in `[0.0, 1.0]` relative to the display screen. With `'TILEMODE'` off, the corners use paper-space coordinates. The current viewport's descriptor is always first in the list.

16.40.7 Side Effects

None.

16.40.8 Examples

- A single viewport with `'TILEMODE'` on: `'((1 (0.0 0.0) (1.0 1.0)))'`.
- Four equal tiled viewports:

```
((5 (0.5 0.0) (1.0 0.5))
  (2 (0.5 0.5) (1.0 1.0))
  (3 (0.0 0.5) (0.5 1.0))
  (4 (0.0 0.0) (0.5 0.5)))
```

16.40.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.40.10 Source Notes

- [vports (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-929FC8B2-1235-4B2A-B330-FF2E219E017A.html>)
- Status: documented (Phase 3 body fill).

16.41 Function Entry: SETVIEW

16.41.1 Name

`'setview'`

16.41.2 Class

Function

16.41.3 Syntax

`(setview view_descriptor [vport_id])`

16.41.4 Arguments and Values

- `'view_descriptor'`: an entity-definition list similar to that returned by `'tblsearch'` for the `'VIEW'` symbol table.
- `'vport_id'` (optional): integer; the destination viewport. `'0'` selects the current viewport. The viewport identifier is held by the `'CVPORT'` system variable.

16.41.5 Description

Establishes a view for a specified viewport.

16.41.6 Return Values

The `'view_descriptor'` on success, or `'nil'` on failure.

16.41.7 Side Effects

Updates the named viewport's view to the supplied descriptor.

16.41.8 Examples

Not documented on the vendor page.

16.41.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.41.10 Source Notes

- [setview (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-EEABEF66-423A-4D2B-BE16-AA6A56B08...htm>)
- Status: documented (Phase 3 body fill).

16.42 Function Entry: ARX

16.42.1 Name

`'arx'`

16.42.2 Class

Function

16.42.3 Syntax

`(arx)`

16.42.4 Arguments and Values

None.

16.42.5 Description

Returns a list naming the ObjectARX applications currently loaded into the AutoCAD process.

16.42.6 Return Values

A list of strings. File extensions vary by platform: `'arx'` on Windows / Web; `'bundle'` or `'crx'` on macOS.

16.42.7 Side Effects

None.

16.42.8 Examples

- Windows / Web: `('\"acapp.arx\" \"acmted.arx\" ...)`.
- macOS: `('\"acapp.bundle\" \"acmted.crx\" ...)`.

16.42.9 Availability

- AutoCAD 2026: documented (Windows, macOS, Web).
- BricsCAD V26: not applicable. BricsCAD uses the BRX runtime; querying the BRX module list is via a different interface.

- Status: AutoCAD documented; BricsCAD does not expose this name.

16.42.10 Source Notes

- [arx (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-D74F3A2B-F506-415F-8F5A-258764C2E26D.htm>)
- Status: documented (Phase 3 body fill).

16.43 Function Entry: CVUNIT

16.43.1 Name

`'cvunit'`

16.43.2 Class

Function

16.43.3 Syntax

`(cvunit value from-unit to-unit)`

16.43.4 Arguments and Values

- `'value'`: integer, real, or list (a 2D / 3D point).
- `'from-unit'`: string naming the source unit.
- `'to-unit'`: string naming the target unit.

16.43.5 Description

Converts `'value'` from one unit of measurement to another. Unit names must exist in `'acad.unt'` (or `'acadlt.unt'` for AutoCAD LT). Returns `'nil'` when a unit name is unknown or when the source and target units are incompatible (e.g. grams to years). For repeated conversions of the same unit pair, convert `'1.0'` once and use the result as a scale factor — except for temperature units, which involve an offset.

16.43.6 Return Values

The converted value as a real (or list of reals if `'value'` is a list); `'nil'` on failure.

16.43.7 Side Effects

None.

16.43.8 Examples

- ‘(cvunit 1 "minute" "second")’ returns ‘60.0’.
- ‘(cvunit 1.0 "inch" "cm")’ returns ‘2.54’.
- ‘(cvunit 1.0 "acre" "sq yard")’ returns ‘4840.0’.
- ‘(cvunit '(1.0 2.5) "ft" "in")’ returns ‘(12.0 30.0)’.

16.43.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.43.10 Source Notes

- [cvunit (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-253E72F1-09B8-473E-A51B-B97BE2E29111/htm>)
- Status: documented (Phase 3 body fill).

16.44 Function Entry: REDRAW

16.44.1 Name

`'redraw'`

16.44.2 Class

Function

16.44.3 Syntax

`(redraw [ename [mode]])`

16.44.4 Arguments and Values

- `'ename'` (optional): an entity name; the entity to redraw.
- `'mode'` (optional): an integer:
 - 1: show entity
 - 2: hide entity
 - 3: highlight entity
 - 4: unhighlight entity

Negative values affect only the header entity of a complex object (polyline, attributed block); positive values affect all subentities.

16.44.5 Description

Redraws the current viewport, or a specified entity in the current viewport. With no arguments, refreshes the entire viewport.

16.44.6 Return Values

Always returns `'nil'`.

16.44.7 Side Effects

Refreshes the screen image of the affected viewport region.

16.44.8 Examples

Not documented on the vendor page.

16.44.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.44.10 Source Notes

- [redraw (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-4A4DCECD-E85A-4860-A58F-56B482278111.html>)
- Status: documented (Phase 3 body fill).

16.45 Function Entry: GRAPHSCR

16.45.1 Name

‘graphscr‘

16.45.2 Class

Function

16.45.3 Syntax

(graphscr)

16.45.4 Arguments and Values

None.

16.45.5 Description

Displays the AutoCAD graphics screen. Equivalent to the AutoCAD GRAPHSCR command. On macOS and Web platforms, the function is recognised but produces no visible effect; the complementary function ‘textscr‘ switches focus to the text screen on Windows.

16.45.6 Return Values

Always returns ‘nil‘.

16.45.7 Side Effects

Focuses the graphics window on Windows.

16.45.8 Examples

Not documented on the vendor page.

16.45.9 Availability

- AutoCAD 2026: documented (Windows; recognised but no-op on macOS / Web).
- BricsCAD V26: presumed compatible.

- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.45.10 Source Notes

- [graphscr (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-FB9ABA5D-6DDA-4085-9E9F-18038EC31.htm>)
- Status: documented (Phase 3 body fill).

16.46 Function Entry: TEXTPAGE

16.46.1 Name

‘textpage‘

16.46.2 Class

Function

16.46.3 Syntax

(textpage)

16.46.4 Arguments and Values

None.

16.46.5 Description

Switches focus from the drawing area to the text screen. Equivalent to ‘textscr‘. On macOS and Web platforms, the function is recognised but produces no visible effect.

16.46.6 Return Values

Always returns ‘nil‘.

16.46.7 Side Effects

Focuses the text-screen window on Windows.

16.46.8 Examples

Not documented on the vendor page.

16.46.9 Availability

- AutoCAD 2026: documented (Windows; recognised but no-op on macOS / Web).
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.46.10 Source Notes

- [textpage (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-C5F1E0AF-CAE8-4338-8C27-5A4235D37.htm>)
- Status: documented (Phase 3 body fill).

16.47 Function Entry: TEXTSCR

16.47.1 Name

`'textscr'`

16.47.2 Class

Function

16.47.3 Syntax

`(textscr)`

16.47.4 Arguments and Values

None.

16.47.5 Description

Switches focus from the drawing area to the text screen. Equivalent to `'textpage'`. On macOS and Web platforms, the function is recognised but produces no visible effect.

16.47.6 Return Values

Always returns `'nil'`.

16.47.7 Side Effects

Focuses the text-screen window on Windows.

16.47.8 Examples

Not documented on the vendor page.

16.47.9 Availability

- AutoCAD 2026: documented (Windows; recognised but no-op on macOS / Web).
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.47.10 Source Notes

- [textscr (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-5F0A5635-0DCB-4A88-BD8E-60EC665341.htm>)
- Status: documented (Phase 3 body fill).

16.48 Function Entry: GRCLEAR

16.48.1 Name

`'grclear'`

16.48.2 Class

Function (legacy / obsolete)

16.48.3 Syntax

`(grclear)`

16.48.4 Arguments and Values

None.

16.48.5 Description

Clears the current viewport. Documented as an obsolete legacy display function; modern code should use `'redraw'` instead.

16.48.6 Return Values

Returns `'nil'`.

16.48.7 Side Effects

Clears the viewport on legacy display devices.

16.48.8 Examples

Not documented on the vendor page.

16.48.9 Availability

- AutoCAD 2026: documented as legacy / obsolete.
- BricsCAD V26: presumed compatible (legacy function, behaviour may be a no-op).
- Status: AutoCAD documented as obsolete; BricsCAD presumed-both pending Phase 4 verification.

16.48.10 Source Notes

- [G Functions Reference (AutoLISP 2026) — ‘grclear‘ summary](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-88320FBD-B49E-445A-A5C1-C2ABA98341BC.htm>)
- Status: legacy; per-symbol page not separately published.

16.49 Function Entry: GRDRAW

16.49.1 Name

`'grdraw'`

16.49.2 Class

Function

16.49.3 Syntax

`(grdraw from to color [highlight])`

16.49.4 Arguments and Values

- `'from'`: list of two or three reals; one endpoint of the vector in the current UCS.
- `'to'`: list of two or three reals; the other endpoint.
- `'color'`: integer; AutoCAD color number 0–255. The value `'-1'` selects XOR ink, which complements anything it draws over and erases itself when overdrawn.
- `'highlight'` (optional): integer; if non-zero, the vector is drawn with the device's default highlighting style (typically dashed).

16.49.5 Description

Draws a transient vector between two points in the current viewport. The drawing is automatically clipped to the screen.

16.49.6 Return Values

Always returns `'nil'`.

16.49.7 Side Effects

Draws transient (non-persistent) graphics in the viewport.

16.49.8 Examples

Not documented on the vendor page.

16.49.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.49.10 Source Notes

- `[grdraw (AutoLISP 2026)]`(<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-C00544A7-AF63-403E-98EB-205248EB4...htm>)
- Status: documented (Phase 3 body fill).

16.50 Function Entry: GRTEXT

16.50.1 Name

`'grtext'`

16.50.2 Class

Function

16.50.3 Syntax

`(grtext [box text [highlight]])`

16.50.4 Arguments and Values

- `'box'` (optional): integer; `'0'` or greater selects a screen-menu box, `'-1'` selects the mode status line, `'-2'` selects the coordinate status line.
- `'text'` (optional): string to display. Truncated by the device width.
- `'highlight'` (optional): integer; for screen-menu boxes (`'box' >= 0`) a positive value highlights and zero unhighlights. Ignored for status-line boxes; coordinate tracking can overwrite the `'-2'` area.

16.50.5 Description

Displays the supplied text in a screen-menu area or the status line; does not change the underlying menu item. With no arguments, restores all text areas to default values. Recognised but no-op on macOS and Web platforms.

16.50.6 Return Values

Returns the `'text'` argument when successful, otherwise `'nil'`.

16.50.7 Side Effects

Updates the indicated screen-menu / status-line area.

16.50.8 Examples

Not documented on the vendor page.

16.50.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.50.10 Source Notes

- [grtext (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-5B0B01CA-BDBA-4A77-B840-6EE64591A111.html>)
- Status: documented (Phase 3 body fill).

16.51 Function Entry: GRVECS

16.51.1 Name

`'grvecs'`

16.51.2 Class

Function

16.51.3 Syntax

`(grvecs vlist [trans])`

16.51.4 Arguments and Values

- `'vlist'`: a vector list of optional color integers paired with two point lists, in the form `'([color1] from1 to1 [color2] from2 to2 ...)'`. Color values 0–255 are AutoCAD colors; values above 255 select XOR ink; negative values select the device's highlight style.
- `'trans'` (optional): a 4_4 transformation matrix (a list of four 4-real lists) that repositions or scales every vector in `'vlist'`.

16.51.5 Description

Draws multiple transient vectors in the drawing area. A specified color persists for all subsequent vectors until another color value is given.

16.51.6 Return Values

Always returns `'nil'`.

16.51.7 Side Effects

Draws transient graphics; nothing is committed to the drawing database.

16.51.8 Examples

- Five vertical lines with five different colors:

```
(grvecs '(1 (1 2)(1 5)
            2 (2 2)(2 5)
```

```

3 (3 2)(3 5)
4 (4 2)(4 5)
5 (5 2)(5 5)))

```

- Identity transformation with translation ‘(5,5,0)’:

```

'((1.0 0.0 0.0 5.0)
  (0.0 1.0 0.0 5.0)
  (0.0 0.0 1.0 0.0)
  (0.0 0.0 0.0 1.0))

```

16.51.9 Availability

- AutoCAD 2026: documented.
- BricsCAD V26: presumed compatible.
- Status: AutoCAD documented; BricsCAD presumed-both pending Phase 4 verification.

16.51.10 Source Notes

- [grvecs (AutoLISP 2026)](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-AutoLISP-Reference/files/GUID-44B8C277-4140-4F4C-ACF2-3332CC463111.html>)
- Status: documented (Phase 3 body fill).

16.52 clautolisp Extensions

16.52.1 Overview

This sub-tree documents host-interaction helpers that are **not** present in either AutoCAD or BricsCAD. They are clautolisp-specific additions that improve interactive AutoLISP development without colliding with the vendor namespace. To make the distinction obvious at every call site, the entries use a reserved **CLAL-** prefix; no Autodesk- or Bricsys-documented identifier has ever used that prefix.

A clautolisp-only program that calls a **CLAL-** function will not run unchanged on AutoCAD or BricsCAD: `(boundp 'clal-sysvar-list)` returns `nil` there. Portable code that wants to ride the extension when available should probe with `boundp` before calling:

```
(if (boundp 'clal-sysvar-list)
    (clal-sysvar-list)
    ;; vendor fallback - there isn't one; either accept that the
    ;; query is unavailable or shell out to the SETVAR command.
    nil)
```

16.52.2 Source Notes

- These functions are defined by clautolisp; there is no Autodesk or Bricsys reference page.
- The source of truth is the entries below, the spec-level inventory at `autolisp-spec/documentation/system-variables- inventory.sexp`, and the host-side implementation under `clautolisp/autolisp-builtins-core/source/api.li`.

16.53 Function Entry: CLAL-SYSVAR-LIST

16.53.1 Name

`clal-sysvar-list`

16.53.2 Class

`clautolisp` Extension Function

16.53.3 Syntax

`(clal-sysvar-list)`

16.53.4 Arguments and Values

No arguments.

16.53.5 Description

Return the list of sysvar-name strings known to the current host, sorted lexicographically (case-sensitive ASCII order). The names are upper-cased and ready to pass to `getvar` / `setvar`.

On a default `MockHost` the list has 1836 entries – the same universe as the §16 `System Variables` catalogue. A live AutoCAD or BricsCAD backend would return whatever its engine reports.

16.53.6 Return Values

A proper list of AutoLISP strings. Empty list `nil` if the host defines no sysvars.

16.53.7 Side Effects

None.

16.53.8 Affected By

- The selected host backend (`NullHost` / `MockHost` / `LiveHost`).
- The mock-host catalogue (see §16 `System Variables`).

16.53.9 Exceptional Situations

Raises `:host-not-supported` when the active backend has not specialised `host-sysvar-names` (e.g. the bare `NullHost`).

16.53.10 Examples

```
;; Count the sysvars the host knows.  
(length (clal-sysvar-list))  
=> 1836
```

```
;; First few entries.  
(mapcar 'identity (cdddr (cdddr (cdddr (clal-sysvar-list)))))  
=> ("ACADPREFIX" "ACADVER" ... )
```

16.53.11 Availability

- clautolisp: documented (this entry); implemented since clautolisp 1.0.76.
- AutoCAD: NOT present.
- BricsCAD: NOT present.

16.53.12 Source Notes

- clautolisp-only extension; see this chapter's Overview for the rationale and probing convention.

16.54 Function Entry: CLAL-SYSVAR-APROPOS

16.54.1 Name

`clal-sysvar-afropos`

16.54.2 Class

clautolisp Extension Function

16.54.3 Syntax

`(clal-sysvar-afropos pattern)`

16.54.4 Arguments and Values

- `pattern` – string. Compared case-insensitively against every sysvar name in `(clal-sysvar-list)`. The empty string matches every entry; a string that matches no entry returns `nil`.

16.54.5 Description

Return the subset of `(clal-sysvar-list)` whose names contain `pattern` as a (case-insensitive) substring. Useful at the REPL for discovery: `(clal-sysvar-afropos "ANG")` surfaces ANGBASE, ANGDIR, and the snap/orient angle family.

16.54.6 Return Values

A proper list of AutoLISP strings – a (possibly empty) subset of `(clal-sysvar-list)`, in the same lexicographic order.

16.54.7 Side Effects

None.

16.54.8 Affected By

- Same as `clal-sysvar-list`.

16.54.9 Exceptional Situations

- `:invalid-argument` when `pattern` is not a string.
- `:host-not-supported` when the host backend has not specialised `host-sysvar-names` (e.g. `NullHost`).

16.54.10 Examples

```
(clal-sysvar-apropos "ANG")  
=> ("ANGBASE" "ANGDIR")
```

```
(clal-sysvar-apropos "ang") ; case-insensitive  
=> ("ANGBASE" "ANGDIR")
```

```
(clal-sysvar-apropos "ERRNO")  
=> ("ERRNO")
```

```
(clal-sysvar-apropos "DEFINITELY-NOT-A-SUBSTRING")  
=> nil
```

```
(length (clal-sysvar-apropos "")) ; empty pattern -> all  
=> 1836
```

16.54.11 Availability

- clautolisp: documented (this entry); implemented since clautolisp 1.0.76.
- AutoCAD: NOT present.
- BricsCAD: NOT present.

16.54.12 Source Notes

- clautolisp-only extension; see this chapter's Overview.

16.55 Function Entry: CLAL-SYSTEM-CODEPAGE

16.55.1 Name

clal-system-codepage

16.55.2 Class

clautolisp Extension Function

16.55.3 Syntax

(clal-system-codepage)

16.55.4 Arguments and Values

No arguments.

16.55.5 Description

Return the canonical clautolisp codepage spelling for the host's `SYSCODEPAGE` system variable. Wraps (`getvar "SYSCODEPAGE"`) and canonicalises the raw value:

Raw sysvar value	Canonical return string
"ANSI_NNNN"	"CP-NNNN"
"WINDOWS-NNNN"	"CP-NNNN"
"CPNNNN"	"CP-NNNN"
"CP-NNNN"	"CP-NNNN" (unchanged)
"UTF-8"	"UTF-8"
"ISO-8859-1"	"ISO-8859-1"
"US-ASCII"	"US-ASCII"
"" (empty)	"ANSI"
anything else	unchanged

clautolisp's launch wiring populates `SYSCODEPAGE` from `LC_ALL / LANG / LC_CTYPE / -e` at startup (see [issues/closed/encoding-dispatch.issue](#)), so a Western-European `en_US.UTF-8` session reports "UTF-8"; a French `fr_FR.WINDOWS-1252` session reports "CP-1252". On a live BricsCAD bridge, `SYSCODEPAGE` arrives in vendor format ("ANSI_1252") and folds onto the same canonical answer clautolisp would produce in-process.

16.55.6 Return Values

A non-empty AutoLISP string.

16.55.7 Side Effects

None.

16.55.8 Affected By

- (getvar "SYSCODEPAGE") on the active host.
- The launch-time locale chain (clautolisp only).

16.55.9 Exceptional Situations

:host-not-supported when the active host backend does not implement host-getvar.

16.55.10 Examples

```
;; UTF-8 locale.  
(clal-system-codepage) => "UTF-8"  
  
;; fr_FR.WINDOWS-1252 locale (clautolisp folds to "CP-1252").  
(clal-system-codepage) => "CP-1252"  
  
;; Vendor-side bridge (alfe + BricsCAD).  
(clal-system-codepage) => "CP-1252"
```

16.55.11 Availability

- clautolisp: documented (this entry); implemented since clautolisp 1.0.80.
- AutoCAD: NOT present (SYSCODEPAGE is, but the canonicalising wrapper is clautolisp-only).
- BricsCAD: NOT present (same).

16.55.12 Source Notes

- clautolisp-only extension. See encoding-dispatch.issue for the canonicalisation table and the full launch-resolution chain.

16.56 Function Entry: CLAL-DRAWING-CODEPAGE

16.56.1 Name

`clal-drawing-codepage`

16.56.2 Class

clautolisp Extension Function

16.56.3 Syntax

`(clal-drawing-codepage)`

16.56.4 Arguments and Values

No arguments.

16.56.5 Description

Return the canonical clautolisp codepage spelling for the host's `DWGCODEPAGE` system variable, using the same canonicalisation table as `clal-system-codepage`. `DWGCODEPAGE` records the codepage the current drawing was authored under; without a loaded drawing it defaults to `SYSCODEPAGE`.

The function exists separately from `clal-system-codepage` so user code can compare the two (typically through `clal-codepage-mismatch-p`) and warn before consuming drawing- authored text whose interpretation depends on the codepage.

16.56.6 Return Values

A non-empty AutoLISP string.

16.56.7 Side Effects

None.

16.56.8 Affected By

- `(getvar "DWGCODEPAGE")` on the active host.
- The most-recently-loaded drawing's encoding header.

16.56.9 Exceptional Situations

`:host-not-supported` when the active host backend does not implement `host-getvar`.

16.56.10 Examples

```
;; No drawing loaded; defaults to SYSCODEPAGE.  
(clal-drawing-codepage) => "UTF-8"
```

```
;; After loading a drawing authored under a Czech Windows machine.  
(clal-drawing-codepage) => "CP-1250"
```

16.56.11 Availability

- clautolisp: documented (this entry); implemented since clautolisp 1.0.80.
- AutoCAD: NOT present.
- BricsCAD: NOT present.

16.56.12 Source Notes

- clautolisp-only extension.

16.57 Function Entry: CLAL-CODEPAGE-MISMATCH-P

16.57.1 Name

`clal-codepage-mismatch-p`

16.57.2 Class

clautolisp Extension Predicate

16.57.3 Syntax

`(clal-codepage-mismatch-p)`

16.57.4 Arguments and Values

No arguments.

16.57.5 Description

Return T when the canonical DWGCODEPAGE differs from the canonical SYSCODEPAGE, nil otherwise. Cheap defensive check before consuming drawing-authored text: a mismatch means strings authored under one ANSI codepage may garble when interpreted under another (the classic "Czech-authored drawing read on a French machine" failure mode).

The comparison runs on the canonicalised form, so "ANSI_1252" vs "WINDOWS-1252" correctly registers as no mismatch.

16.57.6 Return Values

T or nil.

16.57.7 Side Effects

None.

16.57.8 Affected By

- Same as `clal-system-codepage` and `clal-drawing-codepage`.

16.57.9 Exceptional Situations

`:host-not-supported` when `host-getvar` is unimplemented.

16.57.10 Examples

```
;; Same codepage on both sides.  
(clal-codepage-mismatch-p) => nil
```

```
;; CP-1250 drawing on a CP-1252 host.  
(clal-codepage-mismatch-p) => T
```

16.57.11 Availability

- clautolisp: documented (this entry); implemented since clautolisp 1.0.80.
- AutoCAD: NOT present.
- BricsCAD: NOT present.

16.57.12 Source Notes

- clautolisp-only extension.

16.58 Function Entry: CLAL-FILE-ENCODING

16.58.1 Name

clal-file-encoding

16.58.2 Class

clautolisp Extension Function

16.58.3 Syntax

(clal-file-encoding path)

16.58.4 Arguments and Values

- **path** – AutoLISP string. Filename to sniff; resolved against the AutoLISP search path the same way **LOAD** would. Need not be a **.lsp** source file; any file is accepted.

16.58.5 Description

Open **path**, read up to the first four bytes, and return the canonical clautolisp encoding implied by the byte-order mark:

BOM bytes	Returned string
EF BB BF	"UTF-8-BOM"
FF FE	"UTF-16-LE"
FE FF	"UTF-16-BE"
FF FE 00 00	"UTF-32-LE"
00 00 FE FF	"UTF-32-BE"
no BOM	"ANSI"

UTF-32-LE and UTF-16-LE share their first two bytes; the sniffer checks UTF-32 first so the two-byte UTF-16 prefix doesn't mask a four-byte UTF-32 BOM.

clal-file-encoding is the clautolisp counterpart to BricsCAD's **vle-file-encoding**. Differences:

- Returns canonical clautolisp strings (e.g. "UTF-16-LE") rather than vendor strings ("UTF-16LE").

- Preserves the UTF-8-BOM / plain-UTF-8 distinction. BricsCAD reports plain "UTF-8" in both cases; clautolisp reports "UTF-8-BOM" when the marker is present, letting user code decide between rewriting the BOM and dropping it.
- Returns "ANSI" rather than the bare empty string when no BOM is detected — the file might be ANSI, UTF-8 without BOM, or something else; without a marker the encoding is undetermined and falls back to the host code page.

16.58.6 Return Values

- AutoLISP string when the file exists and is readable.
- `nil` when the file cannot be resolved; also sets `ERRNO` to 73 (same convention as `LOAD`'s missing-file path).

16.58.7 Side Effects

- Opens `path` for reading and immediately closes it.
- Sets `ERRNO` to 73 on resolution failure, 0 on success.

16.58.8 Affected By

- The AutoLISP search-path used by `LOAD`.

16.58.9 Exceptional Situations

- Returns `nil` rather than signalling for the common "file not found" case; user code can chain `clal-file-encoding` into a presence check.

16.58.10 Examples

```
;; UTF-8 source with a BOM.
(clal-file-encoding "lib/with-bom.lsp")
=> "UTF-8-BOM"
```

```
;; BOMless source (could be anything from US-ASCII to ANSI to
;; UTF-8-without-BOM; we don't speculate).
(clal-file-encoding "lib/plain.lsp")
=> "ANSI"
```

```
;; Nonexistent path -> nil + ERRNO 73.  
(clal-file-encoding "/no/such/file") => nil  
(getvar "ERRNO")                     => 73
```

16.58.11 Availability

- clautolisp: documented (this entry); implemented since clautolisp 1.0.83.
- AutoCAD: NOT present.
- BricsCAD: `vle-file-encoding` is the analogue but does not preserve the UTF-8-BOM distinction and uses vendor strings.

16.58.12 Source Notes

- clautolisp-only extension. See `encoding-dispatch.issue` for the cross-dialect translation table this helper's vocabulary matches.

16.59 System Variables

16.59.1 Overview

A **system variable** is a named, typed cell living in the host engine, addressed by string name from AutoLISP via `getvar` and `setvar`. System variables are a namespace distinct from AutoLISP symbols; they are not affected by `set`, `setq`, or lexical bindings, and they survive across LISP file loads for the lifetime of the host session (subject to their **saved in scope**).

This chapter enumerates every system variable that either AutoCAD 2026 ENU or BricsCAD V25 documents in its reference, with the fields harvested by the Phase-1.5 inventory pass (see `autolisp-spec/documentation/system-variables-inventor`).

16.59.2 Normative Rules

The following rules apply uniformly to every entry in this chapter unless that entry explicitly overrides them.

1. Name case-insensitivity System-variable names are case-insensitive on both AutoCAD and BricsCAD. The canonical form in this specification is uppercase (e.g. `EXTNAMES`, `CMDECHO`); (`getvar "extnames"`) is equivalent to (`getvar "EXTNAMES"`) on both hosts.
2. Unknown names `getvar` returns `nil` when `varname` does not name a defined system variable. `setvar` signals an AutoLISP error in the same case. Applications that probe for the presence of a sysvar must therefore use `getvar` first, not `setvar`.
3. Integer value range Where an entry's **Type** is `integer` or `short integer`, the host clamps the supplied value to the signed-16-bit range `[~-32768~, +32767~]` regardless of the wider Common-Lisp integer type of the supplied value. `~setvar` on an out-of-range value signals an AutoLISP error.
4. Type coercion on `setvar` `setvar` accepts a value whose AutoLISP type matches the entry's declared **Type**. The host applies no implicit numeric coercion: (`setvar "CMDECHO" 1.0`) signals an error even though `CMDECHO` is an integer; the caller must supply the integer 1 explicitly.
5. Read-only semantics An entry marked **Read-Only: yes** may be read with `getvar` but attempting `setvar` on it signals an AutoLISP error. Read-only status is set by the host and cannot be cleared from AutoLISP.

6. Angular sysvars: radians vs display units For **ANGBASE** and **SNAPANG** specifically, **setvar** interprets the supplied value as **radians** regardless of **AUNITS**. This differs from the interactive **SETVAR** command, which interprets values in the current angular display units. **getvar** returns radians in both cases. (Autodesk reference: **setvar** (AutoLISP).)
7. Saved-in scope The **Scope** field records the host-side persistence of the value: **drawing** values persist with the current drawing file, **registry** values persist across drawings under the user account, **user preference** values persist per-user profile, and **session** values are recomputed every time the host starts.
8. Defaults marked as non-literal A **Default** written as **varies by drawing** / **varies by host** / **varies by session** / **varies by registry** / **varies by user preference** indicates that the vendor reference does not specify a constant initial value: the value is derived from the drawing template, the operating-system platform, the registry / user profile, or the live session respectively. Conformance tests must not assume a specific value in these cases.
9. Vendor divergence Where the entry's **Divergence** row is filled, AutoCAD and BricsCAD ship **different** defaults, types, ranges, or saved-in scopes for the same name. Portable programs must read divergent sysvars defensively and either probe the type before assuming a value shape or accept both vendors' conventions.

16.59.3 Source Notes

- [AutoCAD 2026 Commands and System Variables Reference](<https://help.autodesk.com/cloudhelp/2026/ENU/AutoCAD-Core/files/GUID-4FEBA606-95E0-4DC4-A116-257ED86DCD58.htm>)
- [BricsCAD V25 System Variable Reference](https://help.bricsys.com/document/_systemvariablereference--SV_system_variable_reference_intro/V25/EN_US)
- `autolisp-spec/documentation/system-variables-inventory.sexp` (machine-readable inventory; one plist per sysvar)
- `issues/closed/deferred-system-variables-inventory-completion.issue` (Phase-1.5 build-pipeline notes)

16.60 System Variable Entry: 3DCONVERSIONMODE

16.60.1 Name

3DCONVERSIONMODE

16.60.2 Type

integer

16.60.3 Default

1

16.60.4 Read-Only

no

16.60.5 Scope

registry; saved in registry

16.60.6 Description

Used to convert material and light definitions to the current product release.

16.60.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.60.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-D5143EED-F2B3-4652-BC04-53B43BAB381D>

16.61 System Variable Entry: 3DDWFPPREC

16.61.1 Name

3DDWFPPREC

16.61.2 Type

integer

16.61.3 Default

2

16.61.4 Read-Only

no

16.61.5 Scope

drawing; saved in drawing

16.61.6 Description

Controls the precision of 3D DWF or 3D DWFx publishing.

16.61.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.61.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-0D9D7C23-8D2B-4856-BDDD-546812B5AB19>

16.62 System Variable Entry: 3DOSMODE

16.62.1 Name

3DOSMODE

16.62.2 Type

integer

16.62.3 Default

139

16.62.4 Read-Only

no

16.62.5 Scope

registry; saved in registry

16.62.6 Description

Sets running 3D object snaps.

16.62.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.62.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-10DD6098-8B87-45B4-9BB9-325D58695764>

16.63 System Variable Entry: 3DSELECTIONMODE

16.63.1 Name

3DSELECTIONMODE

16.63.2 Type

integer

16.63.3 Default

1

16.63.4 Read-Only

no

16.63.5 Scope

registry; saved in registry

16.63.6 Description

Controls the selection precedence of both visually and physically overlapping objects when using 3D visual styles.

16.63.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.63.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-0FA5D420-2A1C-406B-A267-6619E0B6C760>

16.64 System Variable Entry: ACADLSPASDOC

16.64.1 Name

ACADLSPASDOC

16.64.2 Type

integer

16.64.3 Default

0

16.64.4 Read-Only

no

16.64.5 Scope

registry; saved in registry

16.64.6 Description

Controls whether the acad.lsp file, or acadlt.lsp file for AutoCAD LT, file is loaded into every drawing or just the first drawing opened in a session.

16.64.7 Read/Written By

- load

16.64.8 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.64.9 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-E0049DB8-DC45-47A1-A8B7-67012F7AEDB8>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/acadlspasdoc-system-variable>

16.65 System Variable Entry: ACADPREFIX

16.65.1 Name

ACADPREFIX

16.65.2 Type

string

16.65.3 Default

varies by host

16.65.4 Read-Only

yes

16.65.5 Scope

session; saved in not saved

16.65.6 Description

Lists the Support File Search paths specified in the Options dialog box, Files tab.

16.65.7 Read/Written By

- findfile
- getenv

16.65.8 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.65.9 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-CA03058E-999B-4BA0-BA38-61DFC5EB3214>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/acadprefix-system-variable>

16.66 System Variable Entry: ACADVER

16.66.1 Name

ACADVER

16.66.2 Type

string

16.66.3 Default

varies by session

16.66.4 Read-Only

yes

16.66.5 Scope

session; saved in not saved

16.66.6 Description

AutoCAD version: Shows the AutoCAD® compatible program version number.

16.66.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.66.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/acadver-system-variable>

16.67 System Variable Entry: ACISHLRRESOLUTION

16.67.1 Name

ACISHLRRESOLUTION

16.67.2 Type

real

16.67.3 Default

-1.0

16.67.4 Read-Only

no

16.67.5 Scope

user preference; saved in user preference

16.67.6 Description

Hidden line removal resolution: Controls the smallest distance used for Hidden Line Removal calculation. Negative value is Auto-calibration based on the size of the model (recommended). For very small entities the value can be set to 0.001 or smaller. BricsCAD only

16.67.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.67.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/acishlrresolution-system-variable>

16.68 System Variable Entry: ACISOUTVER

16.68.1 Name

ACISOUTVER

16.68.2 Type

short integer

16.68.3 Default

70

16.68.4 Read-Only

no

16.68.5 Scope

session; saved in not saved

16.68.6 Description

Acisout version: Controls the ACIS version of the SAT files for the ACISOUT command.

16.68.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.68.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/acisoutver-system-variable>

16.69 System Variable Entry: ACISSAVEASMODE

16.69.1 Name

ACISSAVEASMODE

16.69.2 Type

short integer

16.69.3 Default

0

16.69.4 Read-Only

no

16.69.5 Scope

registry; saved in registry

16.69.6 Description

Acis save as mode: Controls the explode mode of ACIS entities (3DSolids, Bodies, Regions) when saved to R12. BricsCAD only

16.69.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.69.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/acissaveasmode-system-variable>

16.70 System Variable Entry: ACTIVITYINSIGHTSPATH

16.70.1 Name

ACTIVITYINSIGHTSPATH

16.70.2 Type

string

16.70.3 Default

"C:\\
username}\\\\\\\\"

16.70.4 Read-Only

no

16.70.5 Scope

registry; saved in registry

16.70.6 Description

Specifies the location Activity Insights activities are recorded.

16.70.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.70.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-06C78FAB-4DC9-467C-9D6A-82E1DD0F64B2>

16.71 System Variable Entry: ACTIVITYINSIGHTSSTATE

16.71.1 Name

ACTIVITYINSIGHTSSTATE

16.71.2 Type

integer

16.71.3 Default

0

16.71.4 Read-Only

no

16.71.5 Scope

session; saved in not saved

16.71.6 Description

Indicates whether the Activity Insights palette is open or closed.

16.71.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.71.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-7B01C113-BB74-4EC2-8B9E-633D3EE9A323>

16.72 System Variable Entry: ACTIVITYINSIGHTSSUPPORT

16.72.1 Name

ACTIVITYINSIGHTSSUPPORT

16.72.2 Type

integer (bit-coded)

16.72.3 Default

3

16.72.4 Read-Only

no

16.72.5 Scope

registry; saved in registry

16.72.6 Description

Enables or disables the Activity Insights feature.

16.72.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.72.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-EAA81DD3-E8BD-45AE-AC1C-B757209A9ABC>

16.73 System Variable Entry: ACTIVITYINSIGHTSVIEWED-LOGGING

16.73.1 Name

ACTIVITYINSIGHTSVIEWEDLOGGING

16.73.2 Type

integer (bit-coded)

16.73.3 Default

1

16.73.4 Read-Only

no

16.73.5 Scope

registry; saved in registry

16.73.6 Description

Turns on or off logging Opened activities, such as opening a drawing.

16.73.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.73.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-34D77CB3-E9E2-4956-835C-8EDFAA1D00CE>

16.74 System Variable Entry: ACTPATH

16.74.1 Name

ACTPATH

16.74.2 Type

string

16.74.3 Default

””

16.74.4 Read-Only

no

16.74.5 Scope

registry; saved in registry

16.74.6 Description

Specifies the additional paths to load action macros from for playback.

16.74.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.74.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-E63ED007-A45B-46EE-9777-3665AA593B6C>

16.75 System Variable Entry: ACTRECORDERSTATE

16.75.1 Name

ACTRECORDERSTATE

16.75.2 Type

integer

16.75.3 Default

0

16.75.4 Read-Only

yes

16.75.5 Scope

session; saved in not saved

16.75.6 Description

Specifies the current state of the Action Recorder.

16.75.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.75.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-6254B281-6523-4C56-AC9E-CDEE32A27606>

16.76 System Variable Entry: ACTRECPATH

16.76.1 Name

ACTRECPATH

16.76.2 Type

string

16.76.3 Default

varies by host

16.76.4 Read-Only

no

16.76.5 Scope

registry; saved in registry

16.76.6 Description

Specifies the path used to store new action macros.

16.76.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.76.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-DE0E53A7-9DF4-4545-93AC-F57ACA43B391>

16.77 System Variable Entry: ACTUI

16.77.1 Name

ACTUI

16.77.2 Type

integer (bit-coded)

16.77.3 Default

6

16.77.4 Read-Only

no

16.77.5 Scope

registry; saved in registry

16.77.6 Description

Controls the behavior of the Action Recorder panel when recording and playing back macros.

16.77.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.77.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-06266E09-CC31-46F4-B386-7F923026B6D0>

16.78 System Variable Entry: ADAPTIVEGRIDSTEPSIZE

16.78.1 Name

ADAPTIVEGRIDSTEPSIZE

16.78.2 Type

real

16.78.3 Default

4.0

16.78.4 Read-Only

no

16.78.5 Scope

user preference; saved in user preference

16.78.6 Description

Adaptive grid step size: Controls the snap spacing for 'Adaptive Grid Snap' mode of SNAPTYPE system variable, in pixels. Also Controls the step size of the Manipulator ruler. BricsCAD only

16.78.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.78.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/adaptivestepsize-system-variable>

16.79 System Variable Entry: ADCSTATE

16.79.1 Name

ADCSTATE

16.79.2 Type

integer

16.79.3 Default

varies by host

16.79.4 Read-Only

yes

16.79.5 Scope

session; saved in not saved

16.79.6 Description

Indicates whether the DesignCenter window is open or closed.

16.79.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.79.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-E26B699F-2274-4C02-883E-757FCFB41357>

16.80 System Variable Entry: AFLAGS

16.80.1 Name

AFLAGS

16.80.2 Type

integer (bit-coded), range 0 to 63

16.80.3 Default

16

16.80.4 Read-Only

no

16.80.5 Scope

session; saved in not saved

16.80.6 Description

Sets options for attributes.

16.80.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short; default AutoCAD 16, BricsCAD 0.

16.80.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-4C49E401-7CCA-45B9-A126-36E597416236>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/aflags-system-variable>

16.81 System Variable Entry: ALIGNDIMENSIONONISOMETRIC

16.81.1 Name

ALIGNDIMENSIONONISOMETRIC

16.81.2 Type

integer

16.81.3 Default

1

16.81.4 Read-Only

no

16.81.5 Scope

registry; saved in registry

16.81.6 Description

Dimension alignment: Enables isometric dimensions. Dimensions are aligned to the geometry. BricsCAD only

16.81.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.81.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/aligndimensiononisometric-system-variable>

16.82 System Variable Entry: ALLOWEDBENDANGLES

16.82.1 Name

ALLOWEDBENDANGLES

16.82.2 Type

short integer (bit-coded), range 1 to 31

16.82.3 Default

1

16.82.4 Read-Only

no

16.82.5 Scope

registry; saved in registry

16.82.6 Description

Allowed bend angles: Sets allowed bend angles for MEP elements. BricsCAD only

16.82.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.82.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/allowedbendangles-system-variable>

16.83 System Variable Entry: ALLOWTABEXTERNALMOVE

16.83.1 Name

ALLOWTABEXTERNALMOVE

16.83.2 Type

integer

16.83.3 Default

1

16.83.4 Read-Only

no

16.83.5 Scope

user preference; saved in user preference

16.83.6 Description

Move tabs externally (Mac & Linux): Allows a tab to be moved to another tab control, in the documents tab. BricsCAD only

16.83.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.83.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/allowtabexternalmove-system-variable>

16.84 System Variable Entry: ALLOWTABMOVE

16.84.1 Name

ALLOWTABMOVE

16.84.2 Type

integer

16.84.3 Default

1

16.84.4 Read-Only

no

16.84.5 Scope

user preference; saved in user preference

16.84.6 Description

Move tabs (Mac & Linux): Allows a tab to be dragged horizontally, in the documents tab. BricsCAD only

16.84.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.84.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/allowtabmove-system-variable>

16.85 System Variable Entry: ALLOWTABSPLIT

16.85.1 Name

ALLOWTABSPLIT

16.85.2 Type

integer

16.85.3 Default

1

16.85.4 Read-Only

no

16.85.5 Scope

user preference; saved in user preference

16.85.6 Description

Split tabs (Mac & Linux): Allows drag to split the tab control, in the documents tab. BricsCAD only

16.85.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.85.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/allowtabsplit-system-variable>

16.86 System Variable Entry: AMPOWERDIMDISPLAY

16.86.1 Name

AMPOWERDIMDISPLAY

16.86.2 Type

short integer, range 0 to 2

16.86.3 Default

1

16.86.4 Read-Only

no

16.86.5 Scope

drawing; saved in drawing

16.86.6 Description

Mechanical 2D Editor: Controls the opening the Edit Dimensioning dialog box after placing a power dimension. BricsCAD only

16.86.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.86.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/ampowerdimdisplay-system-variable>

16.87 System Variable Entry: AMSYMSCALE

16.87.1 Name

AMSYMSCALE

16.87.2 Type

real

16.87.3 Default

1.0

16.87.4 Read-Only

no

16.87.5 Scope

drawing; saved in drawing

16.87.6 Description

Mechanical2D annotation scaling: Controls the display of Mechanical2D symbols and text in Model Space.

16.87.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.87.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/amsymscale-system-variable>

16.88 System Variable Entry: ANGBASE

16.88.1 Name

ANGBASE

16.88.2 Type

real

16.88.3 Default

0.0

16.88.4 Read-Only

no

16.88.5 Scope

drawing; saved in drawing

16.88.6 Description

Sets the zero (0) base angle with respect to the current UCS.

16.88.7 Read/Written By

- angle
- angtos
- getangle
- getorient
- rtos
- setvar

16.88.8 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: default AutoCAD 0.0000, BricsCAD 0.0.

16.88.9 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-B7CAB5F3-16BC-4E06-97BC-AAAEC052727E>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/angbase-system-variable>

16.89 System Variable Entry: ANGDIR

16.89.1 Name

ANGDIR

16.89.2 Type

integer

16.89.3 Default

0

16.89.4 Read-Only

no

16.89.5 Scope

drawing; saved in drawing

16.89.6 Description

Sets the direction of positive angles.

16.89.7 Read/Written By

- angle
- angtos
- getangle
- getorient

16.89.8 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.89.9 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-A432574D-35B6-4D9E-8D8F-4259F2066234>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/angdir-system-variable>

16.90 System Variable Entry: ANNOALLVISIBLE

16.90.1 Name

ANNOALLVISIBLE

16.90.2 Type

integer

16.90.3 Default

1

16.90.4 Read-Only

no

16.90.5 Scope

drawing; saved in drawing

16.90.6 Description

Hides or displays annotative objects that do not support the current annotation scale.

16.90.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.90.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-D8E50F6F-FB71-4A20-A3B9-7701C0518B81>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/annoallvisible-system-variable>

16.91 System Variable Entry: ANNOAUTOSCALE

16.91.1 Name

ANNOAUTOSCALE

16.91.2 Type

integer (bit-coded)

16.91.3 Default

-4

16.91.4 Read-Only

no

16.91.5 Scope

registry; saved in registry

16.91.6 Description

Updates annotative objects to support the annotation scale when the annotation scale is changed.

16.91.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.91.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-15AC894C-4B21-4543-AD86-F86C8BFAC6E6>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/annoautoscale-system-variable>

16.92 System Variable Entry: ANNOMONITOR

16.92.1 Name

ANNOMONITOR

16.92.2 Type

integer

16.92.3 Default

-2

16.92.4 Read-Only

no

16.92.5 Scope

drawing; saved in drawing

16.92.6 Description

Turns the annotation monitor on or off. When the annotation monitor is turned on, a yellow warning badge displays on all non-associative dimensions and leaders.

16.92.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.92.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-EEF0796B-9C18-476A-A0E8-876BC5346C97>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/annomonitor-system-variable>

16.93 System Variable Entry: ANNOSCALEZOOM

16.93.1 Name

ANNOSCALEZOOM

16.93.2 Type

integer

16.93.3 Default

0

16.93.4 Read-Only

no

16.93.5 Scope

registry; saved in registry

16.93.6 Description

Determines whether the mouse wheel zoom in paperspace viewports is controlled by specific zoom scales or independent of viewport scales (legacy behavior).

16.93.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.93.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-73EB72D3-3C77-471B-AED7-EE1629180E9B>

16.94 System Variable Entry: ANNOTATIVEDWG

16.94.1 Name

ANNOTATIVEDWG

16.94.2 Type

integer

16.94.3 Default

0

16.94.4 Read-Only

no

16.94.5 Scope

drawing; saved in drawing

16.94.6 Description

Specifies whether or not the drawing will behave as an annotative block when inserted into another drawing.

16.94.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.94.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-1720C9E3-B398-42FB-A5CA-6C983399FB04>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/annotativedwg-system-variable>

16.95 System Variable Entry: ANTIALIASRENDER

16.95.1 Name

ANTIALIASRENDER

16.95.2 Type

short integer (bit-coded), range 1 to 5

16.95.3 Default

2

16.95.4 Read-Only

no

16.95.5 Scope

user preference; saved in user preference

16.95.6 Description

Anti-alias amount for render: Controls the smoothness of the output of the RENDER command. For values higher than 1, an anti-aliased output is calculated, at a cost, this increases with bigger values. BricsCAD only

16.95.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.95.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/antialiasrender-system-variable>

16.96 System Variable Entry: ANTIALIASSCREEN

16.96.1 Name

ANTIALIASSCREEN

16.96.2 Type

short integer (bit-coded), range 1 to 5

16.96.3 Default

1

16.96.4 Read-Only

no

16.96.5 Scope

user preference; saved in user preference

16.96.6 Description

Anti-alias amount for screen: Controls the smoothness of display output. When the value is greater than 1, the RedSdkLineSmoothing option is ignored. For values greater than 1, an anti-aliased output is calculated. This calculation incurs a performance cost which increases with the value. BricsCAD only

16.96.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.96.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/antialiasscreen-system-variable>

16.97 System Variable Entry: APBOX

16.97.1 Name

APBOX

16.97.2 Type

integer

16.97.3 Default

0

16.97.4 Read-Only

no

16.97.5 Scope

registry; saved in registry

16.97.6 Description

Turns the display of the AutoSnap aperture box on or off.

16.97.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.97.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-A13A75DC-5D0F-4730-834C-4AA078077B12>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/apbox-system-variable>

16.98 System Variable Entry: APERTURE

16.98.1 Name

APERTURE

16.98.2 Type

integer, range 1 to 50

16.98.3 Default

10

16.98.4 Read-Only

no

16.98.5 Scope

registry; saved in registry

16.98.6 Description

Controls the size of the object target box.

16.98.7 Read/Written By

- osnap

16.98.8 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.98.9 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-2AA8CF8E-267F-4BD6-80CD-0E7C48B8A9F1>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/aperture-system-variable>

16.99 System Variable Entry: APPAUTOLOAD

16.99.1 Name

APPAUTOLOAD

16.99.2 Type

integer (bit-coded)

16.99.3 Default

14

16.99.4 Read-Only

no

16.99.5 Scope

user preference; saved in user preference

16.99.6 Description

Controls when plug-in applications are loaded.

16.99.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.99.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-33E75B9A-93AC-4FBF-B941-0F99A6AC3617>

16.100 System Variable Entry: APPLYGLOBALOPACITIES

16.100.1 Name

APPLYGLOBALOPACITIES

16.100.2 Type

integer

16.100.3 Default

0

16.100.4 Read-Only

no

16.100.5 Scope

registry; saved in registry

16.100.6 Description

Applies transparency settings to all palettes.

16.100.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.100.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-AE962D09-C241-4212-9916-D1EE5BA7BCE1>

16.101 System Variable Entry: APSTATE

16.101.1 Name

APSTATE

16.101.2 Type

integer

16.101.3 Default

0

16.101.4 Read-Only

yes

16.101.5 Scope

session; saved in not saved

16.101.6 Description

Indicates whether the Block Authoring Palettes window in the Block Editor is open or closed.

16.101.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.101.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-1F6ECE63-71DC-4EBF-8E49-2C50100632C0>

16.102 System Variable Entry: AREA

16.102.1 Name

AREA

16.102.2 Type

real

16.102.3 Default

0.0

16.102.4 Read-Only

yes

16.102.5 Scope

session; saved in not saved

16.102.6 Description

Stores the last area computed by the AREA and MEASUREGEOM commands.

16.102.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.102.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-CD078D73-FE99-49A0-8FFA-0D2FFA906167>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/area-system-variable>

16.103 System Variable Entry: AREAPREC

16.103.1 Name

AREAPREC

16.103.2 Type

short integer (bit-coded)

16.103.3 Default

-1

16.103.4 Read-Only

no

16.103.5 Scope

registry; saved in registry

16.103.6 Description

Area precision: Controls the number of decimal places displayed for areas, if area properties are formatted with the PROPUNITS system variable.
BricsCAD only

16.103.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.103.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/areaprec-system-variable>

16.104 System Variable Entry: AREAUNITS

16.104.1 Name

AREAUNITS

16.104.2 Type

string

16.104.3 Default

"in ft mi µm mm cm m km"

16.104.4 Read-Only

no

16.104.5 Scope

registry; saved in registry

16.104.6 Description

Area units: Controls a list of units used to display areas, if area properties are formatted with the PROPUNITS system variable. If empty, all areas match the drawing. Note: The string contains a space-separated list of unit abbreviations. BricsCAD only

16.104.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.104.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/areaunits-system-variable>

16.105 System Variable Entry: ARRAYASSOCIATIVITY

16.105.1 Name

ARRAYASSOCIATIVITY

16.105.2 Type

integer

16.105.3 Default

1

16.105.4 Read-Only

no

16.105.5 Scope

registry; saved in registry

16.105.6 Description

Sets the default behavior of new arrays to be associative or nonassociative.

16.105.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.105.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-298A0544-E6C3-496F-9FA1-12AEC2FF5647>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/arrayassociativity-system-variable>

16.106 System Variable Entry: ARRAYEDITSTATE

16.106.1 Name

ARRAYEDITSTATE

16.106.2 Type

integer

16.106.3 Default

0

16.106.4 Read-Only

yes

16.106.5 Scope

session; saved in not saved

16.106.6 Description

Indicates whether the drawing is in the array editing state, which is activated while editing an associative array's source objects.

16.106.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.106.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-E71DAD51-8377-4B32-B33D-095E45672F18>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/arrayeditstate-system-variable>

16.107 System Variable Entry: ARRAYTYPE

16.107.1 Name

ARRAYTYPE

16.107.2 Type

integer

16.107.3 Default

0

16.107.4 Read-Only

no

16.107.5 Scope

registry; saved in registry

16.107.6 Description

Specifies the default array type.

16.107.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.107.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-ED05306A-0D24-4FE7-9B50-1207BCA2B4B8>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/arraytype-system-variable>

16.108 System Variable Entry: ASSISTANTSTATE

16.108.1 Name

ASSISTANTSTATE

16.108.2 Type

integer

16.108.3 Default

0

16.108.4 Read-Only

yes

16.108.5 Scope

registry; saved in registry

16.108.6 Description

Indicates whether the Autodesk Assistant palette is open, closed or hidden.

16.108.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.108.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-9F26CB82-3F66-4116-90F9-810ACFE6A613>

16.109 System Variable Entry: ATTDIA

16.109.1 Name

ATTDIA

16.109.2 Type

integer

16.109.3 Default

1

16.109.4 Read-Only

no

16.109.5 Scope

registry; saved in registry

16.109.6 Description

Controls whether the INSERT command uses a dialog box for attribute value entry.

16.109.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.109.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-DD7AE7D3-D9D7-49E7-948E-079D8451FFA2>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/attdia-system-variable>

16.110 System Variable Entry: ATTFULLUPDATE

16.110.1 Name

ATTFULLUPDATE

16.110.2 Type

integer

16.110.3 Default

1

16.110.4 Read-Only

no

16.110.5 Scope

registry; saved in registry

16.110.6 Description

Reset attributes after editing the block record: If on, the attributes in the associated block references are synchronized. That is, their parameters, such as rotation angle, position, height of the font, etc. (except the text values), are set in accordance with those of the attribute definitions. Additionally, the missing attributes are restored, and those that have no definitions are removed. If off, nothing is done. BricsCAD only

16.110.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.110.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/attfullupdate-system-variable>

16.111 System Variable Entry: ATTIPE

16.111.1 Name

ATTIPE

16.111.2 Type

integer

16.111.3 Default

0

16.111.4 Read-Only

no

16.111.5 Scope

registry; saved in registry

16.111.6 Description

Controls which Text Formatting toolbar is displayed with the in-place editor for modifying multiline attributes.

16.111.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.111.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-1DD42986-CA88-424A-BCC8-6C4DD64488C5>

16.112 System Variable Entry: ATTMODE

16.112.1 Name

ATTMODE

16.112.2 Type

integer

16.112.3 Default

1

16.112.4 Read-Only

no

16.112.5 Scope

drawing; saved in drawing

16.112.6 Description

Controls display of attributes.

16.112.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.112.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-D6DE0459-AE5C-4FFF-9E1C-2468D532C49D>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/attmode-system-variable>

16.113 System Variable Entry: ATTMULTI

16.113.1 Name

ATTMULTI

16.113.2 Type

integer

16.113.3 Default

1

16.113.4 Read-Only

no

16.113.5 Scope

registry; saved in registry

16.113.6 Description

Controls whether multiline attributes can be created.

16.113.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.113.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-9386111C-705B-4B4D-B7F9-36C38F67DB84>

16.114 System Variable Entry: ATTRACTIONDISTANCE

16.114.1 Name

ATTRACTIONDISTANCE

16.114.2 Type

short integer

16.114.3 Default

4

16.114.4 Read-Only

no

16.114.5 Scope

user preference; saved in user preference

16.114.6 Description

Grips attraction distance: Sets the grip attraction distance. See also the ENABLEATTRACTION system variable. BricsCAD only

16.114.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.114.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/attractiondistance-system-variable>

16.115 System Variable Entry: ATTREQ

16.115.1 Name

ATTREQ

16.115.2 Type

integer

16.115.3 Default

1

16.115.4 Read-Only

no

16.115.5 Scope

registry; saved in registry

16.115.6 Description

Controls whether INSERT uses default attribute settings during insertion of blocks.

16.115.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.115.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-7A7229F8-C99D-42C5-8784-4231767611F8>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/attreq-system-variable>

16.116 System Variable Entry: AUDITCTL

16.116.1 Name

AUDITCTL

16.116.2 Type

integer

16.116.3 Default

0

16.116.4 Read-Only

no

16.116.5 Scope

registry; saved in registry

16.116.6 Description

Controls whether AUDIT creates an audit report (ADT) file.

16.116.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.116.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-05A7363C-4A43-4984-8262-DA7BEE7B860C>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/auditctl-system-variable>

16.117 System Variable Entry: AUDITERRORCOUNT

16.117.1 Name

AUDITERRORCOUNT

16.117.2 Type

short integer

16.117.3 Default

varies by session

16.117.4 Read-Only

yes

16.117.5 Scope

session; saved in not saved

16.117.6 Description

Audit Error Count: The number of errors found in the last audit (AUDIT command).

16.117.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.117.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/auditerrordcount-system-variable>

16.118 System Variable Entry: AUNITS

16.118.1 Name

AUNITS

16.118.2 Type

integer (bit-coded)

16.118.3 Default

0

16.118.4 Read-Only

no

16.118.5 Scope

drawing; saved in drawing

16.118.6 Description

Sets units for angles.

16.118.7 Read/Written By

- angtof
- angtos
- getangle
- getorient

16.118.8 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.118.9 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-C7C0F6A5-7982-43DB-97F9-5B9B0044E9FA>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/aunits-system-variable>

16.119 System Variable Entry: AUPREC

16.119.1 Name

AUPREC

16.119.2 Type

integer (bit-coded)

16.119.3 Default

0

16.119.4 Read-Only

no

16.119.5 Scope

drawing; saved in drawing

16.119.6 Description

Sets the display precision for angular units and coordinates.

16.119.7 Read/Written By

- angtos
- getangle
- getorient

16.119.8 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.119.9 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-EE1ED20C-1096-4299-820F-83F1BC9B96F3>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/auprec-system-variable>

16.120 System Variable Entry: AUTOCOMPLETEDDELAY

16.120.1 Name

AUTOCOMPLETEDDELAY

16.120.2 Type

real

16.120.3 Default

0.3

16.120.4 Read-Only

no

16.120.5 Scope

registry; saved in registry

16.120.6 Description

Auto complete delay: Controls the delay before features display at the Command line. See also the AUTOCOMPLETEMODE system variable. Values between 0.0 and 10.0 are accepted.

16.120.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.120.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/autocompletedelay-system-variable>

16.121 System Variable Entry: AUTOCOMPLETEMODE

16.121.1 Name

AUTOCOMPLETEMODE

16.121.2 Type

short integer (bit-coded), range 0 to 63

16.121.3 Default

47

16.121.4 Read-Only

no

16.121.5 Scope

registry; saved in registry

16.121.6 Description

Auto complete mode: Controls the types features shown at the Command line.

16.121.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.121.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/autocompletemode-system-variable>

16.122 System Variable Entry: AUTODWFPUBLISH

16.122.1 Name

AUTODWFPUBLISH

16.122.2 Type

integer

16.122.3 Default

0

16.122.4 Read-Only

no

16.122.5 Scope

registry; saved in registry

16.122.6 Description

Controls whether DWF (Design Web Format) files are created automatically when you save or close drawing (DWG) files.

16.122.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.122.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-67B17228-BFB3-4223-BF64-EE09FF2A4C57>

16.123 System Variable Entry: AUTOMATICCONNECTION

16.123.1 Name

AUTOMATICCONNECTION

16.123.2 Type

integer

16.123.3 Default

1

16.123.4 Read-Only

no

16.123.5 Scope

registry; saved in registry

16.123.6 Description

Automatic connection: Controls automatic creation of connections for the BIMLINEARSOLID and BIMAPPLYPROFILE commands. BricsCAD only

16.123.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.123.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/automaticconnection-system-variable>

16.124 System Variable Entry: AUTOMATICPUB

16.124.1 Name

AUTOMATICPUB

16.124.2 Type

integer

16.124.3 Default

0

16.124.4 Read-Only

no

16.124.5 Scope

user preference; saved in user preference

16.124.6 Description

Controls whether electronic files (DWF/PDF) are created automatically when you save or close drawing (DWG) files.

16.124.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.124.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-04E06B5A-D6BA-41A9-B5C8-BF3DC6BE9C83>

16.125 System Variable Entry: AUTOMATICSTAIRSECTIONBEHAVIOR

16.125.1 Name

AUTOMATICSTAIRSECTIONBEHAVIOR

16.125.2 Type

short integer

16.125.3 Default

0

16.125.4 Read-Only

no

16.125.5 Scope

registry; saved in registry

16.125.6 Description

Automatic stair section behavior: Controls the generation of 2D representations of BIM stair entities during a section generation. Affects only the automatic stair sectioning behavior. See the BIMGENERATE2DSTAIR command. BricsCAD only

16.125.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.125.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/automaticstairsectionbehavior-system-variable>

16.126 System Variable Entry: AUTOMATICTEES

16.126.1 Name

AUTOMATICTEES

16.126.2 Type

integer

16.126.3 Default

0

16.126.4 Read-Only

no

16.126.5 Scope

registry; saved in registry

16.126.6 Description

Automatic tees: Controls the automatic creation of T type connections during the BIMFLOWCONNECT command. BricsCAD only

16.126.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.126.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/automatictees-system-variable>

16.127 System Variable Entry: AUTOPLACEMENT

16.127.1 Name

AUTOPLACEMENT

16.127.2 Type

integer

16.127.3 Default

1

16.127.4 Read-Only

no

16.127.5 Scope

registry; saved in registry

16.127.6 Description

Controls whether placement suggestions are displayed as you insert a block.

16.127.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.127.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-84265932-8B02-4740-A178-EF1BBD7CB01F>

16.128 System Variable Entry: AUTORESETSCALES

16.128.1 Name

AUTORESETSCALES

16.128.2 Type

short integer

16.128.3 Default

0

16.128.4 Read-Only

no

16.128.5 Scope

registry; saved in registry

16.128.6 Description

Purge unused scales: Controls how unused annotation scales are managed, when a drawing containing a large number of scales is loaded. A large number of annotation scales decreases performance. BricsCAD only

16.128.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.128.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/autoresetscales-system-variable>

16.129 System Variable Entry: AUTOSAVECHECKSONLYFIRSTBITDBMOD

16.129.1 Name

AUTOSAVECHECKSONLYFIRSTBITDBMOD

16.129.2 Type

integer

16.129.3 Default

1

16.129.4 Read-Only

no

16.129.5 Scope

user preference; saved in user preference

16.129.6 Description

Ignore all but first bit of DBMOD for autosave: Does not create autosave files for drawings, when they have been viewed but not edited (includes zoom and pan actions). BricsCAD only

16.129.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.129.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/autosavecheckonlyfirstbitdbmod-system-variable>

16.130 System Variable Entry: AUTOSNAP

16.130.1 Name

AUTOSNAP

16.130.2 Type

integer (bit-coded), range 0 to 127

16.130.3 Default

63

16.130.4 Read-Only

no

16.130.5 Scope

registry; saved in registry

16.130.6 Description

Controls the display of the AutoSnap marker, tooltip, and magnet.

16.130.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short; default AutoCAD 63, BricsCAD 127.

16.130.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-BE7947EB-2A08-4406-A169-4C5E125B1F4D>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/autosnap-system-variable>

16.131 System Variable Entry: AUTOTRACKINGVECCOLOR

16.131.1 Name

AUTOTRACKINGVECCOLOR

16.131.2 Type

short integer, range 1 to 255

16.131.3 Default

171

16.131.4 Read-Only

no

16.131.5 Scope

user preference; saved in user preference

16.131.6 Description

Auto tracking vector color: Controls the color of polar/snap tracking markers.

16.131.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.131.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/autotrackingveccolor-system-variable>

16.132 System Variable Entry: AUTOVPFITTING

16.132.1 Name

AUTOVPFITTING

16.132.2 Type

integer

16.132.3 Default

1

16.132.4 Read-Only

no

16.132.5 Scope

registry; saved in registry

16.132.6 Description

Automatically resize viewports: Controls if viewport borders automatically adjust to fit, when a viewport is updated. BricsCAD only

16.132.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.132.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/a/autovpfitting-system-variable>

16.133 System Variable Entry: BACKGROUNDPLOT

16.133.1 Name

BACKGROUNDPLOT

16.133.2 Type

integer, range 0 to 3

16.133.3 Default

2

16.133.4 Read-Only

no

16.133.5 Scope

registry; saved in registry

16.133.6 Description

Controls whether background plotting is turned on or off for plotting and publishing.

16.133.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: type AutoCAD Integer, BricsCAD Short.

16.133.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-713029B7-B5AC-4860-BE2E-74878D418EA4>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/backgroundplot-system-variable>

16.134 System Variable Entry: BACKZ

16.134.1 Name

BACKZ

16.134.2 Type

real

16.134.3 Default

0.0

16.134.4 Read-Only

yes

16.134.5 Scope

drawing; saved in drawing

16.134.6 Description

Stores the back clipping plane offset from the target plane for the current viewport, in drawing units.

16.134.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+. Divergence: default AutoCAD 0.0000, BricsCAD 0.0; read-only AutoCAD True, BricsCAD False.

16.134.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-EC6CF0EF-05A7-4D12-9437-46360B2BBF62>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/backz-system-variable>

16.135 System Variable Entry: BACTIONBARMODE

16.135.1 Name

BACTIONBARMODE

16.135.2 Type

integer

16.135.3 Default

1

16.135.4 Read-Only

no

16.135.5 Scope

registry; saved in registry

16.135.6 Description

Indicates whether the action bars or the legacy action objects are displayed in the Block Editor.

16.135.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.135.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-4E589CD2-EB52-4CF6-949F-4789C53F6286>

16.136 System Variable Entry: BACTIONCOLOR

16.136.1 Name

BACTIONCOLOR

16.136.2 Type

string

16.136.3 Default

"7"

16.136.4 Read-Only

no

16.136.5 Scope

registry; saved in registry

16.136.6 Description

Sets the text color of actions in the Block Editor.

16.136.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.136.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-7BEFF137-53F2-4D69-A19B-63A68EE4A54E>

16.137 System Variable Entry: BASEFILE

16.137.1 Name

BASEFILE

16.137.2 Type

string

16.137.3 Default

varies by registry

16.137.4 Read-Only

no

16.137.5 Scope

registry; saved in registry

16.137.6 Description

Template: The file path and default template file name for new drawings.
If empty, uses built-in defaults. BricsCAD only

16.137.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.137.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/basefile-system-variable>

16.138 System Variable Entry: BCFSOURCEURL

16.138.1 Name

BCFSOURCEURL

16.138.2 Type

string

16.138.3 Default

varies by registry

16.138.4 Read-Only

no

16.138.5 Scope

registry; saved in registry

16.138.6 Description

BCF source url: The address (URL) of the BCF source. BricsCAD only

16.138.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.138.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bcfsourceurl-system-variable>

16.139 System Variable Entry: BCONSTATUSMODE

16.139.1 Name

BCONSTATUSMODE

16.139.2 Type

integer

16.139.3 Default

0

16.139.4 Read-Only

no

16.139.5 Scope

session; saved in not saved

16.139.6 Description

Turns the constraint display status on and off and controls the shading of objects based on their constraint level.

16.139.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.139.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-30193F93-378C-4A90-91E0-4737A772330C>

16.140 System Variable Entry: BCONVERTLAYER

16.140.1 Name

BCONVERTLAYER

16.140.2 Type

integer

16.140.3 Default

1

16.140.4 Read-Only

no

16.140.5 Scope

registry; saved in registry

16.140.6 Description

Controls whether the object layer in the block definition will become Layer 0 after objects are converted into blocks.

16.140.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.140.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-63A8986F-1FCC-4480-947B-E576260FC4D4>

16.141 System Variable Entry: BDEPENDENCYHIGHLIGHT

16.141.1 Name

BDEPENDENCYHIGHLIGHT

16.141.2 Type

integer

16.141.3 Default

1

16.141.4 Read-Only

no

16.141.5 Scope

registry; saved in registry

16.141.6 Description

Controls whether or not dependent objects are dependency highlighted when a parameter, action, or grip is selected in the Block Editor.

16.141.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.141.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-6F4561D1-9A00-4C57-9D14-959A4B11F6AB>

16.142 System Variable Entry: BEDITASSOCMODE

16.142.1 Name

BEDITASSOCMODE

16.142.2 Type

integer

16.142.3 Default

1

16.142.4 Read-Only

no

16.142.5 Scope

registry; saved in registry

16.142.6 Description

Associative identifiers in BEDIT: Controls if additional service data is generated during the BEDIT command. This enables the automatic re-association of constraints and dimensions attached to the references of the block, including references in other documents. BricsCAD only

16.142.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.142.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/beditassocmode-system-variable>

16.143 System Variable Entry: BGCOREPUBLISH

16.143.1 Name

BGCOREPUBLISH

16.143.2 Type

integer

16.143.3 Default

1

16.143.4 Read-Only

no

16.143.5 Scope

registry; saved in registry

16.143.6 Description

Controls whether background publishing uses a single or multiple cores.

16.143.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.143.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-CEA83608-0AC0-4188-B413-DE586EBB103C>

16.144 System Variable Entry: BGRIPOBJCOLOR

16.144.1 Name

BGRIPOBJCOLOR

16.144.2 Type

string

16.144.3 Default

"141"

16.144.4 Read-Only

no

16.144.5 Scope

registry; saved in registry

16.144.6 Description

Sets the color of grips in the Block Editor.

16.144.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.144.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-929B1BF9-B156-438C-A609-83B7FC0385FD>

16.145 System Variable Entry: BGRIPOBJSIZE

16.145.1 Name

BGRIPOBJSIZE

16.145.2 Type

integer

16.145.3 Default

8

16.145.4 Read-Only

no

16.145.5 Scope

registry; saved in registry

16.145.6 Description

Sets the display size of custom grips in the Block Editor relative to the screen display.

16.145.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.145.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-1DF5F69B-ADF3-4936-ADCF-D2A841FF9019>

16.146 System Variable Entry: BILLOFMATERIALSSETTINGS

16.146.1 Name

BILLOFMATERIALSSETTINGS

16.146.2 Type

short integer (bit-coded)

16.146.3 Default

10

16.146.4 Read-Only

no

16.146.5 Scope

registry; saved in registry

16.146.6 Description

Bill of materials defaults: Sets the default options for BOMs (Bill Of Materials). BricsCAD only

16.146.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.146.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/billofmaterialssettings-system-variable>

16.147 System Variable Entry: BIMDEFAULTPROPERTIES-PATH

16.147.1 Name

BIMDEFAULTPROPERTIESPATH

16.147.2 Type

string

16.147.3 Default

"bimproj_{user.xml};bimproj_{IFC.xml};bimproj_{quantity.xml}"

16.147.4 Read-Only

no

16.147.5 Scope

registry; saved in registry

16.147.6 Description

Default properties path: The file paths for properties, loaded when a new document is opened. BricsCAD only

16.147.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.147.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bimdefaultpropertiespath-system-variable>

16.148 System Variable Entry: BIMMATCHPROP

16.148.1 Name

BIMMATCHPROP

16.148.2 Type

short integer, range 0 to 1

16.148.3 Default

1

16.148.4 Read-Only

no

16.148.5 Scope

drawing; saved in drawing

16.148.6 Description

Match BIM Properties: Matches BIM properties during the MATCHPROP command. BricsCAD only

16.148.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.148.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bimmatchprop-system-variable>

16.149 System Variable Entry: BIMOSMODE

16.149.1 Name

BIMOSMODE

16.149.2 Type

short integer, range 0 to 3

16.149.3 Default

3

16.149.4 Read-Only

no

16.149.5 Scope

registry; saved in registry

16.149.6 Description

BIM snap mode: Overrides the OSMODE and 3DOSMODE system variables for BIM entities. BricsCAD only

16.149.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.149.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bimosmode-system-variable>

16.150 System Variable Entry: BIMPROFILESTANDARDS

16.150.1 Name

BIMPROFILESTANDARDS

16.150.2 Type

string

16.150.3 Default

varies by registry

16.150.4 Read-Only

no

16.150.5 Scope

registry; saved in registry

16.150.6 Description

Profile's standards: Controls the profiles standards used in the Profiles dialog box and panel. Separate entries with semicolons (;). BricsCAD only

16.150.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.150.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/improfilestandards-system-variable>

16.151 System Variable Entry: BINDTYPE

16.151.1 Name

BINDTYPE

16.151.2 Type

integer

16.151.3 Default

0

16.151.4 Read-Only

no

16.151.5 Scope

session; saved in not saved

16.151.6 Description

Specifies the default or controls the naming behavior to be applied to "named objects" in an xref when a bind or an edit-in-place operation is performed on it.

16.151.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.151.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-1B410F79-266A-4D64-BEA6-87191A0B0D30>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bindtype-system-variable>

16.152 System Variable Entry: BKGCOLOR

16.152.1 Name

BKGCOLOR

16.152.2 Type

string

16.152.3 Default

"RGB:24,25,28"

16.152.4 Read-Only

no

16.152.5 Scope

registry; saved in registry

16.152.6 Description

Background color: Controls the background color of the drawing window in model space. BricsCAD only

16.152.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.152.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bkgcolor-system-variable>

16.153 System Variable Entry: BKGCOLORPS

16.153.1 Name

BKGCOLORPS

16.153.2 Type

string

16.153.3 Default

"RGB:250,250,250"

16.153.4 Read-Only

no

16.153.5 Scope

registry; saved in registry

16.153.6 Description

Paper space background color: Controls the background color of the drawing window in paper space. BricsCAD only

16.153.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.153.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bkgcolorps-system-variable>

16.154 System Variable Entry: BLIPMODE

16.154.1 Name

BLIPMODE

16.154.2 Type

integer (bit-coded)

16.154.3 Default

0

16.154.4 Read-Only

no

16.154.5 Scope

registry; saved in registry

16.154.6 Description

Blip mode: Determines whether or not marker blips are displayed.

16.154.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.154.8 Source Notes

- AutoCAD: AutoCAD 2026 ENU per-sysvar page (name match; GUID not yet captured)
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/blipmode-system-variable>

16.155 System Variable Entry: BLOCKCREATEMODE

16.155.1 Name

BLOCKCREATEMODE

16.155.2 Type

integer

16.155.3 Default

0

16.155.4 Read-Only

no

16.155.5 Scope

user preference; saved in user preference

16.155.6 Description

Sets the behavior for selected objects after creating blocks with the BLOCK and -BLOCK commands. The objects are either deleted, retained, or replaced with an instance of the block.

16.155.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.155.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-284CAB90-961D-4A2C-8BE4-A44ED750884C>

16.156 System Variable Entry: BLOCKEDITLOCK

16.156.1 Name

BLOCKEDITLOCK

16.156.2 Type

integer

16.156.3 Default

0

16.156.4 Read-Only

no

16.156.5 Scope

registry; saved in registry

16.156.6 Description

Disallows opening of the Block Editor and editing of dynamic block definitions.

16.156.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.156.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-BBBAEABC-E8D3-4E32-8D03-81067ADF4AD0>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/blockeditlock-system-variable>

16.157 System Variable Entry: BLOCKEDITOR

16.157.1 Name

BLOCKEDITOR

16.157.2 Type

integer

16.157.3 Default

0

16.157.4 Read-Only

yes

16.157.5 Scope

session; saved in not saved

16.157.6 Description

Indicates whether or not the Block Editor is open.

16.157.7 Compatibility

AutoCAD and BricsCAD. AutoCAD all; BricsCAD V20+.

16.157.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-171545A1-72C9-4BD0-97CC-BC06C694C3E9>
- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/blockeditor-system-variable>

16.158 System Variable Entry: BLOCKEXCLUDECOLOR

16.158.1 Name

BLOCKEXCLUDECOLOR

16.158.2 Type

integer

16.158.3 Default

7

16.158.4 Read-Only

no

16.158.5 Scope

registry; saved in registry

16.158.6 Description

Sets the color highlighting for identified instances that should not be converted into blocks.

16.158.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.158.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-2226EB5E-5283-4EB4-B2F2-67D22C3D74B4>

16.159 System Variable Entry: BLOCKIFYMODE

16.159.1 Name

BLOCKIFYMODE

16.159.2 Type

short integer (bit-coded), range 0 to 871

16.159.3 Default

71

16.159.4 Read-Only

no

16.159.5 Scope

registry; saved in registry

16.159.6 Description

Blockify settings: Controls the behavior of the BLOCKIFY command. BricsCAD only

16.159.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.159.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/blockifymode-system-variable>

16.160 System Variable Entry: BLOCKIFYTOLERANCE

16.160.1 Name

BLOCKIFYTOLERANCE

16.160.2 Type

real

16.160.3 Default

-1.0

16.160.4 Read-Only

no

16.160.5 Scope

registry; saved in registry

16.160.6 Description

Blockify tolerance: Controls the relative tolerance used in the BLOCKIFY command to determine if two entities are equal. A negative value means the program will determine the optimal tolerance (recommended). BricsCAD only

16.160.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.160.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/blockifytolerance-system-variable>

16.161 System Variable Entry: BLOCKINCLUDECOLOR

16.161.1 Name

BLOCKINCLUDECOLOR

16.161.2 Type

integer

16.161.3 Default

3

16.161.4 Read-Only

no

16.161.5 Scope

registry; saved in registry

16.161.6 Description

Sets the color highlighting for identified instances that can be converted into blocks.

16.161.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.161.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-247FC996-9F06-4B95-B884-DC189C66527F>

16.162 System Variable Entry: BLOCKLEVELOFDETAIL

16.162.1 Name

BLOCKLEVELOFDETAIL

16.162.2 Type

short integer

16.162.3 Default

1

16.162.4 Read-Only

no

16.162.5 Scope

drawing; saved in drawing

16.162.6 Description

Block Level of detail: Controls the block level of detail (LOD). BricsCAD only

16.162.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.162.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/blocklevelofdetail-system-variable>

16.163 System Variable Entry: BLOCKMRULIST

16.163.1 Name

BLOCKMRULIST

16.163.2 Type

integer

16.163.3 Default

50

16.163.4 Read-Only

no

16.163.5 Scope

registry; saved in registry

16.163.6 Description

Controls the number of most recently used blocks displayed in the Recent tab of the Blocks palette.

16.163.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.163.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-DDBD0D1B-F4DB-4E4A-8A38-016A2CBE5A20>

16.164 System Variable Entry: BLOCKNAVIGATE

16.164.1 Name

BLOCKNAVIGATE

16.164.2 Type

string

16.164.3 Default

””

16.164.4 Read-Only

no

16.164.5 Scope

registry; saved in registry

16.164.6 Description

Controls the folder, file, and blocks that are displayed in the Libraries tab of the Blocks palette.

16.164.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.164.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-44E6D5B6-5C89-422B-AA74-08780DF15E71>

16.165 System Variable Entry: BLOCKREDEFINEMODE

16.165.1 Name

BLOCKREDEFINEMODE

16.165.2 Type

integer

16.165.3 Default

1

16.165.4 Read-Only

no

16.165.5 Scope

registry; saved in registry

16.165.6 Description

Controls whether a dialog box or a prompt on the command line displays when inserting a block from the Blocks palette with the same name as an existing block definition.

16.165.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.165.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-15A13868-7582-4CCD-A8FC-9AF2D05E689C>

16.166 System Variable Entry: BLOCKSDATACOLLECTION

16.166.1 Name

BLOCKSDATACOLLECTION

16.166.2 Type

integer

16.166.3 Default

1

16.166.4 Read-Only

no

16.166.5 Scope

registry; saved in registry

16.166.6 Description

Controls whether the content data used when working with smart blocks is shared to the data collection service.

16.166.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.166.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-5AD60349-4CA6-4E2C-982E-8DAEF5E9FF53>

16.167 System Variable Entry: BLOCKSPATH

16.167.1 Name

BLOCKSPATH

16.167.2 Type

string

16.167.3 Default

varies by user preference

16.167.4 Read-Only

no

16.167.5 Scope

user preference; saved in user preference

16.167.6 Description

Blocks path: Specifies the file path used for the fifth folder on the left side of the Select Drawing File dialog box, opened with the INSERT command when the Browse option is selected. BricsCAD only

16.167.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.167.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/blockspath-system-variable>

16.168 System Variable Entry: BLOCKSRECENTFOLDER

16.168.1 Name

BLOCKSRECENTFOLDER

16.168.2 Type

string

16.168.3 Default

varies by host

16.168.4 Read-Only

no

16.168.5 Scope

registry; saved in registry

16.168.6 Description

Sets the path where the recently inserted or created blocks are stored.

16.168.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.168.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-2B47D65C-B88A-42C6-BE79-590901360A5D>

16.169 System Variable Entry: BLOCKSTATE

16.169.1 Name

BLOCKSTATE

16.169.2 Type

integer

16.169.3 Default

0

16.169.4 Read-Only

yes

16.169.5 Scope

session; saved in not saved

16.169.6 Description

Reports whether the Blocks palette is open or closed.

16.169.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.169.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-585DDD87-F28A-4150-A883-1A92C4604CAA>

16.170 System Variable Entry: BLOCKSYNCFOLDER

16.170.1 Name

BLOCKSYNCFOLDER

16.170.2 Type

string

16.170.3 Default

varies by host

16.170.4 Read-Only

no

16.170.5 Scope

registry; saved in registry

16.170.6 Description

Sets the path where the recent and favorite blocks are stored.

16.170.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.170.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-B03DC983-2680-4D73-B246-780990CBF4F3>

16.171 System Variable Entry: BLOCKTARGETCOLOR

16.171.1 Name

BLOCKTARGETCOLOR

16.171.2 Type

integer

16.171.3 Default

3

16.171.4 Read-Only

no

16.171.5 Scope

registry; saved in registry

16.171.6 Description

Sets the color highlighting for the specified instance from which the new block definition is created.

16.171.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.171.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-175D4755-223D-410A-B58E-71DE9243D43A>

16.172 System Variable Entry: BLOCKTESTWINDOW

16.172.1 Name

BLOCKTESTWINDOW

16.172.2 Type

integer

16.172.3 Default

0

16.172.4 Read-Only

yes

16.172.5 Scope

session; saved in not saved

16.172.6 Description

Indicates whether or not a test block window is current.

16.172.7 Compatibility

AutoCAD only. AutoCAD all; BricsCAD: not present.

16.172.8 Source Notes

- AutoCAD: <https://help.autodesk.com/view/ACD/2026/ENU/?guid=GUID-25FB9865-B874-4E34-9BE9-DD50B1BFAB22>

16.173 System Variable Entry: BMAUTOUPDATE

16.173.1 Name

BMAUTOUPDATE

16.173.2 Type

short integer

16.173.3 Default

1

16.173.4 Read-Only

no

16.173.5 Scope

registry; saved in registry

16.173.6 Description

Update external components: Controls when external assembly components are reloaded to reflect the changes in their definition files. BricsCAD only

16.173.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.173.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bmautoupdate-system-variable>

16.174 System Variable Entry: BMEXTERNALIZEILLEGALSYMBOLS

16.174.1 Name

BMEXTERNALIZEILLEGALSYMBOLS

16.174.2 Type

short integer

16.174.3 Default

3

16.174.4 Read-Only

no

16.174.5 Scope

registry; saved in registry

16.174.6 Description

Illegal symbols treatment: Defines treatment of symbols that are not allowed in file names. BricsCAD only

16.174.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.174.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bmexternalizeillegalsymbols-system-variable>

16.175 System Variable Entry: BMFORMTEMPLATEPATH

16.175.1 Name

BMFORMTEMPLATEPATH

16.175.2 Type

string

16.175.3 Default

varies by user preference

16.175.4 Read-Only

no

16.175.5 Scope

user preference; saved in user preference

16.175.6 Description

BMFORM template path: The file path and name of the default BMFORM command Template file. BricsCAD only

16.175.7 Compatibility

BricsCAD only. AutoCAD: not present; BricsCAD V20+.

16.175.8 Source Notes

- BricsCAD: <https://help.bricsys.com/en-us/document/system-variable-reference/b/bmformtemplatepath-system-variable>